

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG**



BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH

**Phát triển hệ thống quản lý tài liệu bảo mật
dựa trên kiến trúc Microservices và quy trình
DevSecOps**

Môn học NT114 - Đồ án chuyên ngành
Giảng viên hướng dẫn: ThS. Lê Anh Tuấn

Thành viên	MSSV
Huỳnh Lê Đại Thắng	23521422
Nguyễn Trường Duy	23520380

TP. Hồ Chí Minh, tháng 06 năm 2026

Lời cảm ơn

Trong quá trình thực hiện đồ án chuyên ngành với đề tài “Phát triển hệ thống quản lý tài liệu bảo mật dựa trên kiến trúc Microservices và quy trình DevSecOps”, nhóm đã có dịp đưa kiến thức về phát triển phần mềm, kiến trúc microservices, bảo mật ứng dụng và tự động hóa triển khai vào một hệ thống tương đối hoàn chỉnh. Quá trình này có không ít khó khăn, từ giới hạn tài nguyên và chi phí cho đến các vấn đề phát sinh khi triển khai và vận hành hệ thống. Vượt qua những trở ngại đó là một phần đáng giá của trải nghiệm, giúp nhóm trưởng thành hơn trong cách xây dựng và vận hành một hệ thống thực tế.

Nhóm xin gửi lời cảm ơn đến thầy ThS. Lê Anh Tuấn đã định hướng, góp ý và hỗ trợ nhóm trong quá trình xác định phạm vi, thiết kế kiến trúc và hoàn thiện sản phẩm. Những góp ý trong quá trình thực hiện giúp nhóm nhìn nhận rõ hơn mối liên hệ giữa yêu cầu nghiệp vụ, yêu cầu bảo mật và khả năng vận hành của hệ thống.

Nhóm cũng xin cảm ơn nhà trường và khoa đã tạo điều kiện để nhóm được thực hiện đồ án theo hướng thực hành, qua đó rèn luyện kỹ năng xây dựng hệ thống, kiểm thử, triển khai và trình bày kết quả kỹ thuật một cách có hệ thống. Mặc dù đã cố gắng hoàn thiện, báo cáo và sản phẩm vẫn có thể còn thiếu sót. Nhóm mong nhận được góp ý để tiếp tục cải thiện hệ thống trong các giai đoạn sau.

Tóm tắt

DocVault là một hệ thống quản lý tài liệu bảo mật được xây dựng theo kiến trúc microservices, hướng đến bài toán quản lý vòng đời tài liệu trong môi trường có yêu cầu phân quyền, kiểm soát tải xuống, lưu vết audit và hỗ trợ compliance. Hệ thống cho phép người dùng tạo tài liệu, tải lên phiên bản file, gửi duyệt, phê duyệt, xuất bản, lưu trữ và truy vết các hoạt động liên quan. Các vai trò chính gồm Viewer, Editor, Approver, Compliance Officer và Admin. Điểm nhóm muốn nhấn mạnh là quyền truy cập không được quyết định bởi một yếu tố đơn lẻ: nó là kết quả tổng hợp của RBAC, ACL, trạng thái tài liệu và mức phân loại bảo mật, và được thực thi ở backend.

Về mặt ứng dụng, DocVault gồm một frontend Next.js, một API Gateway và năm microservice NestJS lần lượt lo metadata, nội dung file, workflow, audit và notification. Xác thực dùng Keycloak; metadata nằm ở PostgreSQL, audit log ở MongoDB, còn file ở MinIO/S3. Trong các kiểm soát bảo mật, nhóm dành nhiều công nhất cho ba thứ: ranh giới Compliance Officer metadata-only, grant token ngắn hạn cho preview/download, và audit hash-chain để chuỗi sự kiện kiểm tra được tính toàn vẹn; các phần như ACL deny-overrides, malware scanning, DLP và retention evidence hỗ trợ quanh ba trục đó.

Về mặt DevSecOps, nhóm dựng một pipeline Jenkins chạy từ kiểm thử và các bước quét bảo mật (SAST, SCA, quét filesystem/image, IaC, validate Terraform) cho tới build image, cập nhật GitOps, kiểm tra Argo CD và quét động bằng OWASP ZAP sau triển khai. Cả hai hợp phần đều đã chạy được và có bằng chứng vận hành thật; phần còn lại chủ yếu là gia cố cho quy mô production, chẳng hạn chuẩn hóa provenance, bắt buộc xác minh chữ ký tại admission và diễn tập khôi phục dữ liệu.

Từ khóa: DocVault, quản lý tài liệu, bảo mật ứng dụng, microservices, DevSecOps, RBAC, ACL, audit, DLP, Jenkins, GitOps.

Abstract

DocVault is a secure document management system designed around a microservices architecture and a DevSecOps-oriented delivery process. The system addresses document lifecycle management in an environment that requires role-based access control, controlled file access, auditability, and compliance evidence. Core workflows include document creation, file upload and versioning, review submission, approval, publication, archiving, preview/download authorization, audit inspection, and evidence export. A central design point is that an access decision is never made from a single attribute: it is the combined result of role, ACL, document status, and classification, and it is always enforced at the backend rather than in the user interface.

The application layer consists of a Next.js frontend, an API Gateway, and five NestJS microservices handling metadata, file content, workflow, audit, and notification. Keycloak provides identity and access management; metadata lives in PostgreSQL, audit events in MongoDB, and file objects in MinIO/S3. Among the security controls, three received the most attention: the metadata-only boundary for the Compliance Officer, short-lived grant tokens for preview/download, and a tamper-evident audit hash chain; ACL deny-overrides, malware scanning, DLP, and retention evidence support these three.

On the DevSecOps side, a Jenkins pipeline runs from testing and security scans (SAST, SCA, filesystem and image scanning, IaC, Terraform validation) through image build, GitOps updates, Argo CD health checks, and post-deploy dynamic scanning with OWASP ZAP. Both scopes are implemented and backed by real operational evidence; the remaining work is mostly production-grade hardening, such as standardized provenance, admission-time signature enforcement, and data recovery drills.

Keywords: DocVault, document management, application security, microservices, DevSecOps, RBAC, ACL, audit, DLP, Jenkins, GitOps.

Mục lục

Danh sách hình vẽ

Danh sách bảng

Danh mục từ viết tắt

Từ viết tắt	Tiếng Anh	Diễn giải trong báo cáo
DMS	Document Management System	Hệ thống quản lý tài liệu.
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò.
ACL	Access Control List	Danh sách quyền truy cập theo người dùng, vai trò, nhóm hoặc tất cả.
OIDC	OpenID Connect	Lớp định danh dùng trên OAuth 2.0, được Keycloak sử dụng cho đăng nhập.
JWT	JSON Web Token	Token mang thông tin định danh và vai trò của người dùng.
CI/CD	Continuous Integration / Continuous Delivery	Tích hợp, kiểm thử, đóng gói và triển khai tự động.
DevSecOps	Development, Security, Operations	Tích hợp bảo mật vào quy trình phát triển và vận hành.
SAST	Static Application Security Testing	Kiểm thử bảo mật bằng phân tích mã nguồn tĩnh.
SCA	Software Composition Analysis	Phân tích thành phần phụ thuộc và lỗ hổng thư viện.
DAST	Dynamic Application Security Testing	Kiểm thử bảo mật động trên ứng dụng đang chạy.
IaC	Infrastructure as Code	Mô tả hạ tầng bằng mã, ví dụ Terraform/Kubernetes manifest.
SBOM	Software Bill of Materials	Danh mục thành phần phần mềm và phụ thuộc.
DLP	Data Loss Prevention	Cơ chế phát hiện và giảm rủi ro rò rỉ dữ liệu nhạy cảm.
EKS	Elastic Kubernetes Service	Dịch vụ Kubernetes được quản lý trên AWS.
GitOps	Git Operations	Mô hình dùng Git làm nguồn sự thật cho trạng thái triển khai.
API	Application Programming Interface	Giao diện lập trình ứng dụng giữa frontend, gateway và các service.
AWS	Amazon Web Services	Nền tảng điện toán đám mây dùng để triển khai hạ tầng đề tài.
ERD	Entity Relationship Diagram	Sơ đồ quan hệ thực thể mô tả dữ liệu và ranh giới sở hữu.
VPC	Virtual Private Cloud	Mạng riêng ảo trên AWS chứa các subnet của cụm EKS.

NAT	Network Address Translation	Cổng cho phép node trong private subnet truy cập Internet ra ngoài.
CIDR	Classless Inter-Domain Routing	Cách biểu diễn dải địa chỉ IP, dùng giới hạn truy cập Kubernetes API.
TLS	Transport Layer Security	Giao thức mã hóa kênh truyền cho lưu lượng HTTPS.
S3	Simple Storage Service	Dịch vụ lưu trữ object của AWS dùng cho file tài liệu và backup.
KMS	Key Management Service	Dịch vụ quản lý khóa mã hóa của AWS.
SSE-KMS	Server-Side Encryption with KMS	Mã hóa phía máy chủ cho object S3 bằng khóa KMS.
IRSA	IAM Roles for Service Accounts	Cấp quyền AWS cho từng Kubernetes ServiceAccount qua OIDC.
IAM	Identity and Access Management	Dịch vụ quản lý định danh và quyền truy cập của AWS.
IMDSv2	Instance Metadata Service version 2	Phiên bản metadata service yêu cầu token, giảm rủi ro SSRF trên EC2.
CVE	Common Vulnerabilities and Exposures	Định danh chuẩn cho lỗ hổng bảo mật đã được công bố.
CVSS	Common Vulnerability Scoring System	Hệ thống chấm điểm mức độ nghiêm trọng của lỗ hổng.
SLSA	Supply-chain Levels for Software Artifacts	Khung chuẩn hóa provenance của artifact trong chuỗi cung ứng.
RPO	Recovery Point Objective	Mức dữ liệu tối đa chấp nhận mất khi khôi phục sau sự cố.
RTO	Recovery Time Objective	Thời gian tối đa chấp nhận để khôi phục dịch vụ sau sự cố.
MVP	Minimum Viable Product	Sản phẩm khả dụng tối thiểu đủ để minh họa nghiệp vụ cốt lõi.

Chương 1

Tổng quan đề tài

1.1 Bối cảnh

Tài liệu trong một tổ chức không chỉ là file để lưu và tìm lại. Nó có vòng đời: được tạo, chỉnh sửa, gửi duyệt, xuất bản rồi lưu trữ; và ở mỗi bước, việc ai được xem, ai được tải, ai được phê duyệt đều cần có ranh giới rõ ràng. Tài liệu cũng có thể là hợp đồng, báo cáo tài chính, hồ sơ nhân sự hay biên bản phê duyệt, tức là loại dữ liệu mà mọi thao tác lên nó nên để lại dấu vết để sau này còn truy lại được.

Các hệ thống quản lý tài liệu phổ biến đã làm tốt việc lưu trữ, phân quyền cơ bản và tìm kiếm. Nhóm chọn đề tài DocVault không nhằm dựng lại một hệ thống đầy đủ tính năng như vậy, mà muốn đi sâu hơn vào những khía cạnh nhóm thấy đáng học và thường ít được đặt làm trọng tâm: quản lý vòng đời tài liệu một cách có kiểm soát, kiểm soát truy cập nhiều lớp theo mức nhạy cảm của tài liệu, ghi nhận audit log có thể truy vết và kiểm tra tính toàn vẹn, và đưa toàn bộ quy trình phát hành vào một pipeline DevSecOps có bằng chứng. Nói cách khác, mục tiêu của nhóm thiên về chiều sâu bảo mật, khả năng truy vết và quy trình, hơn là bề rộng tính năng.

Một nguyên tắc nhóm đặt ra từ đầu là chính sách bảo mật phải được thực thi ở backend. Giao diện có thể ẩn hoặc làm mờ thao tác để gọn gàng hơn, nhưng quyết định cho phép hay từ chối phải nằm ở phía service và được kiểm thử tại đó, vì với tài liệu nhạy cảm thì một thao tác bị bỏ lọt ở backend mới là rủi ro thật, còn việc ẩn nút trên màn hình chỉ là cải thiện trải nghiệm.

Mặt thứ hai của đề tài là quy trình phát hành. Nhóm chọn DevSecOps không phải vì đó là từ khóa thời thượng, mà vì nếu kiểm soát bảo mật chỉ được kiểm tra một lần ở cuối thì rất khó lặp lại và dễ bỏ sót khi code thay đổi. DevSecOps đưa các bước kiểm tra vào sớm và rải đều khắp vòng đời: mã nguồn, thư viện phụ thuộc, cấu hình hạ tầng, Docker image, rồi đến ứng dụng đang chạy sau khi triển khai. NIST SSDF và hướng dẫn DevSecOps của OWASP cũng đi theo tư tưởng này: tự động hóa kiểm thử và thu thập bằng chứng để giảm rủi ro và làm cho mỗi lần phát hành có thể kiểm chứng lại được [1],

[2].

DocVault vì vậy được đặt ra với hai phần gắn vào nhau: một hệ thống quản lý tài liệu thực thi chính sách bảo mật ở backend, và một quy trình DevSecOps lo việc kiểm thử, đóng gói và triển khai hệ thống đó một cách có dấu vết.

1.2 Vấn đề cần giải quyết

Đề tài tách thành hai nhóm vấn đề. Nhóm ứng dụng là xây dựng một hệ thống quản lý tài liệu có phân quyền rõ, có quy trình phê duyệt, lưu file an toàn và có audit trail đủ để truy vết. Nhóm quy trình là đưa kiểm soát bảo mật vào pipeline, sao cho mỗi lần đổi mã nguồn đều được kiểm tra tự động thay vì phụ thuộc vào việc nhớ chạy thủ công. Hai nhóm này phải làm song song, vì một hệ thống bảo mật tốt mà phát hành thủ công thì vẫn dễ trượt ở khâu triển khai.

Cụ thể hơn, nhóm rút ra các yêu cầu sau khi phân tích những câu hỏi nêu ở phần bối cảnh:

- Người dùng thuộc các vai trò khác nhau phải nhìn thấy và thao tác trên tài liệu theo đúng phạm vi quyền hạn.
- Tài liệu hỗ trợ bốn mức phân loại: public, internal, confidential và secret.
- Luồng nghiệp vụ phải hỗ trợ tạo bản nháp, tải file, gửi duyệt, phê duyệt, từ chối và lưu trữ.
- File nhạy cảm không nên được cấp URL tải trực tiếp nếu cần đi qua đường stream có kiểm soát.
- Compliance Officer được xem metadata, audit và bằng chứng tuân thủ, nhưng không được truy cập nội dung file.
- Các thao tác quan trọng phải được audit và có khả năng kiểm tra tính toàn vẹn.
- Pipeline phải có các bước kiểm thử và quét bảo mật trước, trong và sau khi build/deploy.

Trong số này, ranh giới của Compliance Officer là yêu cầu nhóm cân nhắc nhiều nhất: một người có quyền đọc toàn bộ audit log rất dễ được mặc định cho luôn quyền đọc file, nhưng đó lại chính là lỗ hổng mà vai trò compliance cần tránh.

1.3 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng và trình bày một hệ thống quản lý tài liệu bảo mật tên DocVault theo hướng microservices, có tích hợp các kiểm soát bảo mật ở cả runtime và quy trình DevSecOps.

Các mục tiêu cụ thể gồm:

1. Thiết kế kiến trúc tổng thể cho hệ thống quản lý tài liệu bảo mật, gồm frontend, API Gateway, các microservice nghiệp vụ và các thành phần hạ tầng.
2. Xây dựng các chức năng chính của hệ thống: đăng nhập, quản lý tài liệu, version file, workflow phê duyệt, phân quyền, preview/download, audit, security dashboard, evidence và retention.
3. Thiết kế mô hình phân quyền dựa trên RBAC, ACL, trạng thái tài liệu, phân loại bảo mật và quy tắc deny-overrides.
4. Xây dựng cơ chế audit hash-chain và các bằng chứng phục vụ compliance.
5. Tích hợp các kiểm soát bảo mật như malware scanning, DLP, grant token ngắn hạn và kiểm soát truy cập nội dung file.
6. Xây dựng pipeline DevSecOps với Jenkins, SonarQube, Dependency Check, Trivy, Checkov, Terraform validation, Docker build/scan, GitOps, Argo CD, smoke test và OWASP ZAP.
7. Kiểm thử và đánh giá hệ thống dựa trên các kịch bản runtime, security evidence và pipeline evidence.

1.4 Phạm vi thực hiện

Phạm vi đề tài gồm hai hợp phần có quan hệ chặt chẽ. Hợp phần ứng dụng bao gồm giao diện web, các dịch vụ backend, luồng nghiệp vụ tài liệu, kiểm soát truy cập, audit, DLP, quét mã độc, retention và evidence center. Hợp phần DevSecOps bao gồm hạ tầng AWS EKS được quản lý bằng Terraform, quy trình CI/CD trên Jenkins, kho ảnh Harbor, GitOps với Argo CD, các cổng kiểm soát bảo mật và cơ chế giám sát sau triển khai.

Tại thời điểm hoàn thành báo cáo, pipeline đã tích hợp SAST, SCA, quét secret, Trivy, Checkov, Terraform Validate, Kyverno CLI, build và quét container image, đẩy artifact lên Harbor, cập nhật GitOps, kiểm tra trạng thái Argo CD, smoke test và DAST bằng OWASP ZAP. Hạ tầng triển khai đã sử dụng Amazon S3 với SSE-KMS, IAM Role for Service Account (IRSA), AWS Secrets Manager kết hợp External Secrets Operator, backup cơ sở dữ liệu, Prometheus, Grafana và Loki. Harbor đã lưu thông tin SBOM và trạng thái ký của artifact. Các nội dung còn lại tập trung vào gia cố vận hành ở quy mô production, thay vì bổ sung các thành phần nền tảng của quy trình.

Bảng 1.1: Phạm vi và trạng thái thực hiện của đề tài

Nhóm nội dung	Mô tả	Trạng thái
Ứng dụng web	Frontend Next.js, các màn hình dashboard, documents, approvals, audit, security, evidence, retention.	Đã triển khai
Backend microservices	Gateway, metadata-service, document-service, workflow-service, audit-service, notification-service.	Đã triển khai
Bảo mật runtime	RBAC, ACL, grant token, compliance boundary, DLP, malware scanning, audit hash-chain.	Đã triển khai
Local infrastructure	Postgres, MongoDB, MinIO, Keycloak qua Docker Compose.	Đã triển khai
DevSecOps pipeline	Jenkins pipeline với các stage kiểm thử, quét bảo mật, build, Harbor, GitOps, Argo CD, smoke test và DAST.	Đã triển khai
Hạ tầng cloud-native	Terraform, EKS, S3/KMS, IRSA, External Secrets, backup, Harbor và observability.	Đã triển khai
Gia cố vận hành production	Remote Terraform state, private node, admission enforcement, provenance chuẩn hóa, NetworkPolicy giới hạn egress, log persistence và diễn tập khôi phục.	Đang hoàn thiện

1.5 Đóng góp của đề tài

Đề tài có bốn đóng góp. Thứ nhất là một hệ thống quản lý tài liệu chạy được, bao phủ luồng từ khởi tạo, tải lên, phê duyệt đến lưu trữ. Thứ hai là mô hình kiểm soát truy cập nhiều lớp, kết hợp vai trò, ACL, mức phân loại, trạng thái và quyền sở hữu thay vì chỉ dựa vào một trong số đó. Thứ ba là cách đưa yêu cầu compliance vào ngay trong thiết kế: audit hash-chain để phát hiện sửa đổi, retention evidence và evidence packet metadata-only. Thứ tư, và cũng là phần nhóm đầu tư nhiều công nhất, là một quy trình DevSecOps truy vết được từ mã nguồn qua container artifact và GitOps revision đến workload trên EKS, với các cổng kiểm soát chất lượng, bảo mật chuỗi cung ứng, quản lý bí mật và giám sát vận hành gắn vào từng chặng.

1.6 Phân công công việc

Đề tài do hai thành viên cùng thực hiện theo hình thức đồng phụ trách: cả hai cùng tham gia khảo sát, thiết kế và kiểm thử toàn hệ thống, đồng thời mỗi người nhận vai trò chính ở một số hạng mục để bảo đảm tiến độ. Bảng 1.2 tóm tắt phân công theo vai trò chính và vai trò phối hợp.

Bảng 1.2: Phân công công việc giữa các thành viên

Hạng mục	Phụ trách chính	Phối hợp
Thiết kế kiến trúc tổng thể	Cả hai	—
Frontend Next.js và permission-aware UI	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Backend microservices và bảo mật runtime	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Mô hình phân quyền (RBAC, ACL, classification)	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Audit hash-chain, DLP và evidence	Cả hai	—
Pipeline DevSecOps (Jenkins, các security gate)	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Hạ tầng cloud-native (Terraform, EKS, GitOps)	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Quản lý secret, registry và observability	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Kiểm thử, thu thập bằng chứng và viết báo cáo	Cả hai	—

1.7 Cấu trúc báo cáo

Báo cáo gồm tám chương chính. Chương 1 trình bày bối cảnh, mục tiêu và phạm vi đề tài. Chương 2 trình bày cơ sở lý thuyết và công nghệ nền tảng. Chương 3 phân tích yêu cầu và thiết kế tổng thể. Chương 4 mô tả thiết kế dữ liệu, API và chính sách bảo mật. Chương 5 trình bày quá trình triển khai ứng dụng DocVault. Chương 6 trình bày quy trình DevSecOps và hạ tầng triển khai. Chương 7 mô tả kiểm thử, đánh giá và bằng chứng. Chương 8 tổng kết kết quả đạt được, hạn chế và hướng phát triển.

Chương 2

Cơ sở lý thuyết và công nghệ nền tảng

2.1 Hệ thống quản lý tài liệu bảo mật

Hệ thống quản lý tài liệu (Document Management System - DMS) lo việc lưu trữ, tổ chức, tìm kiếm, chia sẻ và kiểm soát vòng đời tài liệu; trong môi trường doanh nghiệp nó quản lý cả file lẫn metadata, phiên bản, người sở hữu, trạng thái phê duyệt và lịch sử thao tác. Phần lý thuyết này không nhắc lại định nghĩa đó dài dòng, mà tập trung vào điều khiến một DMS trở nên *bảo mật*.

Khác biệt nằm ở chỗ thực thi chính sách. Một tài liệu nhạy cảm không thể chỉ được bảo vệ bằng nơi lưu trữ; nó phải được bảo vệ bằng một chính sách truy cập nhất quán ở backend, đặc biệt với các thao tác như preview, stream, presign download, export evidence hay truy vấn audit. Rủi ro cần tránh là tình huống frontend chỉ ẩn nút thao tác trong khi backend vẫn nhận lời gọi API, vì khi đó chính sách bảo mật không còn đáng tin.

Vì lý do đó, DocVault đặt backend làm nơi ra quyết định cuối cùng. Frontend được phép hiển thị hoặc làm mờ thao tác để trải nghiệm tốt hơn, nhưng việc cho phép hay từ chối luôn thuộc về metadata-service, document-service, gateway và audit-service. Nguyên tắc này chi phối gần như mọi thiết kế ở các chương sau.

2.2 Khảo sát các hệ thống tương tự

Trên thị trường đã có nhiều hệ thống quản lý tài liệu, từ giải pháp thương mại đến mã nguồn mở. Việc đối chiếu với chúng giúp định vị rõ hơn hướng đi mà DocVault chọn. Bảng 2.1 so sánh DocVault với ba đại diện tiêu biểu trên các tiêu chí liên quan trực tiếp đến phạm vi đề tài; phần so sánh này dựa trên tài liệu và đặc điểm công khai của các sản phẩm chứ không phải kết quả kiểm thử trực tiếp của nhóm, nên chỉ mang tính định vị tương đối.

- **Microsoft SharePoint:** nền tảng quản lý tài liệu và cộng tác thương mại, mạnh về tích hợp hệ sinh thái Microsoft 365, phân quyền và compliance; tuy nhiên là dịch vụ đóng, chủ yếu vận hành trên đám mây của nhà cung cấp và khó tùy biến sâu chính sách truy cập ở mức mã nguồn.
- **Alfresco:** hệ thống ECM mã nguồn mở (có bản thương mại), hỗ trợ vòng đời tài liệu, phân quyền và audit; kiến trúc nghiêng về monolith/Java truyền thống và không đặt trọng tâm vào quy trình DevSecOps đi kèm.
- **Nextcloud:** giải pháp self-hosted phổ biến cho lưu trữ và chia sẻ file, dễ triển khai; mạnh về chia sẻ và đồng bộ nhưng kiểm soát theo classification, DLP và audit tamper-evident không phải trọng tâm mặc định.

Bảng 2.1: So sánh DocVault với một số hệ thống quản lý tài liệu tiêu biểu

Tiêu chí	SharePoint	Alfresco	Nextcloud	DocVault
Mô hình triển khai	Cloud đóng	Self-hosted	Self-hosted	Self-hosted
RBAC + ACL deny- overrides	Có	Có	Một phần	Có
Phân loại + kiểm soát theo classification	Có	Một phần	Hạn chế	Có
DLP nâng classification tự động	Một phần	Hạn chế	Hạn chế	Có
Audit tamper-evident (hash-chain)	Hạn chế	Hạn chế	Hạn chế	Có
Compliance boundary (metadata-only)	Hạn chế	Hạn chế	Hạn chế	Có
DevSecOps pipeline đi kèm	Không	Không	Không	Có

Điểm khác biệt của DocVault không nằm ở việc thay thế các hệ thống trên về quy mô tính năng, mà ở cách gộp ba yếu tố trong cùng một đề tài: (1) kiểm soát truy cập nhiều lớp theo classification với ranh giới compliance metadata-only, (2) audit tamper-evident bằng hash-chain, và (3) đưa toàn bộ quy trình phát hành vào pipeline DevSecOps có bằng chứng. Hướng tiếp cận này bám sát mục tiêu học thuật của đề tài: minh họa cách bảo mật được đưa xuyên suốt từ thiết kế ứng dụng đến quy trình triển khai, thay vì chỉ dựng một DMS đầy đủ tính năng.

2.3 Kiến trúc microservices

Microservices chia hệ thống thành nhiều service nhỏ, mỗi service sở hữu một năng lực nghiệp vụ và phát triển, kiểm thử, triển khai tương đối độc lập. DocVault dùng cách tách

này để phân định các bounded context:

- Gateway chịu trách nhiệm xác thực, routing và truyền ngữ cảnh.
- Metadata-service sở hữu metadata, ACL, trạng thái và policy.
- Document-service sở hữu upload/download file và tích hợp object storage.
- Workflow-service sở hữu state machine phê duyệt.
- Audit-service sở hữu audit trail và hash-chain.
- Notification-service nhận thông báo nghiệp vụ trong môi trường demo/dev.

Lý do tách quan trọng nhất với một hệ thống bảo mật là kéo lưu trữ file ra khỏi nơi quyết định quyền. File thật nằm ở MinIO/S3, còn quyền truy cập nằm ở metadata-service. Khi người dùng muốn tải file, document-service không tự ý quyết định mà phải dựa vào policy hoặc grant token do metadata-service cấp và xác minh lại, nên không service nào một mình nắm trọn cả file lẫn quyền.

2.4 Xác thực và phân quyền

2.4.1 Keycloak, OIDC và JWT

Keycloak là hệ thống quản lý định danh và truy cập mã nguồn mở, hỗ trợ các chuẩn phổ biến như OAuth 2.0 và OpenID Connect [3]. Trong DocVault, Keycloak được sử dụng để quản lý người dùng, vai trò, nhóm và phát hành token. Sau khi đăng nhập, người dùng nhận JWT chứa thông tin định danh và các claim cần thiết như user id, username, roles và groups.

Gateway và các service phía sau dùng JWT để xác định actor của request. Gateway là entry point chính, xác thực token và truyền các header ngữ cảnh như `x-user-id`, `x-roles`, `x-groups` xuống các service nội bộ. Nhờ đó service nghiệp vụ không phụ thuộc trực tiếp vào phiên đăng nhập frontend mà vẫn đủ thông tin để thực thi chính sách.

2.4.2 RBAC

RBAC gán quyền cho vai trò thay vì gán trực tiếp cho từng người dùng, rồi người dùng nhận một hoặc nhiều vai trò. Trong DocVault có năm vai trò:

- **viewer**: đọc tài liệu đã xuất bản nếu policy cho phép.
- **editor**: tạo tài liệu, chỉnh sửa metadata, upload file và gửi duyệt.
- **approver**: xem và phê duyệt hoặc từ chối tài liệu đang chờ duyệt.
- **compliance_officer**: xem audit, metadata và evidence trong phạm vi compliance, nhưng không được truy cập nội dung file.
- **admin**: quản trị hệ thống và có quyền rộng nhất.

RBAC đủ để kiểm soát quyền ở mức route hay chức năng, nhưng nó dừng lại ở mức vai trò: nó không nói được rằng riêng tài liệu này thì người dùng A được tải còn vai trò editor nói chung thì không. Đó là lý do DocVault bổ sung ACL ở mức từng tài liệu.

2.4.3 ACL và deny-overrides

ACL gán rule truy cập cho từng tài liệu. Một rule áp cho user, role, group hoặc tất cả, mang permission như `READ`, `DOWNLOAD`, `WRITE`, `APPROVE` và effect `ALLOW` hoặc `DENY`.

Khi nhiều rule cùng khớp, DocVault chọn chiến lược deny-overrides: chỉ cần có một rule `DENY` phù hợp thì kết quả là từ chối, kể cả khi tồn tại rule `ALLOW` khác. Nhóm chọn hướng này thay vì allow-overrides vì với tài liệu `SECRET`, một lệnh cấm bị ghi đè nguy hiểm hơn nhiều so với một quyền cho phép bị bỏ sót; khi không chắc, hệ thống nên nghiêng về từ chối.

2.5 Phân loại tài liệu và kiểm soát nội dung

DocVault sử dụng bốn mức phân loại tài liệu:

- **PUBLIC**: tài liệu công khai trong phạm vi người dùng đã xác thực.
- **INTERNAL**: tài liệu nội bộ.
- **CONFIDENTIAL**: tài liệu mật, cần kiểm soát preview/download nghiêm ngặt hơn.
- **SECRET**: tài liệu rất nhạy cảm, chỉ một số vai trò/đối tượng được phép truy cập.

Mức phân loại không dừng ở metadata hiển thị mà còn ảnh hưởng đến chính sách truy cập. Chẳng hạn, tài liệu `CONFIDENTIAL` hoặc `SECRET` có thể không trả về presigned URL trực tiếp mà phải đi qua endpoint stream, để document-service áp thêm kiểm soát như watermark posture và tái xác thực grant.

2.6 Audit, hash-chain và compliance evidence

Audit log ghi lại các sự kiện quan trọng trong hệ thống: tạo tài liệu, upload file, submit, approve, reject, download authorized/denied, DLP detected, malware blocked, retention archive và các thao tác compliance. Một audit log hữu ích thì ngoài việc có dữ liệu còn phải cho phép kiểm tra tính toàn vẹn.

DocVault dùng hash-chain cho audit events. Mỗi event mang một hash phụ thuộc vào nội dung của chính nó và hash của event trước. Nếu ai đó sửa thẳng một event cũ trong storage, các hash phía sau sẽ lệch và verify-chain phát hiện được. Cần nói rõ giới hạn: cơ chế này không biến audit log thành lưu trữ bất biến tuyệt đối, nó chỉ làm cho việc sửa đổi để lại dấu vết, tức tamper-evident, và mức đó là đủ cho phạm vi đồ án.

Ngoài audit query, hệ thống còn xuất evidence packet gom metadata, version checksum, workflow history, retention evidence, trạng thái verify-chain và các audit event liên quan đến một tài liệu. Một chi tiết được giữ rất kỹ: evidence packet không bao giờ chứa nội dung file, presigned URL, grant token hay object key nhạy cảm.

2.7 DevSecOps

DevSecOps đưa bảo mật vào trong quá trình phát triển và vận hành thay vì để dành kiểm tra ở cuối dự án. Ý tưởng là rải các kiểm soát ra nhiều điểm của pipeline: mã nguồn, thư viện phụ thuộc, hạ tầng, Docker image, ứng dụng sau triển khai, và giữ lại artifact làm bằng chứng. Cái lợi không chỉ là phát hiện sớm mà còn là tính lặp lại: mỗi lần thay đổi đều đi qua cùng một bộ kiểm tra.

Ở DocVault, tinh thần đó được hiện thực bằng Jenkins pipeline. Ngoài chạy test, pipeline còn chạy SAST, SCA, Trivy filesystem scan, Trivy image scan, Checkov IaC scan, Terraform validate, smoke test và DAST bằng OWASP ZAP. Các bước này bám theo tư tưởng “shift-left security” trong các tài liệu DevSecOps [1], [2].

2.8 Các nhóm kiểm thử bảo mật trong pipeline

Bảng 2.2: Các nhóm kiểm thử bảo mật trong DevSecOps pipeline

Nhóm	Mục tiêu	Ví dụ áp dụng trong DocVault	Công cụ
SAST	Phân tích mã nguồn tĩnh để phát hiện lỗi chất lượng và bảo mật.	Kiểm tra backend/frontend TypeScript, quality gate và code smell.	SonarQube
SCA	Phân tích thư viện phụ thuộc và lỗ hổng đã biết.	Quét dependency của monorepo Node.js.	OWASP Dependency Check
FS scan	Quét filesystem repository.	Phát hiện lỗ hổng trong file hệ thống, lockfile, cấu hình.	Trivy
Image scan	Quét Docker image sau build.	Phát hiện CVE trong image backend/frontend.	Trivy
IaC scan	Quét cấu hình hạ tầng.	Kiểm tra Terraform/Kubernetes manifest.	Checkov
DAST	Kiểm thử ứng dụng đang chạy.	Baseline scan web app sau deploy.	OWASP ZAP

2.9 GitOps và Argo CD

GitOps lấy Git làm nguồn sự thật cho trạng thái triển khai. Khi pipeline build xong image mới, thay vì chạy lệnh sửa cluster thủ công, nó cập nhật Helm values hoặc manifest trong một branch GitOps; Argo CD theo dõi branch đó và đồng bộ trạng thái mong muốn xuống Kubernetes [4]. Khác biệt thực tế là không ai `kubectl apply` trực tiếp lên cụm, mọi thay đổi triển khai đều đi qua một commit.

DocVault tận dụng đúng điểm này. Pipeline cập nhật các file values theo từng service, rồi Argo CD sync các application gateway, metadata-service, document-service, workflow-service, audit-service, notification-service và web. Việc tách build pipeline khỏi deployment controller giúp mỗi bên giữ một trách nhiệm, đồng thời để lại lịch sử thay đổi triển khai ngay trong Git.

2.10 Tổng hợp công nghệ và công cụ

Để có cái nhìn tổng thể, phần này tổng hợp các công nghệ và công cụ chính được sử dụng trong đề tài, chia thành hai nhóm: nhóm xây dựng ứng dụng (Bảng 2.3) và nhóm quy trình DevSecOps cùng hạ tầng triển khai (Bảng 2.4). Các phiên bản được ghi nhận theo cấu hình thực tế trong repository tại thời điểm hoàn thành báo cáo.

Bảng 2.3: Công nghệ xây dựng ứng dụng DocVault

Thành phần	Công nghệ	Phiên bản	Vai trò
Frontend	Next.js, React	16.2, 19.2	Giao diện web theo App Router.
UI & form	Tailwind CSS, Radix UI, React Hook Form, Zod	4.x, 7.7, 4.3	Thiết kế giao diện, form và validation phía client.
Server state	TanStack Query	5.90	Quản lý dữ liệu server, cache và đồng bộ.
Backend	NestJS	10.0	Framework cho gateway và các microservice.
Ngôn ngữ	TypeScript	5.x	Ngôn ngữ chính cho frontend và backend.
Xác thực	Keycloak, passport-jwt, jwks-rsa	—	OIDC/JWT, xác minh token bằng JWKS RS256.
ORM	Prisma + adapter-pg	7.5	Truy cập PostgreSQL bằng driver adapter.
Metadata DB	PostgreSQL	—	Lưu metadata, ACL, workflow history.
Audit DB	MongoDB, Mongoose	—	Lưu audit event và hash-chain.
Object storage	MinIO / Amazon S3	—	Lưu blob file tài liệu.
Rate limit	@nestjs/throttler	6.5	Giới hạn tần suất request theo cấu hình chung.
Monorepo	pnpm, Turbo	9.15, 2.1	Quản lý workspace và điều phối build.

Bảng 2.4: Công cụ DevSecOps và hạ tầng triển khai

Nhóm	Công cụ	Vai trò trong đề tài
CI/CD	Jenkins, Shared Library	Điều phối pipeline, tách CI validation và CD release.
Secret scan	TruffleHog	Phát hiện credential bị commit trong repository.
SAST	SonarQube	Phân tích mã nguồn tĩnh và Quality Gate.
SCA	OWASP Dependency-Check	Quét lỗ hổng thư viện phụ thuộc theo CVSS.
FS/Image scan	Trivy	Quét filesystem và Docker image.
IaC scan	Checkov	Quét cấu hình Terraform, Kubernetes, Helm.
Policy-as-code	Kyverno CLI	Kiểm tra policy trên manifest đã render.
IaC	Terraform	Mô tả hạ tầng AWS dưới dạng mã.
Container	Docker BuildKit	Build và cache image theo batch.
Registry	Harbor	Private registry, lưu SBOM và trạng thái ký.
Ký artifact	Cosign	Ký và xác minh image digest.
GitOps	Git, Argo CD	Đồng bộ desired state theo mô hình app-of-apps.
Orchestration	Amazon EKS, Helm, Kustomize	Chạy workload trên Kubernetes.
Secret & key	AWS Secrets Manager, External Secrets, KMS	Quản lý bí mật và khóa mã hóa (customer-managed key, xoay vòng hằng năm).
DAST	OWASP ZAP	Quét động ứng dụng sau triển khai.
Observability	Prometheus, Grafana, Loki	Thu thập metric, log và trực quan hóa.

Chương 3

Phân tích yêu cầu và thiết kế tổng thể

3.1 Tổng quan bài toán

DocVault hướng đến bài toán quản lý tài liệu trong môi trường có nhiều vai trò và nhiều mức độ nhạy cảm dữ liệu. Một tài liệu đi qua nhiều trạng thái: bản nháp, chờ duyệt, đã xuất bản và đã lưu trữ. Trong quá trình đó, mỗi vai trò chỉ được thực hiện những thao tác phù hợp. Editor có thể tạo và gửi duyệt tài liệu; Approver có thể phê duyệt hoặc từ chối; Viewer chỉ được truy cập tài liệu đã xuất bản nếu policy cho phép; Compliance Officer có thể xem audit và evidence nhưng không được truy cập nội dung file; Admin có quyền quản trị rộng hơn.

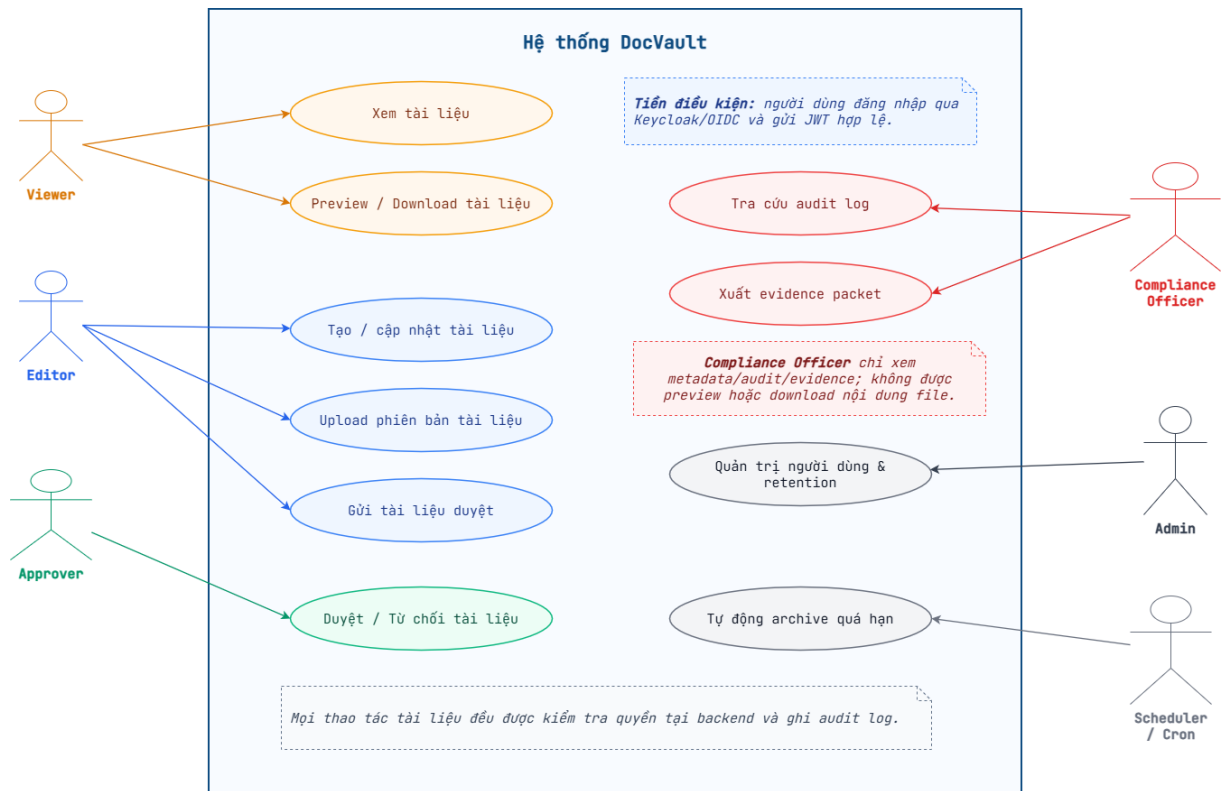
Bài toán ở đây không dừng ở việc lưu file, mà là kiểm soát file theo ngữ cảnh. Cùng một file có thể được phép preview với người này nhưng bị từ chối với người khác; có thể được cấp presigned URL khi là tài liệu public nhưng phải stream qua backend khi là tài liệu confidential; có thể cho xem metadata nhưng không cho tải nội dung. Vì vậy, hệ thống cần một mô hình policy đủ rõ và được thực thi ở backend.

3.2 Tác nhân hệ thống

Bảng 3.1: Các tác nhân chính trong DocVault

Tác nhân	Vai trò trong hệ thống
Viewer	Người dùng đọc tài liệu đã được xuất bản trong phạm vi policy cho phép; có thể preview/download tài liệu public hoặc tài liệu được cấp quyền.
Editor	Người tạo và quản lý tài liệu của mình; có thể tạo draft, chỉnh metadata, upload phiên bản, gửi duyệt và archive khi có quyền.
Approver	Người kiểm tra tài liệu đang ở trạng thái chờ duyệt; có thể approve hoặc reject theo quy trình.
Compliance Officer	Người kiểm tra tuân thủ; được xem audit, metadata/evidence được phép, nhưng bị chặn preview, stream, presign và download nội dung file.
Admin	Người quản trị hệ thống; có quyền rộng hơn trên tài liệu, thành viên, evidence và retention.
Scheduler / Cron	Tác nhân hệ thống (không phải người dùng); chạy nền theo lịch để tự động archive tài liệu đến hạn retention và dọn tài liệu trong thùng rác quá hạn lưu giữ.

Hình 3.1 tổng hợp các use case chính của từng tác nhân trong một sơ đồ thống nhất. Ngoài bốn vai trò người dùng, sơ đồ bổ sung tác nhân Scheduler/Cron đại diện cho các tác vụ nền theo lịch. Sơ đồ làm rõ ba điểm thiết kế cốt lõi của DocVault: mọi tác nhân phải đăng nhập qua Keycloak/OIDC trước khi thao tác; mọi thao tác trên tài liệu đều được kiểm tra quyền tại backend và ghi audit log; và Compliance Officer dù xem được metadata, audit, evidence vẫn luôn bị từ chối preview hoặc download nội dung file.



Hình 3.1: Sơ đồ use case tổng thể của DocVault theo tác nhân

3.3 Yêu cầu chức năng

Các yêu cầu chức năng được nhóm theo miền nghiệp vụ như sau:

3.3.1 Xác thực và phân quyền

Hệ thống cần hỗ trợ đăng nhập thông qua Keycloak, đọc thông tin người dùng hiện tại và phân quyền theo vai trò. Navigation, command palette và các thao tác trên UI cần thay đổi theo role, nhưng backend vẫn phải là nơi quyết định cuối cùng. Các request không có token, token hết hạn hoặc token không hợp lệ phải bị từ chối.

3.3.2 Quản lý tài liệu

Người dùng có quyền có thể tạo tài liệu mới với tiêu đề, mô tả, classification và tags. Sau khi tạo, tài liệu ở trạng thái DRAFT. Editor hoặc Admin có thể upload file ban đầu hoặc upload phiên bản mới. Hệ thống cần lưu checksum, kích thước, MIME type, tên file và người upload cho từng version.

3.3.3 Workflow phê duyệt

Tài liệu đi qua vòng đời chính DRAFT -> PENDING -> PUBLISHED -> ARCHIVED. Editor gửi tài liệu từ Draft sang Pending. Approver phê duyệt tài liệu Pending sang Published hoặc từ chối để quay về Draft. Tài liệu Published có thể được archive theo quyền. Mỗi transition cần ghi workflow history và audit event.

3.3.4 Preview và download

Người dùng có quyền có thể preview hoặc download tài liệu tùy trạng thái, classification, role, ownership và ACL. Với tài liệu nhạy cảm, hệ thống có thể không trả về URL tải trực tiếp mà trả về endpoint stream có kiểm soát. Compliance Officer luôn bị chặn truy cập nội dung file.

3.3.5 Audit, security và evidence

Hệ thống cần ghi audit event cho các hành động quan trọng, hỗ trợ truy vấn audit theo bộ lọc, kiểm tra hash-chain và hiển thị security summary. Compliance/Admin có thể xuất evidence packet liên quan đến tài liệu, recommendation hoặc compliance flow.

3.3.6 Retention và access review

Tài liệu đã xuất bản cần có retention evidence dựa trên classification. Admin có thể chạy retention job demo để auto-archive các tài liệu đến hạn. Compliance/Admin có thể xem access review cho tài liệu nhạy cảm, broad ACL grant hoặc quyền đã cũ.

3.4 Yêu cầu phi chức năng

Bảng 3.2: Yêu cầu phi chức năng

Nhóm yêu cầu	Mô tả
Bảo mật	Backend phải thực thi policy; token và grant phải được xác minh; nội dung file không được lộ qua evidence hoặc audit.
Truy vết	Các thao tác quan trọng cần tạo audit event; audit chain cần có khả năng verify.
Tách biệt trách nhiệm	Mỗi service sở hữu một bounded context rõ ràng; document-service không tự quyết định policy download.
Khả năng kiểm thử	Có unit test, E2E runtime, frontend test/lint/build và pipeline scan.
Khả năng triển khai	Hỗ trợ local Docker Compose, Docker image, Kubernetes/Helm và GitOps.
Khả năng mở rộng	Kiến trúc cho phép bổ sung message queue, kênh thông báo, môi trường triển khai và replica độc lập; nền tảng hiện tại đã tích hợp SBOM, ký image, KMS, External Secrets và policy-as-code trong CI.

3.5 Mô hình threat cơ bản

Để định hướng thiết kế bảo mật, hệ thống xem xét các nhóm rủi ro sau. Mỗi nhóm được phân loại theo khung STRIDE [5] để làm rõ bản chất mỗi đe dọa và liên kết với biện pháp giảm thiểu tương ứng trong DocVault.

Bảng 3.3: Mô hình threat của DocVault ánh xạ theo STRIDE

Rủi ro	Loại STRIDE	Biện pháp giảm thiểu trong DocVault
Truy cập trái phép metadata	Information Disclosure	Người dùng đoán document id và gọi API chi tiết, workflow history, comments hoặc ACL; backend policy-filter theo role, owner, ACL, classification và status.
Tải nội dung file trái phép	Elevation of Privilege	Gọi trực tiếp presign, stream hoặc download endpoint mà không có quyền; chặn bằng grant token ngắn hạn và tái xác minh ở document-service.
Lạm dụng Compliance Officer	Elevation of Privilege	Vai trò compliance xem được nhiều metadata nhưng luôn bị deny nội dung file ở mọi classification.
Upload file độc hại	Tampering	File EICAR/malware bị chặn trước khi ghi MinIO và không tạo version metadata.
Rò rỉ dữ liệu nhạy cảm	Information Disclosure	DLP phát hiện email, số điện thoại, từ khóa nhạy cảm trong tài liệu public/internal và nâng classification.
Downgrade classification không hợp lệ	Tampering	Tài liệu đã bị DLP phát hiện không được hạ xuống public/internal nếu thiếu quyền và lý do override hợp lệ.
Giả mạo audit event	Spoofing	Người dùng thường không append audit trực tiếp; audit ingest yêu cầu service token.
Sửa audit log cũ	Tampering	Hash-chain phát hiện khi audit event bị thay đổi trực tiếp trong storage.

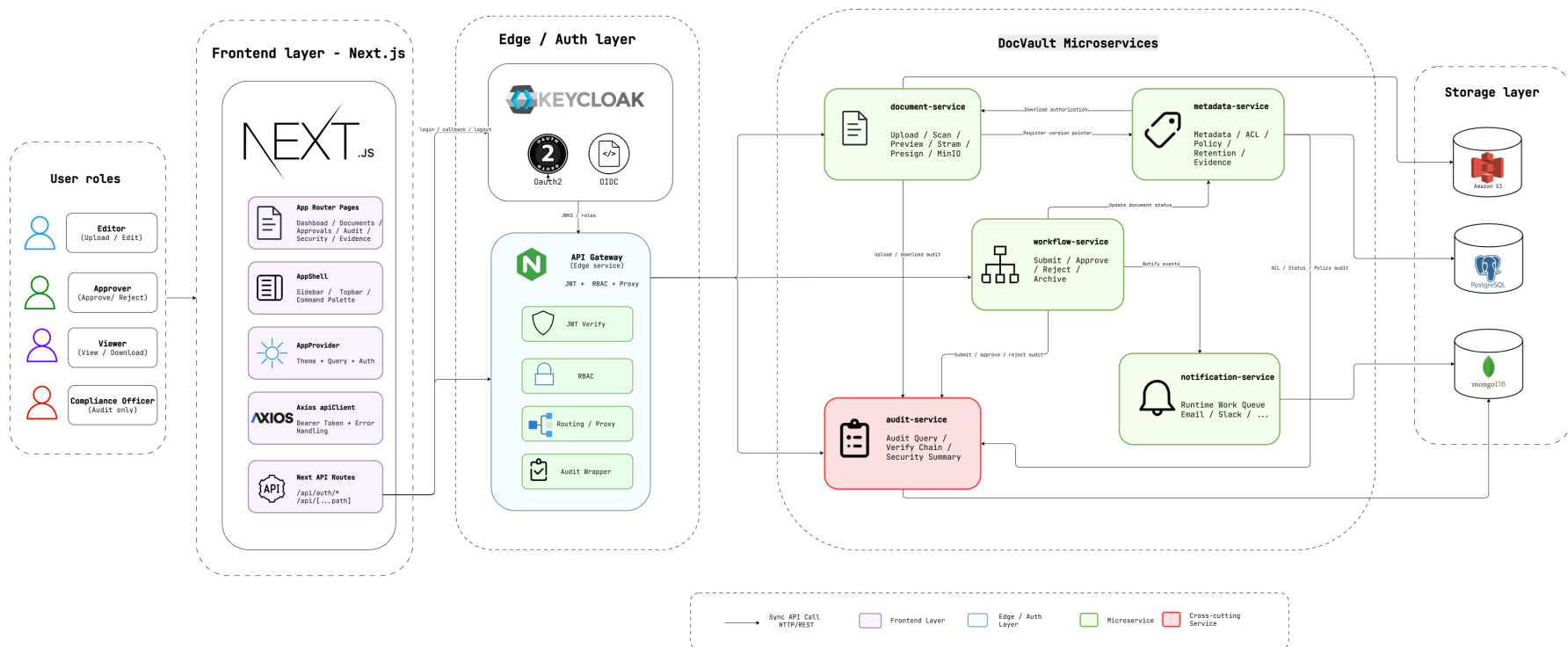
Phần lớn rủi ro tập trung vào hai loại Information Disclosure và Elevation of Privilege, phản ánh đúng đặc thù của một hệ thống quản lý tài liệu nhạy cảm: mối đe dọa chính là lộ dữ liệu và vượt quyền truy cập. Hai loại Tampering và Spoofing liên quan đến tính toàn vẹn của file và audit trail. Các nhóm Repudiation và Denial of Service không phải trọng tâm trong phạm vi đồ án; tính chống chối bỏ được hỗ trợ một phần qua audit hash-chain, còn chống từ chối dịch vụ được xử lý ở mức cơ bản bằng rate limiting.

3.6 Kiến trúc tổng thể

Kiến trúc DocVault gồm bốn lớp: frontend, gateway, microservices và hạ tầng dữ liệu. Frontend Next.js là giao diện người dùng. API Gateway là điểm vào duy nhất cho frontend, chịu trách nhiệm xác thực, role guard, proxy request và truyền ngữ cảnh. Các service phía sau xử lý nghiệp vụ riêng. Hạ tầng dữ liệu gồm Keycloak, PostgreSQL, MongoDB

và MinIO/S3.

Hình ?? mô tả cách các lớp này liên kết với nhau. Trong kiến trúc đó, frontend chỉ giao tiếp qua gateway, còn các microservice phía sau giữ ranh giới nghiệp vụ riêng: metadata-service sở hữu policy và metadata, document-service sở hữu blob file, workflow-service điều phối trạng thái, audit-service ghi nhận sự kiện và notification-service xử lý thông báo.



Hình 3.2: Kiến trúc tổng thể ứng dụng DocVault

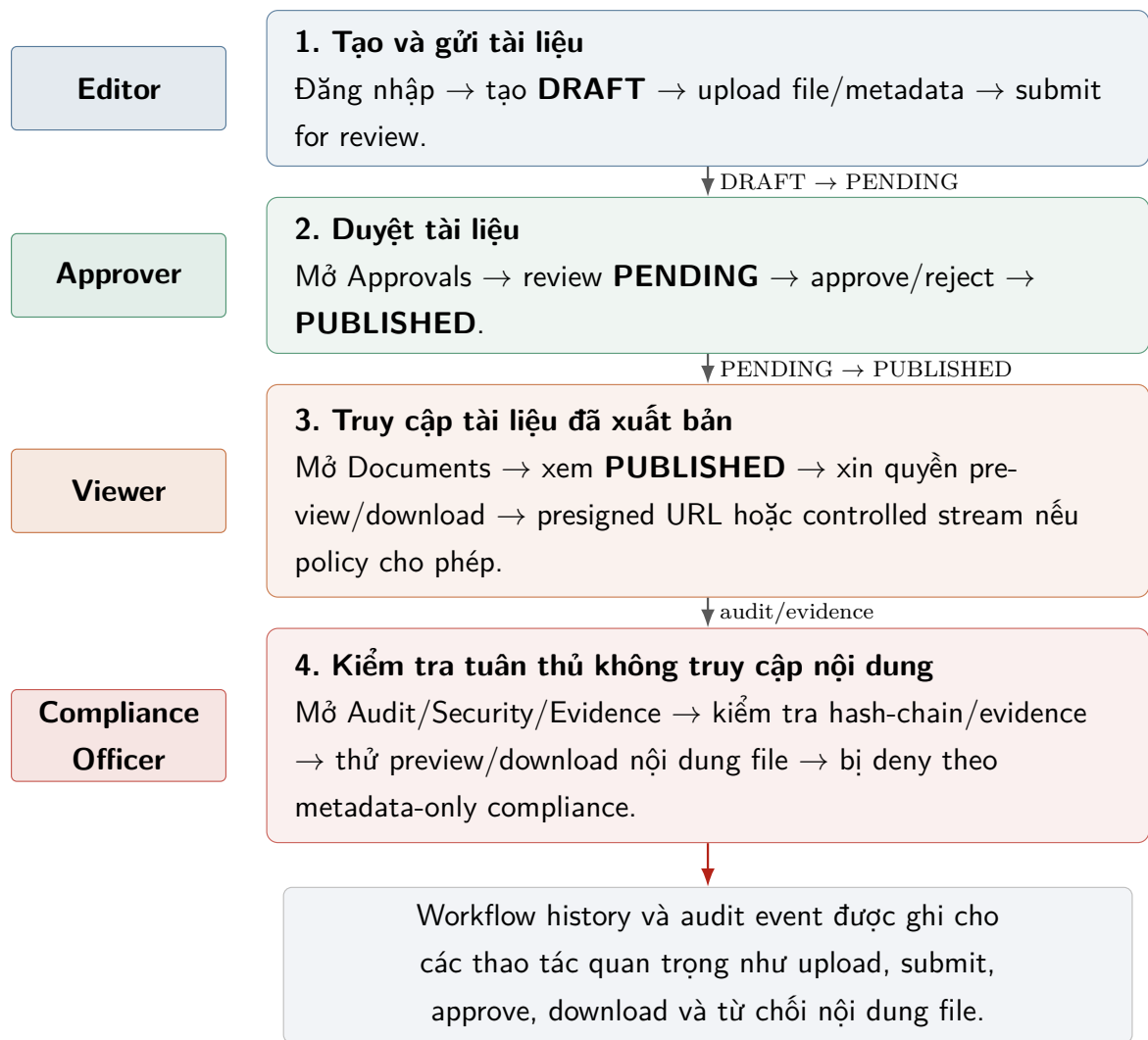
DocVault được tổ chức thành bốn lớp xếp chồng từ client đến hạ tầng dữ liệu, như minh họa ở Hình ??:

- **Lớp client và frontend:** trình duyệt người dùng tải ứng dụng Next.js. Frontend chỉ giữ một API base URL duy nhất trỏ đến gateway và không gọi trực tiếp bất kỳ microservice nào. Người dùng đăng nhập qua Keycloak theo luồng OIDC, sau đó frontend đính kèm JWT vào mỗi request gửi lên gateway.
- **Lớp gateway:** API Gateway là điểm vào duy nhất. Tại đây JWT được xác minh với Keycloak qua JWKS, role được kiểm tra ở mức route, và request hợp lệ được proxy xuống service phù hợp kèm các header ngữ cảnh (`x-user-id`, `x-roles`, `x-groups`, `x-request-id`). Gateway không chứa nghiệp vụ tài liệu.
- **Lớp microservice:** mỗi service sở hữu một bounded context riêng. Metadata-service giữ metadata, ACL, policy và cấp grant token; document-service xử lý blob file và đối tượng lưu trữ; workflow-service điều phối state machine; audit-service ghi và xác minh hash-chain; notification-service nhận thông báo nghiệp vụ. Các service giao tiếp với nhau bằng HTTP nội bộ (Axios) và được miễn rate limit qua header gọi nội bộ.
- **Lớp hạ tầng dữ liệu:** Keycloak quản lý định danh; PostgreSQL lưu metadata; MongoDB lưu audit event; MinIO/S3 lưu file. Mỗi service chỉ truy cập kho dữ liệu thuộc bounded context của nó, tránh việc một service đọc trực tiếp database của service khác.

Cách phân lớp này được thiết kế để không tồn tại đường tắt bỏ qua kiểm soát chính sách. Với một thao tác nhạy cảm như tải file, request luôn xuất phát từ frontend, đi qua gateway để xác thực, rồi mới đến metadata-service để xin phép; document-service chỉ phát file sau khi nhận được grant token hợp lệ. Lớp backend vì thế luôn giữ quyền quyết định cuối cùng về truy cập.

3.7 Luồng nghiệp vụ chính

Luồng demo chính của hệ thống gồm bốn vai trò. Editor đăng nhập, tạo tài liệu, upload file và submit. Approver đăng nhập, kiểm tra tài liệu pending và approve. Viewer đăng nhập, xem tài liệu đã published và download nếu policy cho phép. Compliance Officer đăng nhập, xem audit/evidence nhưng bị chặn preview/download nội dung file. Bản thân chuỗi trạng thái mà tài liệu đi qua trong các luồng này được mô tả chi tiết ở Mục ??.



Hình 3.3: Luồng nghiệp vụ demo chính theo vai trò trong DocVault

3.8 Nguyên tắc thiết kế

Thiết kế DocVault tuân theo các nguyên tắc sau:

1. **Backend authoritative:** frontend chỉ hỗ trợ UX; backend luôn là nơi thực thi policy.
2. **Tách metadata và blob:** metadata nằm ở PostgreSQL, file nằm ở MinIO/S3, version metadata chỉ giữ object key và checksum.
3. **Deny wins:** ACL deny có độ ưu tiên cao hơn allow.
4. **Metadata-only compliance:** compliance có thể kiểm tra hệ thống mà không cần truy cập nội dung file.
5. **Evidence without leakage:** evidence không chứa nội dung file, presigned URL, grant token hoặc object key nhạy cảm.
6. **Security as pipeline gates:** các bước scan/test cần chạy tự động trong pipeline

thay vì phụ thuộc thao tác thủ công.

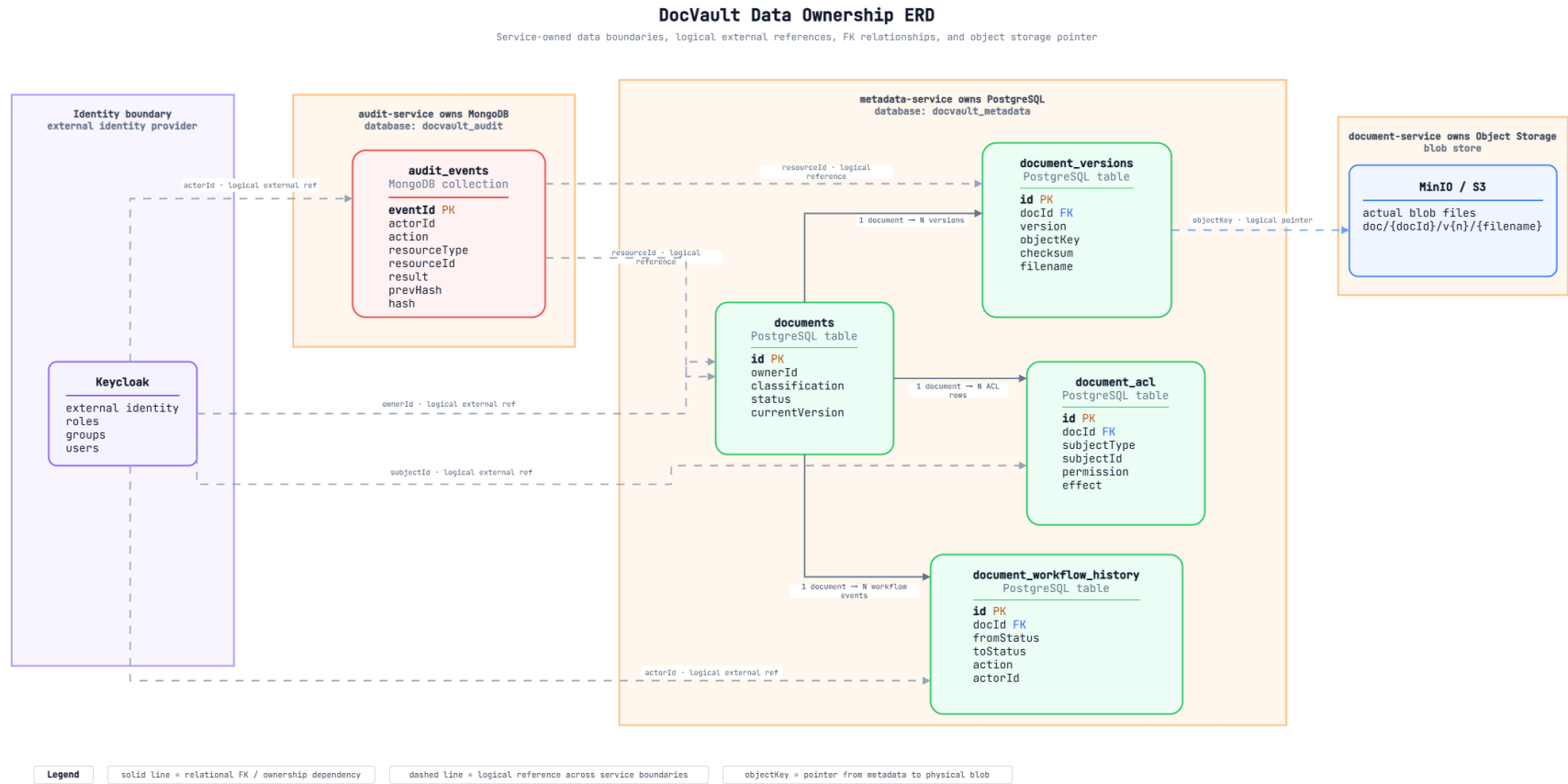
Chương 4

Thiết kế dữ liệu, API và chính sách bảo mật

4.1 Tổng quan thiết kế dữ liệu

DocVault tách dữ liệu thành ba nhóm chính. Metadata tài liệu thuộc metadata-service và lưu trong PostgreSQL. Nội dung file thuộc document-service và lưu trong MinIO/S3. Audit event thuộc audit-service và lưu trong MongoDB. Việc tách này giúp mỗi service sở hữu dữ liệu của mình, tránh việc service khác truy vấn trực tiếp vào database không thuộc bounded context của nó.

Tài liệu nội bộ docs/ERD.md mô tả rõ nguyên tắc này: metadata database lưu documents, document versions, ACL và workflow history; audit database lưu audit events; object storage lưu blob file. Nhờ tách như vậy, hệ thống truy vấn metadata nhanh, lưu được file dung lượng lớn trong object storage, đồng thời giữ audit log độc lập với nghiệp vụ tài liệu.



Hình 4.1: ERD và ranh giới sở hữu dữ liệu trong DocVault

DocVault tổ chức dữ liệu xoay quanh thực thể trung tâm `documents`, với các thực thể vệ tinh `document_versions`, `document_acl`, `document_workflow_history`, `document_comments` và `document_share_links` cùng tham chiếu đến `documents` qua `docId` theo quan hệ một-nhiều (Hình ??). Mỗi tài liệu có thể có nhiều phiên bản, nhiều rule ACL, nhiều bản ghi lịch sử workflow và nhiều comment.

Bên cạnh quan hệ thực thể, mô hình dữ liệu được phân chia theo ranh giới sở hữu tương ứng ba kho lưu trữ. Vùng PostgreSQL (thuộc `metadata-service`) gồm `documents` và các bảng vệ tinh, là nơi lưu metadata và quyền truy cập. Vùng object storage (thuộc `document-service`) lưu blob file; lưu ý `document_versions` chỉ giữ `objectKey` và `checksum` trỏ tới file thật chứ không lưu nội dung. Vùng MongoDB (thuộc `audit-service`) lưu `audit_events` và cố ý không dùng khóa ngoại cứng đến `documents`, vì audit là bounded context độc lập và phải tồn tại được kể cả khi tài liệu bị xóa. Đây cũng là lý do một thao tác như tải file phải đi qua nhiều service: metadata nằm một nơi, file nằm một nơi, và dấu vết kiểm toán nằm ở nơi thứ ba.

4.2 Các thực thể chính

4.2.1 documents

Bảng `documents` là nguồn sự thật cho tài liệu logic. Mỗi record đại diện cho một tài liệu, gồm tiêu đề, mô tả, owner, classification, tags, status, current version và các mốc thời gian như `createdAt`, `updatedAt`, `publishedAt`, `archivedAt`.

Trạng thái tài liệu gồm `DRAFT`, `PENDING`, `PUBLISHED` và `ARCHIVED`. Classification gồm `PUBLIC`, `INTERNAL`, `CONFIDENTIAL` và `SECRET`. Hai trường này là nền tảng cho phần lớn quyết định policy.

4.2.2 document_versions

Bảng `document_versions` lưu thông tin từng phiên bản file. Mỗi version có `docId`, `version number`, `object key`, `checksum`, `size`, `filename`, `contentType` và `createdBy`. File thật không nằm trong PostgreSQL; database chỉ giữ pointer đến object storage.

Checksum SHA-256 hỗ trợ kiểm tra tính nhất quán và làm evidence. Khi export evidence packet, hệ thống có thể chứng minh tài liệu có version cụ thể và checksum cụ thể mà không cần đưa nội dung file vào packet.

4.2.3 document_acl

Bảng `document_acl` lưu các rule truy cập theo tài liệu. Rule có `subjectType`, `subjectId`, `permission` và `effect`. `SubjectType` hỗ trợ `USER`, `ROLE`, `GROUP` và `ALL`. `Permission` hỗ trợ `READ`, `DOWNLOAD`, `WRITE` và `APPROVE`. `Effect` là `ALLOW` hoặc `DENY`.

ACL không thay thế RBAC mà bổ sung cho RBAC. RBAC xác định năng lực chung theo vai trò, trong khi ACL xác định ngoại lệ hoặc quyền cụ thể theo tài liệu. Khi có rule deny phù hợp, hệ thống ưu tiên từ chối.

4.2.4 document_workflow_history

Bảng `document_workflow_history` lưu lịch sử chuyển trạng thái nghiệp vụ. Mỗi record có `docId`, `fromStatus`, `toStatus`, `action`, `actorId`, `reason` và `createdAt`. Bảng này phục vụ truy vấn nhanh lịch sử phê duyệt trong màn hình chi tiết tài liệu, tách khỏi `audit-service` vốn ghi nhiều loại sự kiện hệ thống hơn.

4.2.5 audit_events

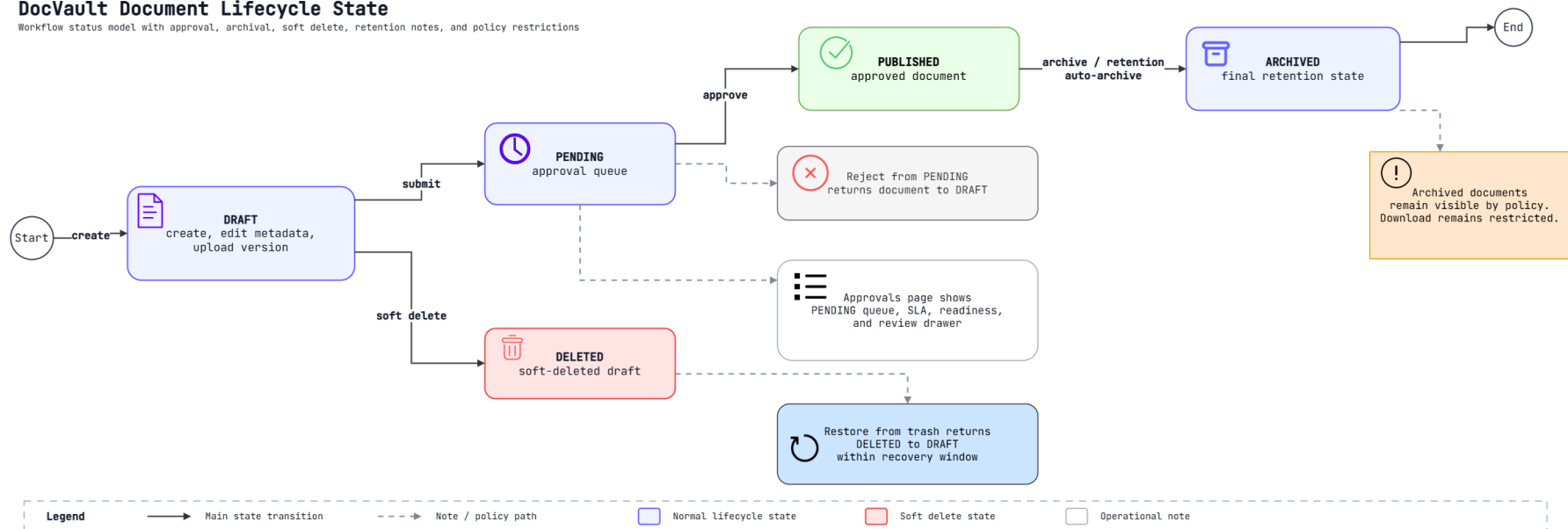
Audit event ghi lại hành động trong hệ thống. Mỗi event chứa `actor`, `action`, `resource type`, `resource id`, `result`, `reason`, `metadata` và `hash`. Audit-service áp dụng hash-chain để phát hiện sửa đổi ngoài luồng. Vì audit là bounded context riêng, audit event không dùng foreign key cứng đến documents.

4.3 Document lifecycle

Luồng trạng thái tài liệu trong DocVault được giữ đủ gọn để phù hợp với phạm vi MVP nhưng vẫn bao phủ demo doanh nghiệp. Các transition chính gồm tạo draft, gửi duyệt, phê duyệt, từ chối và lưu trữ; ngoài ra hệ thống còn mô phỏng soft delete và restore cho tài liệu nháp. Phần này đi sâu vào state machine; cách bốn vai trò tương tác với các trạng thái đó đã được trình bày ở Mục ??.

DocVault Document Lifecycle State

Workflow status model with approval, archival, soft delete, retention notes, and policy restrictions



Hình 4.2: Vòng đời trạng thái tài liệu trong DocVault

DocVault định nghĩa vòng đời tài liệu bằng một state machine với các trạng thái DRAFT, PENDING, PUBLISHED, ARCHIVED, DELETED và các transition có ràng buộc vai trò (Hình ??). Tài liệu mới luôn bắt đầu ở DRAFT. Editor thực hiện SUBMIT để chuyển sang PENDING; Approver thực hiện APPROVE để sang PUBLISHED hoặc REJECT để quay về DRAFT; tài liệu PUBLISHED có thể được ARCHIVE. Ngoài vòng đời chính, hệ thống còn hỗ trợ nhánh soft delete và restore áp dụng cho bản nháp.

Hai nguyên tắc chi phối vòng đời này. Thứ nhất, các transition là một chiều và ràng buộc theo vai trò, nên giữa hai trạng thái bất kỳ không có đường đi tùy ý; workflow-service kiểm tra transition có hợp lệ không trước khi yêu cầu metadata-service cập nhật. Thứ hai, mỗi transition sinh một bản ghi `document_workflow_history` kèm một audit event, đủ để trang chi tiết tài liệu dựng lại timeline nghiệp vụ và để compliance đối chiếu chéo với audit log. Khi tài liệu đi qua APPROVE, hệ thống đặt `publishedAt`; khi đi qua ARCHIVE, hệ thống đặt `archivedAt`.

4.4 Ma trận phân quyền

Policy của DocVault không chỉ dựa trên role. Kết quả truy cập được xác định bởi nhiều yếu tố: role, owner, ACL, classification, status, version existence và trạng thái deleted/archived. Tuy nhiên, ở mức tổng quan, ma trận dưới đây thể hiện ý nghĩa các vai trò chính.

Bảng 4.1: Ma trận quyền tổng quan theo vai trò

Vai trò	Quyền chính
Viewer	Xem danh sách/metadata và preview/download tài liệu đã published trong phạm vi policy.
Editor	Tạo tài liệu, upload version, chỉnh metadata, submit, archive tài liệu thuộc quyền sở hữu hoặc được cấp quyền.
Approver	Xem hàng đợi duyệt, kiểm tra tài liệu pending, approve hoặc reject.
Compliance Officer	Xem audit, security summary, retention, evidence và metadata được phép; luôn bị chặn nội dung file.
Admin	Quản trị tài liệu, thành viên, evidence, retention và các thao tác nhạy cảm.

4.4.1 Quyền xem và tải theo classification

Bảng ?? mô tả quyền truy cập nội dung file trong luồng người dùng thông thường. Ký hiệu **X** là xem trước nội dung và **T** là tải file. Quyền tải chỉ được xét khi tài liệu ở

trạng thái **PUBLISHED** và đã có version; quyền xem trước có thể áp dụng ở các trạng thái workflow khác nếu đã có version. **ACL DENY** phù hợp được ưu tiên hơn quyền role. Các ô ghi “Owner/ACL” yêu cầu người dùng là chủ sở hữu hoặc có **ACL ALLOW** đúng loại quyền.

Bảng 4.2: Quyền xem và tải nội dung theo vai trò và classification

Vai trò	PUBLIC	INTERNAL	CONFIDENTIAL	SECRET
Viewer	X: Có T: Có	X: Không T: Không	X: Owner/ACL READ T: Owner/ACL DOWNLOAD	X: Chỉ owner T: Không
Editor	X: Có T: Có	X: Có T: Có	X: Owner/ACL READ T: Owner/ACL DOWNLOAD	X: Chỉ owner T: Không
Approver	X: Có T: Có	X: Có T: Có	X: Có T: Owner/ACL DOWNLOAD	X: Có T: Owner/ACL DOWNLOAD
Compliance Officer	X: Không T: Không	X: Không T: Không	X: Không T: Không	X: Không T: Không
Admin	X: Có T: Có	X: Có T: Có	X: Có T: Có	X: Có T: Có

Compliance Officer vẫn có thể đọc metadata của tài liệu PUBLISHED/ARCHIVED để phục vụ audit và evidence, nhưng không có quyền xem trước, stream, presign hoặc tải nội dung file ở bất kỳ classification nào. Share link có grant hợp lệ là một cơ chế cấp quyền riêng và không được biểu diễn trong ma trận role cơ sở này.

4.5 Chính sách Compliance Officer

Một quyết định thiết kế quan trọng của DocVault là tách rõ quyền compliance khỏi quyền truy cập nội dung file. Compliance Officer cần xem metadata và audit để đánh giá tuân thủ, nhưng có quyền audit không đồng nghĩa với được tải file. Tách hai quyền này lại giảm rủi ro ai đó lạm dụng vai trò compliance để chạm tới tài liệu nhạy cảm.

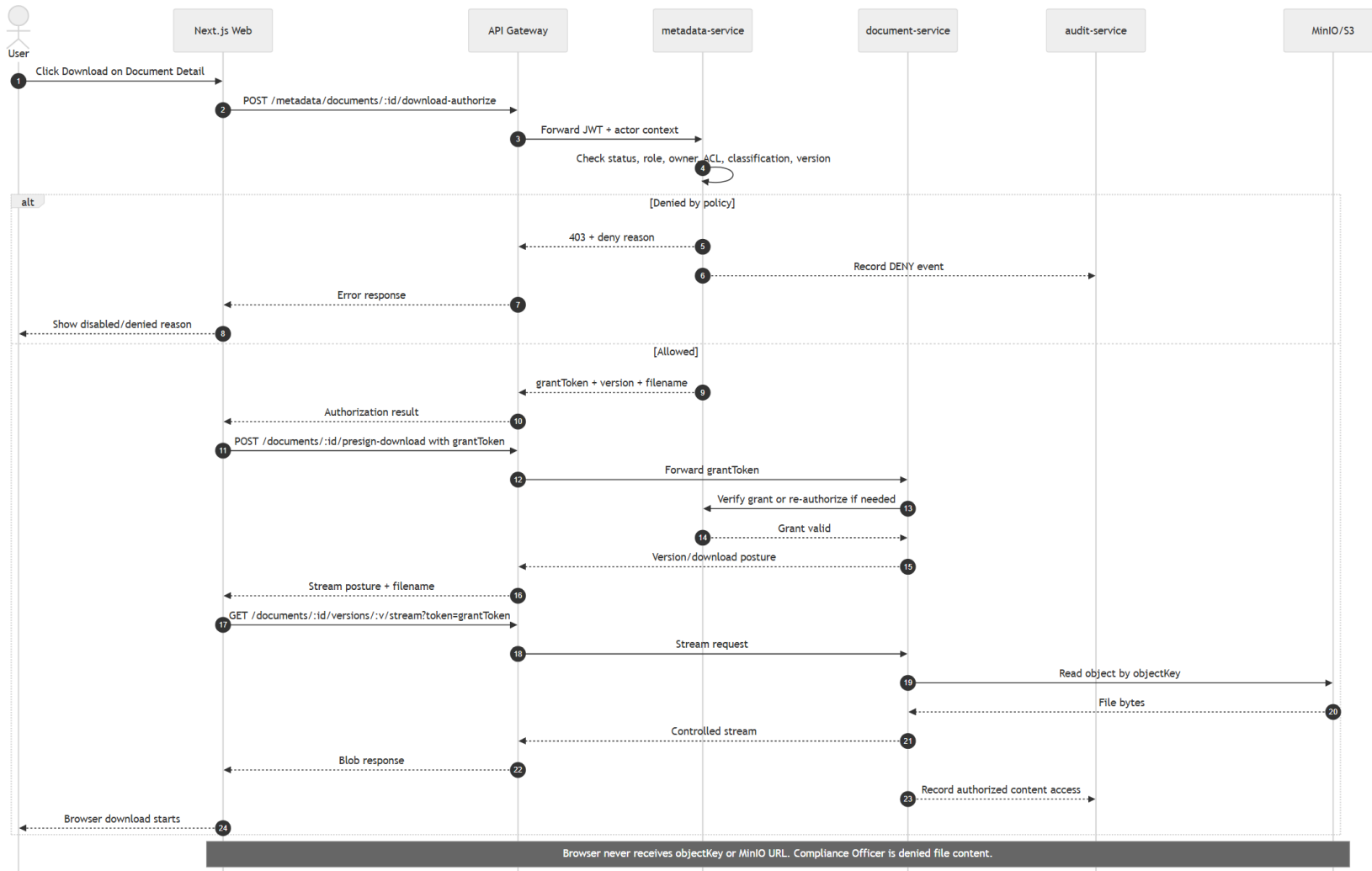
Trong DocVault, Compliance Officer:

- được truy vấn audit events;
- được verify audit hash-chain;
- được xem security summary;
- được xem retention evidence;
- được export evidence packet;
- không được preview, stream, presign hoặc download nội dung file.

Quy tắc này được thực thi ở nhiều tầng: frontend permission helper để hiển thị lý do từ chối, gateway proxy để chặn đường gọi phù hợp và service policy để đảm bảo backend là nguồn quyết định cuối cùng.

4.6 Thiết kế preview/download authorization

Download được thiết kế thành luồng hai bước. Ở bước đầu, frontend gọi metadata-service thông qua gateway để xin authorize. Metadata-service kiểm tra policy và nếu hợp lệ sẽ cấp grant token ngắn hạn. Ở bước sau, frontend dùng grant token để gọi document-service. Document-service xác minh grant token hoặc tái authorize trước khi cấp presigned URL hoặc stream endpoint.

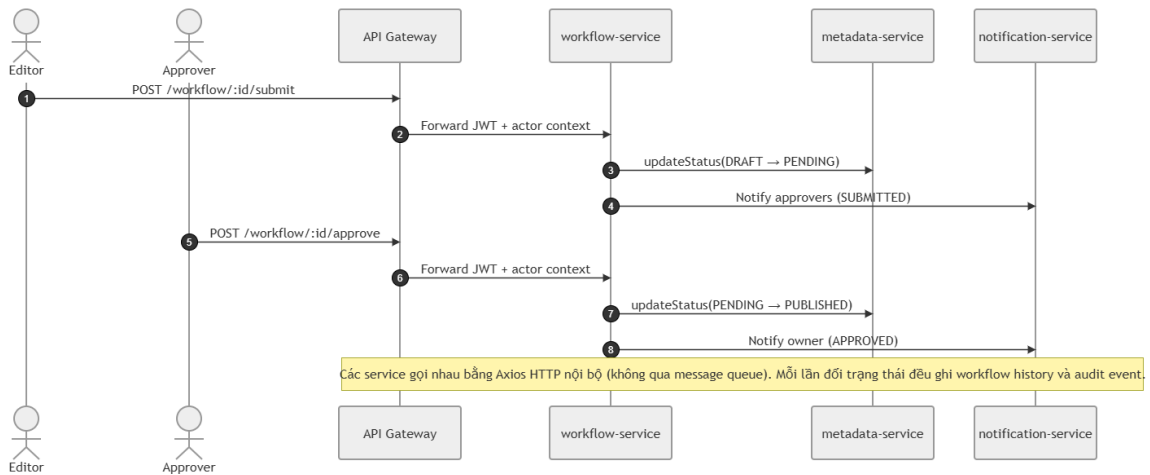


Hình 4.3: Sequence cấp quyền download và stream file

Với tài liệu **PUBLIC** hoặc **INTERNAL**, nếu policy cho phép, hệ thống có thể trả về presigned URL. Với tài liệu **CONFIDENTIAL** hoặc **SECRET**, hệ thống trả về `url: null`, `watermarkRequired: true` và `streamingEndpoint`, để document-service nắm được đường tải nội dung nhạy cảm và kiểm soát chặt hơn.

4.7 Luồng phê duyệt tài liệu

Workflow phê duyệt là luồng đa-service tiêu biểu của DocVault. Editor gửi tài liệu duyệt và Approver phê duyệt; trong cả hai bước, workflow-service đóng vai trò điều phối: gọi metadata-service để đổi trạng thái, rồi gọi notification-service để thông báo cho đúng người nhận. Hình ?? mô tả luồng chính từ **DRAFT** đến **PUBLISHED**.



Hình 4.4: Sequence luồng phê duyệt tài liệu (submit và approve)

Điểm thiết kế đáng chú ý là các service giao tiếp với nhau bằng Axios HTTP nội bộ, không qua message queue, và mỗi lần đổi trạng thái đều ghi workflow history kèm audit event. Luồng từ chối (reject) đưa tài liệu **PENDING** quay lại **DRAFT** kèm lý do, theo cùng mô hình điều phối này.

4.8 Malware scanning và DLP

Document-service thực hiện scanning trước khi ghi object vào MinIO. Với payload EICAR trong môi trường demo, upload bị chặn và không tạo object cũng như không đăng ký metadata version. Đây là một điểm quan trọng: nếu scan chỉ phát hiện sau khi file đã lưu, hệ thống sẽ phải xử lý cleanup phức tạp hơn.

DLP scan phát hiện một số pattern nhạy cảm như email, số điện thoại/số định danh và từ khóa như **secret**, **confidential**, **internal only**. Khi phát hiện dữ liệu nhạy cảm trong tài liệu public/internal, hệ thống nâng classification lên **CONFIDENTIAL**. Người dùng

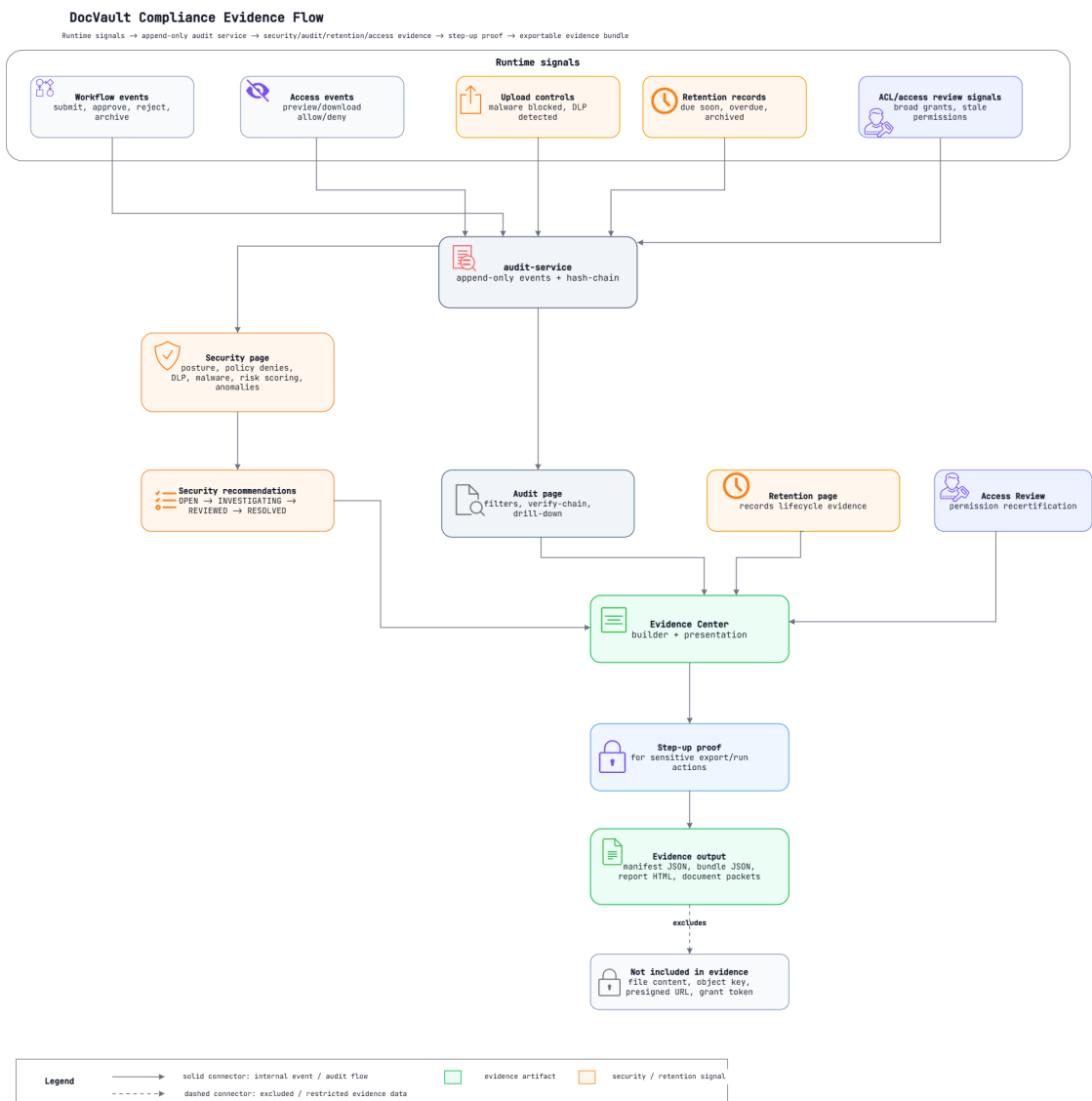
không có quyền không thể hạ classification xuống public/internal nếu không có lý do override hợp lệ.

4.9 Audit ingest boundary

Audit-service không cho người dùng thường append audit event trực tiếp. Service tin cậy phải gửi kèm header service token `x-docvault-service-token`, cấu hình qua `AUDIT_INGEST_TOKEN`. Rào này chặn việc một người có JWT hợp lệ tạo audit event giả để che giấu hoặc làm sai lệch lịch sử.

4.10 Luồng compliance evidence

Compliance evidence trong DocVault được xây dựng từ nhiều tín hiệu runtime: workflow event, access event, upload control, retention record và access review signal. Các tín hiệu này được audit-service ghi nhận theo dạng append-only và hash-chain, sau đó được các màn hình Security, Audit, Retention, Access Review và Evidence Center tổng hợp thành bằng chứng có thể xuất ra ngoài.



Hình 4.5: Luồng tổng hợp compliance evidence trong DocVault

DocVault tổng hợp compliance evidence theo một luồng ba giai đoạn từ tín hiệu runtime đến bằng chứng có thể xuất ra ngoài (Hình ??). Giai đoạn nguồn là các tín hiệu phát sinh trong vận hành: workflow event, access event, upload control (malware/DLP), retention record và access review signal. Giai đoạn lưu trữ là audit-service ghi nhận các tín hiệu này theo dạng append-only kèm hash-chain, tạo thành một dòng sự kiện có thể kiểm tra tính toàn vẹn. Giai đoạn tổng hợp là các màn hình Security, Audit, Retention, Access Review và Evidence Center đọc dòng sự kiện đó và kết tinh thành bằng chứng.

Luồng được thiết kế một chiều từ nguồn đến evidence, nên bằng chứng luôn dẫn xuất từ tín hiệu đã ghi chứ không phải dữ liệu nhập tay, khó bị làm giả sau sự việc. Ngay trước khi xuất, một lần ranh lọc bảo đảm evidence packet không bao giờ chứa file content, object key, presigned URL hay grant token. Compliance Officer nhờ đó kiểm tra được tính tuân thủ và truy vết hoạt động, còn quyền của họ không bị nói rộng thành quyền đọc nội dung tài liệu.

4.11 API contract tổng quan

API của hệ thống được expose qua gateway với base path `/api`. Các nhóm endpoint chính gồm:

Bảng 4.3: Nhóm API chính của DocVault

Nhóm API	Ví dụ endpoint	Mục đích
Auth	<code>/auth/login,</code> <code>/auth/logout</code> <code>/auth/me,</code>	Đăng nhập, callback, logout và lấy người dùng hiện tại.
Metadata	<code>/metadata/documents,</code> <code>/metadata/documents/:id</code>	Tạo, đọc, cập nhật metadata tài liệu.
ACL	<code>/metadata/documents/:id/acl</code>	Thêm/xem rule ACL theo tài liệu.
Version	<code>/metadata/documents/:id/versions</code>	Đăng ký version metadata sau upload.
Download authorization	<code>/metadata/documents/:id/download-authorize</code>	Xin quyền download và grant token.
Preview authorization	<code>/metadata/documents/:id/preview-authorize</code>	Xin quyền preview nội dung file.
Document blob	<code>/documents/:id/upload</code> <code>/documents/:id/presign-download</code> <code>/documents/:id/versions/:v/stream</code>	Upload, presign hoặc stream file.
Workflow	<code>/workflow/:id/submit,</code> <code>/workflow/:id/approve</code>	Chuyển trạng thái tài liệu.
Audit	<code>/audit/query,</code> <code>/audit/verify-chain,</code> <code>/audit/security-summary</code>	Truy vấn audit và kiểm tra hash-chain.
Evidence	<code>/metadata/documents/:id/evidence-packet</code>	Xuất evidence packet metadata-only.
Retention	<code>/metadata/retention/documents</code> <code>/metadata/retention/run</code>	Xem và chạy retention demo.

Chương 5

Triển khai ứng dụng DocVault

5.1 Cấu trúc monorepo

DocVault được tổ chức dưới dạng monorepo sử dụng pnpm và Turbo, gom frontend, backend services, shared libraries, infrastructure và tài liệu vào cùng một repository. Các thư mục chính gồm:

- `apps/web`: ứng dụng frontend Next.js.
- `services/gateway`: API Gateway.
- `services/metadata-service`: service quản lý metadata, ACL, policy và retention.
- `services/document-service`: service xử lý upload/download file và MinIO/S3.
- `services/workflow-service`: service xử lý state machine tài liệu.
- `services/audit-service`: service ghi/truy vấn audit log và verify hash-chain.
- `services/notification-service`: notification sink cho môi trường demo/dev.
- `libs/auth`, `libs/contracts`, `libs/throttler`: các thư viện dùng chung.
- `infra`: Docker Compose, Kubernetes, Helm, Argo CD, Terraform và các cấu hình hạ tầng.
- `docs`: tài liệu thiết kế, hướng dẫn chạy, evidence và screenshot.

Monorepo giúp báo cáo, code, pipeline và evidence nằm chung một vòng đời. Khi một thay đổi chạm tới API contract hoặc security policy, nhóm có thể cập nhật code, test và tài liệu trong cùng một pull request.

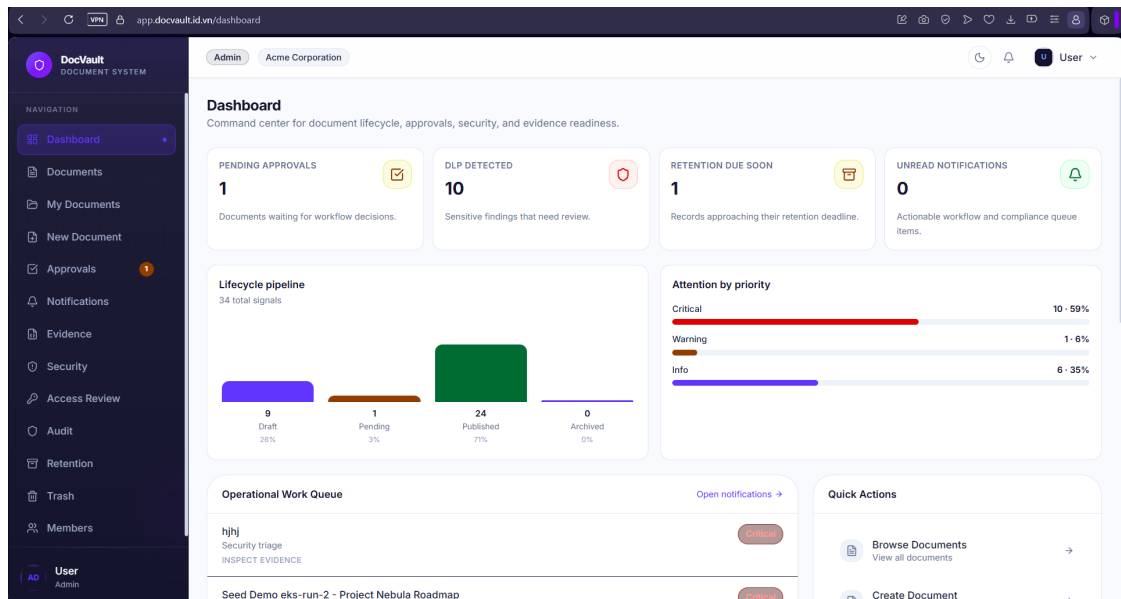
5.2 Frontend Next.js

Frontend được xây dựng bằng Next.js, React, TanStack Query, React Hook Form, Zod, Tailwind CSS và Radix UI. Ứng dụng sử dụng App Router và tổ chức theo các nhóm route đã đăng nhập hoặc chưa đăng nhập. Frontend gọi gateway qua biến môi trường

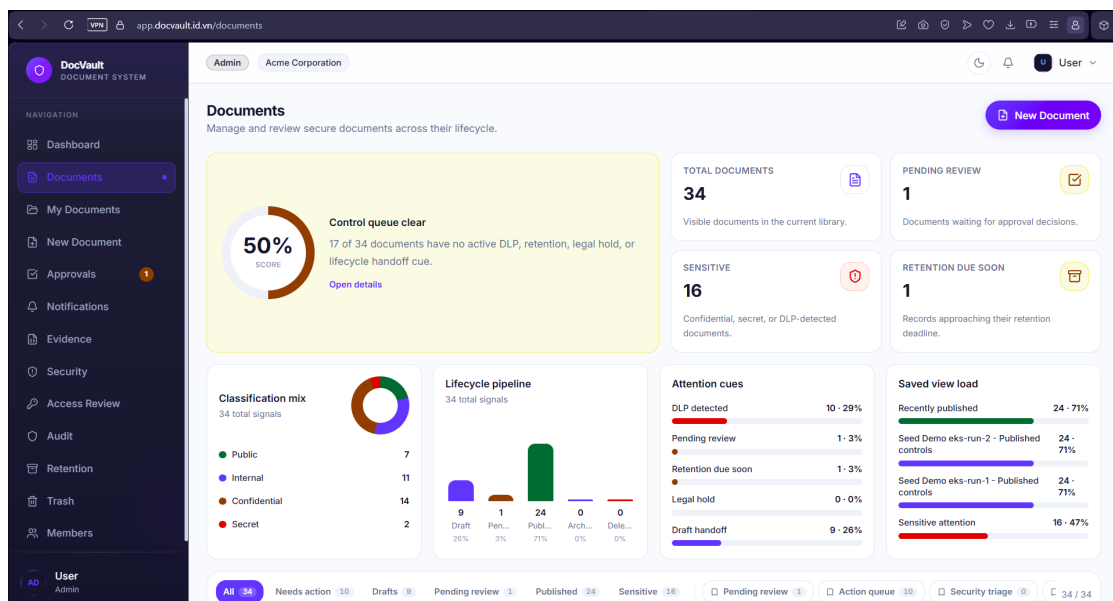
NEXT_PUBLIC_API_BASE_URL, mặc định trỏ đến `http://localhost:3000/api` trong môi trường local.

Frontend làm nhiều việc hơn là hiển thị danh sách tài liệu. Ứng dụng đã bao phủ nhiều luồng vận hành quan trọng:

- Đăng nhập SSO qua Keycloak và quản lý phiên.
- Dashboard tổng quan tài liệu, pending approval, DLP, retention và readiness.
- Documents và My Documents với lọc nâng cao, quick views, saved views, folder tree và phân trang.
- New Document và Edit Document cho tạo/chỉnh metadata và upload file.
- Document Detail với preview/download, version history, workflow timeline, ACL, comments, activity, share link, legal hold, DLP findings, AI guardrails và evidence links.
- Approvals với SLA summary, review drawer, approve/reject và reason preset.
- Audit, Security, Evidence Center, Retention và Access Review cho compliance/admin.
- Notifications, Trash, Members, Profile, Settings và Demo Kit.



Hình 5.1: Màn hình dashboard của DocVault



Hình 5.2: Màn hình danh sách tài liệu

5.2.1 Permission-aware UI

Frontend có helper phân quyền để xác định action nào nên hiển thị, action nào nên bị disable và lý do deny là gì. Chẳng hạn, Compliance Officer được thấy lý do bị chặn download thay vì chỉ thấy nút biến mất. Dù vậy, helper frontend không phải boundary bảo mật cuối cùng; backend mới là nơi quyết định policy, tránh việc người dùng tự gọi API để vượt qua UI.

5.3 Gateway

Gateway là entry point của hệ thống. Service này xác thực JWT từ Keycloak, kiểm tra role ở mức route, expose health endpoint, proxy request xuống service phía sau và truyền các header ngữ cảnh. Các header quan trọng gồm:

- authorization
- x-request-id
- x-user-id
- x-roles
- x-groups

Gateway proxy các nhóm route chính như /metadata, /documents, /workflow, /audit và /notify. Frontend vì thế chỉ cần biết một API base URL, trong khi bên trong hệ thống vẫn tách service theo bounded context.

5.4 Metadata-service

Metadata-service là nguồn sự thật cho tài liệu logic và policy. Service này sở hữu các dữ liệu và nghiệp vụ:

- document record;
- classification và tags;
- current version pointer;
- ACL entries;
- workflow history;
- download/preview authorization;
- retention evidence;
- access impact simulation;
- AI guardrails metadata-only.

Metadata-service là nơi quan trọng nhất để thực thi policy liên quan đến metadata và grant token. Khi document-service cần đăng ký version mới hoặc khi workflow-service cần cập nhật trạng thái, các service này gọi metadata-service thay vì ghi trực tiếp vào database.

Một ví dụ tiêu biểu cho nguyên tắc *backend authoritative* là quy tắc chặn Compliance Officer tải file. Quy tắc này được đặt ngay đầu hàm kiểm tra quyền download trong `policy.service.ts`, trước cả các bước kiểm tra trạng thái và ACL, để bảo đảm không có nhánh nào bỏ sót:

Listing 5.1: Quy tắc deny download của Compliance Officer trong `policy.service.ts`

```
private getDeniedReason(status: string, roles: string[]): string | null {
  if (roles.includes('compliance_officer')) {
    return 'Compliance officers are never allowed to download files';
  }
  if (status !== 'PUBLISHED') {
    return 'Only published documents can be downloaded';
  }
  return null;
}
```

Vì quyết định nằm ở backend chứ không ở frontend, người dùng compliance không thể lách bằng cách gọi thẳng API. Khi quy tắc trả về lý do từ chối, service ném `ForbiddenException` và ghi audit event tương ứng.

Grant token download/preview cũng do metadata-service cấp. Token có thời hạn 300 giây, được ký bằng HMAC-SHA256 và hỗ trợ xoay khóa qua `kid`, sau đó document-service xác minh trước khi phát file:

Listing 5.2: Cấp grant token ngắn hạn cho download

```
const encoded = Buffer.from(JSON.stringify(tokenPayload))
  .toString('base64url');
const signature = createHmac('sha256', signing.secret)
  .update(encoded)
  .digest('base64url');
return `${encoded}.${signature}`;
```

5.5 Document-service

Document-service xử lý phần blob file. Service này nhận upload multipart, tính checksum SHA-256, scan malware/DLP, sinh object key và upload file lên MinIO/S3. Sau khi upload thành công, service gọi metadata-service để đăng ký version mới.

Đối với download, document-service không cấp file một cách độc lập. Service xác minh grant token hoặc tái authorize thông qua metadata-service trước khi trả presigned URL hoặc stream endpoint. Với tài liệu nhạy cảm, service trả về stream endpoint để áp dụng posture kiểm soát tốt hơn.

5.6 Workflow-service

Workflow-service sở hữu logic chuyển trạng thái. Service đọc trạng thái hiện tại từ metadata-service, kiểm tra transition hợp lệ rồi yêu cầu metadata-service cập nhật trạng thái. Các transition chính:

- SUBMIT: DRAFT -> PENDING
- APPROVE: PENDING -> PUBLISHED
- REJECT: PENDING -> DRAFT
- ARCHIVE: PUBLISHED -> ARCHIVED

Sau mỗi transition, workflow-service phát audit event và gọi notification-service. Workflow vẫn là nơi điều phối nghiệp vụ, còn metadata-service vẫn là owner của dữ liệu trạng thái.

5.7 Audit-service

Audit-service ghi và truy vấn audit log. Service hỗ trợ append audit event từ các service tin cậy, query theo actor/action/resource/result/time, verify hash-chain và security summary. Endpoint audit query chỉ dành cho Compliance Officer hoặc Admin.

Hash-chain giúp phát hiện sửa đổi audit event ngoài luồng. Nếu một event cũ bị chỉnh trực tiếp trong database, verify-chain trả về trạng thái không hợp lệ. Trong phạm vi đồ án, đây là cơ chế tamper-evident phù hợp để chứng minh ý tưởng kiểm soát audit.

5.8 Notification-service

Notification-service hiện là notification sink cho môi trường dev/demo. Service nhận payload từ workflow hoặc các sự kiện liên quan và lưu/log để frontend hiển thị notification center. Đây chưa phải hệ thống email, SMS hoặc push notification hoàn chỉnh, nhưng đủ để mô phỏng luồng thông báo trong sản phẩm. Trạng thái chưa hoàn thiện này được ghi nhận lại trong phần hạn chế ở Mục ??.

5.9 Hạ tầng local

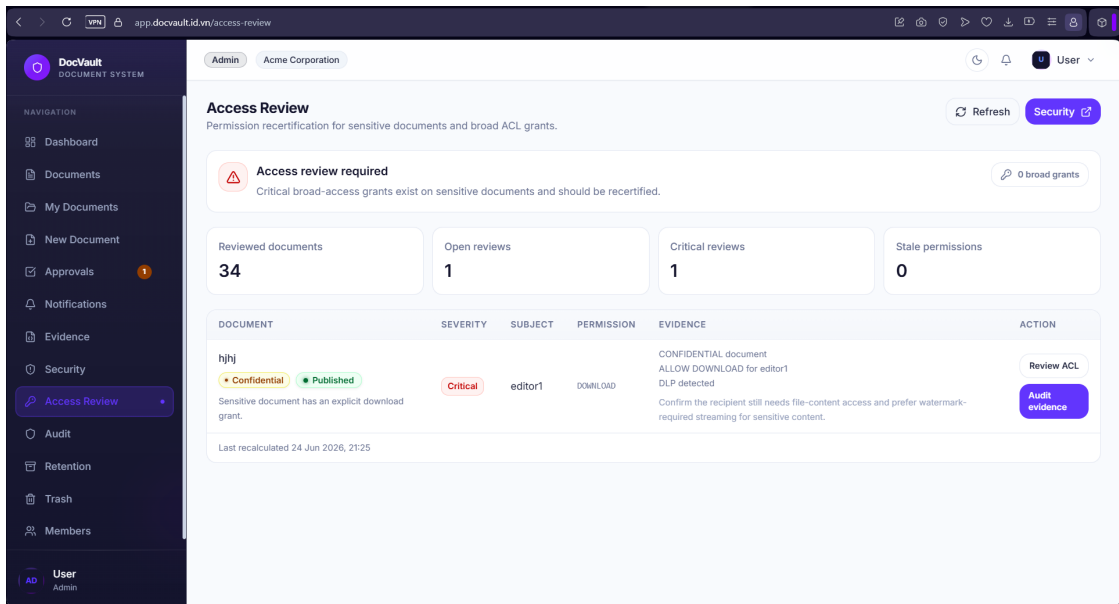
Môi trường local sử dụng Docker Compose để chạy các dependency:

- PostgreSQL cho metadata.
- MongoDB cho audit.
- MongoDB Express để quan sát dữ liệu audit trong dev.
- MinIO cho object storage.
- MinIO init job để tạo bucket và bật cấu hình cần thiết.
- Keycloak cho identity, realm, roles và demo users.

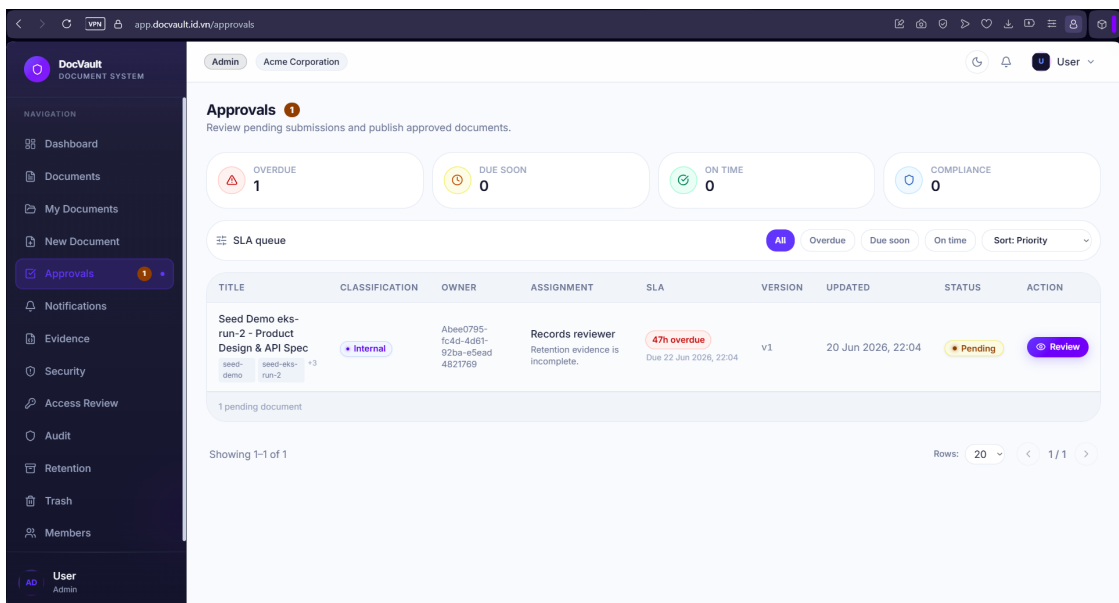
Các service backend chạy trên các port 3000 đến 3005; frontend chạy trên port 3006 (script `pnpm --filter web dev`) để tránh xung đột với gateway và các service backend.

5.10 Các màn hình minh họa

Các hình dưới đây thể hiện một số giao diện chính đã triển khai trong frontend.



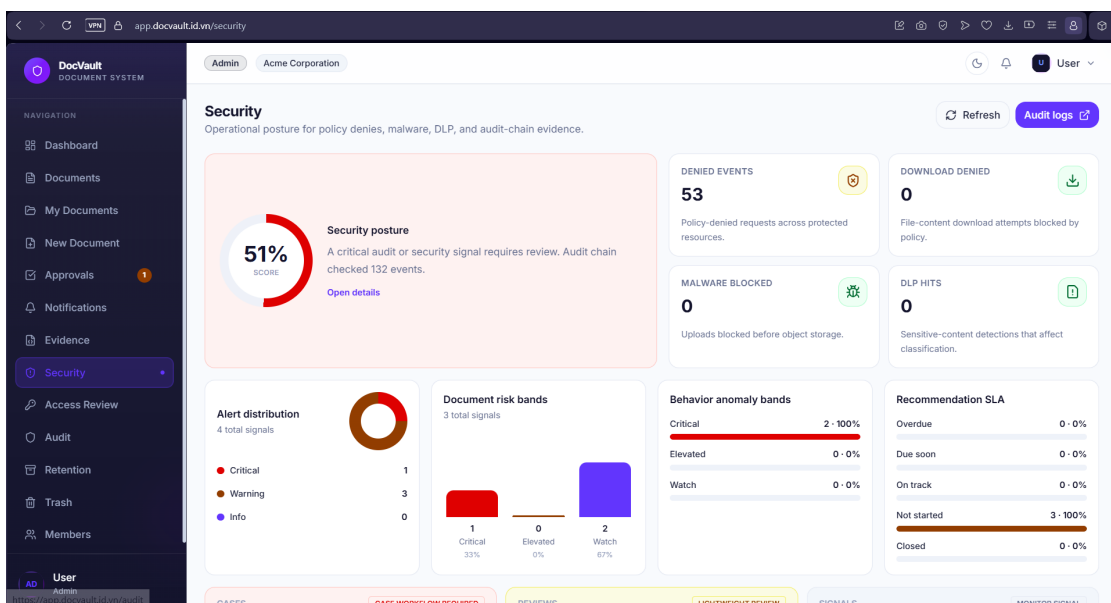
Hình 5.3: Màn hình rà soát quyền truy cập



Hình 5.4: Màn hình phê duyệt tài liệu

TIMESTAMP	ACTOR	ROLES	ACTION	RESOURCE TYPE	RESOURCE ID	RESULT	REASON	TRACE ID
24 Jun 2026, 21:22	Admin One	admin	SECURITY_RECOMMENDATIONS_VIEWED	AUDIT	-	SUCCESS	-	-
23 Jun 2026, 21:14	Admin One	admin	SECURITY_RECOMMENDATIONS_VIEWED	AUDIT	-	SUCCESS	-	-
23 Jun 2026, 15:30	Admin One	admin	SECURITY_RECOMMENDATIONS_VIEWED	AUDIT	-	SUCCESS	-	-
23 Jun 2026, 00:30	Admin One	admin	DOCUMENT_ACL_UPDATED	DOCUMENT	27537fb..2a716cf	SUCCESS	-	5146dc..1422fa
23 Jun 2026, 00:29	Admin One	admin	COMMENT_ADDED	COMMENT	a3eb59c..64af359	SUCCESS	-	b8465b..994f65
23 Jun 2026, 00:29	Admin One	admin	DOCUMENT_DOWNLOAD_AUTHORIZED	DOCUMENT	27537fb..2a716cf	SUCCESS	-	06348b..6626b3
23 Jun 2026, 00:28	Admin One	admin	DOCUMENT_PREVIEW_AUTHORIZED	DOCUMENT	27537fb..2a716cf	SUCCESS	-	171f66..6626b3
23 Jun 2026, 00:28	Admin One	admin	SECURITY_RECOMMENDATIONS_VIEWED	AUDIT	-	SUCCESS	-	-
21 Jun 2026, 01:18	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	637a15..25b29d
21 Jun 2026, 01:18	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	11fc07..b709c8
21 Jun 2026, 01:17	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	fd43b2..78aa20
21 Jun 2026, 01:17	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	6e5614..9afcab
21 Jun 2026, 01:17	Editor One	editor	DOCUMENT_METADATA_READ_DENIED	DOCUMENT	27537fb..2a716cf	DENY	Metadata read d...	19146d..79eb82

Hình 5.5: Màn hình audit log



Hình 5.6: Màn hình security posture

5.11 Luồng demo ứng dụng

Luồng demo chính được thiết kế để chứng minh cả nghiệp vụ và bảo mật:

1. Editor đăng nhập, tạo tài liệu và upload file.
2. Editor submit tài liệu để chuyển từ Draft sang Pending.
3. Approver đăng nhập, mở hàng đợi Approvals và approve tài liệu.
4. Viewer đăng nhập, xem tài liệu đã Published và download nếu policy cho phép.

5. Compliance Officer đăng nhập, mở Audit/Security/Evidence nhưng bị từ chối pre-view/download nội dung file.

Luồng này thể hiện đủ các thành phần chính của hệ thống: identity, gateway, metadata, document storage, workflow, audit, notification và frontend.

Chương 6

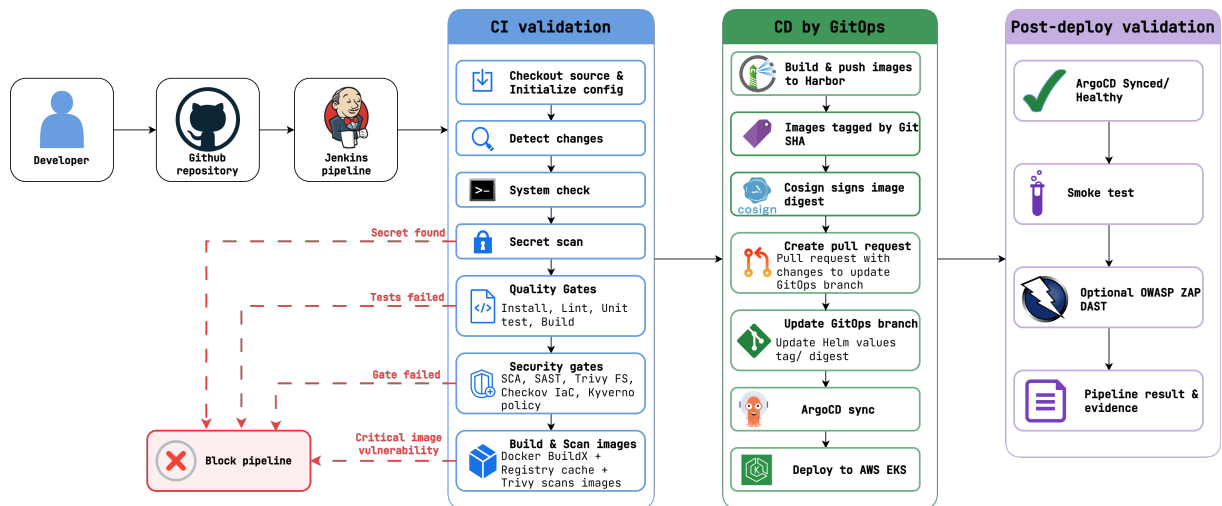
Quy trình DevSecOps và hạ tầng triển khai

6.1 Mục tiêu và phạm vi triển khai

Quy trình DevSecOps của DocVault được xây dựng nhằm tự động hóa vòng đời phát hành từ mã nguồn đến môi trường AWS EKS, đồng thời tích hợp các kiểm soát chất lượng và bảo mật tại nhiều thời điểm của quy trình. Jenkins không triển khai trực tiếp tài nguyên lên Kubernetes. Thành phần này thực hiện kiểm tra mã nguồn, xây dựng và quét container image, đẩy artifact lên Harbor, sau đó cập nhật trạng thái mong muốn trên nhánh GitOps. Argo CD chịu trách nhiệm đối chiếu trạng thái trong Git với trạng thái thực tế và đồng bộ các thay đổi xuống cụm EKS.

Thiết kế trên hướng đến các mục tiêu sau:

- Hạ tầng AWS được mô tả bằng Terraform để có thể tái tạo, kiểm tra và quản lý thay đổi.
- Mã nguồn, thư viện phụ thuộc, secret, filesystem, container image và cấu hình IaC được kiểm tra tự động.
- Image triển khai được lưu trong registry riêng, gắn tag theo Git commit và cố định bằng digest.
- Trạng thái triển khai được lưu trong Git, qua đó hình thành lịch sử thay đổi và cơ sở rollback.
- Giá trị bí mật của ứng dụng không được ghi trực tiếp trong repository mà được đồng bộ từ AWS Secrets Manager.
- Trạng thái sau triển khai được xác minh bằng Argo CD health check, smoke test và DAST.
- Metric, log và trạng thái workload được tập trung để phục vụ giám sát và thu thập bằng chứng.



Hình 6.1: Luồng tổng quan CI, CD theo GitOps và kiểm tra sau triển khai

DocVault tổ chức quy trình phát hành thành ba phân đoạn nối tiếp, từ kiểm tra mã nguồn đến triển khai và xác minh sau triển khai (Hình ??). Phân đoạn CI bắt đầu khi mã nguồn được đẩy lên Git: Jenkins checkout, cài dependency, quét secret, chạy test/lint, rồi áp các security gate gồm SCA, Trivy, SonarQube và kiểm tra IaC. Phân đoạn đóng gói build Docker image, quét image bằng Trivy, đẩy lên Harbor, lấy digest và tùy chọn ký bằng Cosign; chỉ image vượt qua mọi gate mới được phát hành. Phân đoạn CD theo GitOps là nơi Jenkins cập nhật desired state trên nhánh GitOps, Argo CD đối chiếu và đồng bộ xuống EKS, sau đó pipeline chạy kiểm tra sau triển khai gồm Argo CD health check, smoke test và DAST.

Ranh giới quan trọng nhất trong thiết kế là Jenkins không trực tiếp `kubectl apply` lên cụm; nó chỉ ghi trạng thái mong muốn vào Git, còn Argo CD mới là thành phần thực thi thay đổi trên cluster. Trong phiên bản hiện tại, Jenkins cập nhật trực tiếp nhánh `gitops-testing` bằng commit có `[skip ci]` sau khi toàn bộ điều kiện phát hành được thỏa mãn. Bước tạo pull request là phương án gia cố tiếp theo nhằm bổ sung cơ chế rà soát và phê duyệt thay đổi GitOps.

6.2 Kiến trúc hạ tầng trên AWS EKS

6.2.1 Hạ tầng dưới dạng mã với Terraform

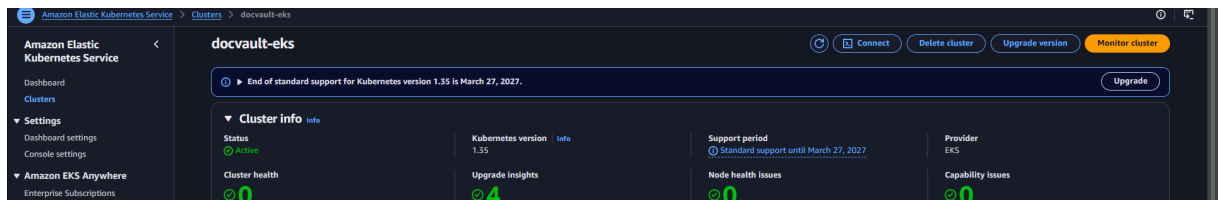
Mã nguồn Terraform được đặt tại `infra/terraform/aws-eks` và được phân tách thành các module cục bộ theo từng nhóm trách nhiệm. Module `network` khởi tạo VPC, public subnet và private subnet trên hai Availability Zone. Module `eks-cluster` khởi tạo EKS control plane, managed node group, security group và các add-on nền tảng gồm CoreDNS, kube-proxy, VPC CNI và AWS EBS CSI Driver. Việc mô tả hạ tầng bằng mã giúp chuẩn hóa cấu hình, hỗ trợ kiểm tra thay đổi và tái tạo môi trường [6].

Cấu hình hỗ trợ hai phương án mạng. Phương án tối ưu chi phí đặt worker node trong public subnet và không sử dụng NAT Gateway, phù hợp với môi trường kiểm thử. Phương án triển khai production đặt worker node trong private subnet và sử dụng NAT Gateway. Endpoint công khai của Kubernetes API có thể được giới hạn bằng danh sách CIDR thay vì cho phép truy cập từ toàn bộ Internet.

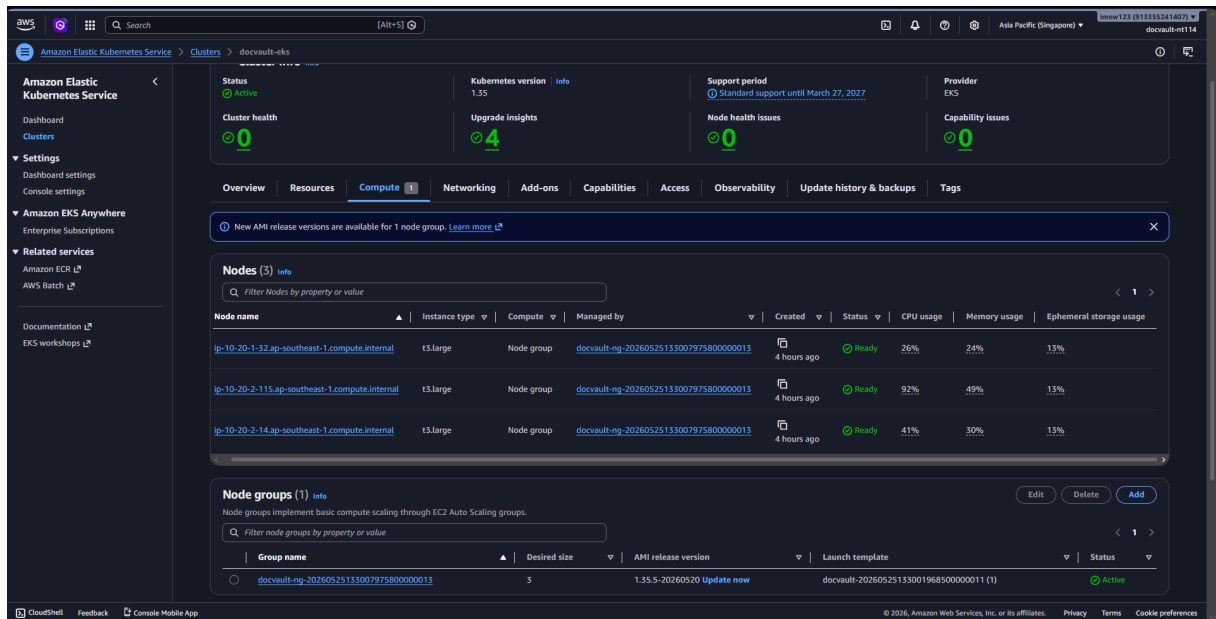
Các biện pháp gia cố bảo mật chính được thể hiện trong Terraform gồm:

- Bật đầy đủ năm nhóm log EKS control plane: API, audit, authenticator, controller manager và scheduler.
- Bắt buộc IMDSv2 trên EC2 node.
- Mã hóa volume gốc của node bằng EBS gp3.
- Bật OIDC provider để cấp quyền AWS theo IRSA cho từng Kubernetes ServiceAccount [7].
- Tạo StorageClass gp3 và cài EBS CSI Driver để hỗ trợ PersistentVolume.
- Tách module lưu trữ tài liệu, backup cơ sở dữ liệu, External Secrets và quyền Jenkins.

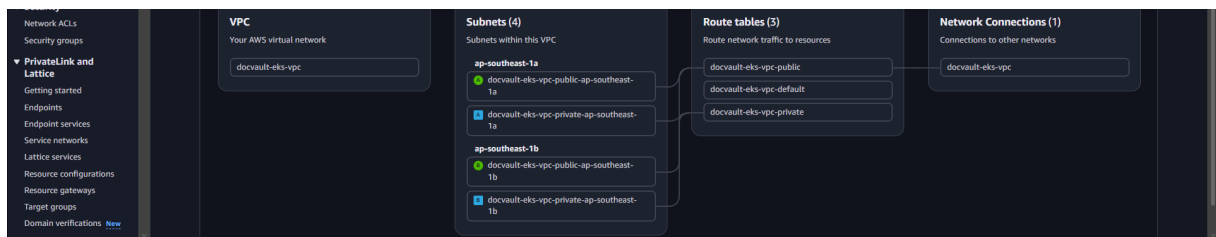
Cấu hình hiện tại sử dụng managed node group có khả năng co giãn từ một đến ba node, trong đó số lượng mong muốn của môi trường kiểm thử là hai node. Loại máy, dung lượng đĩa, phiên bản Kubernetes và số lượng node đều được khai báo dưới dạng biến, cho phép điều chỉnh mà không thay đổi logic của module.



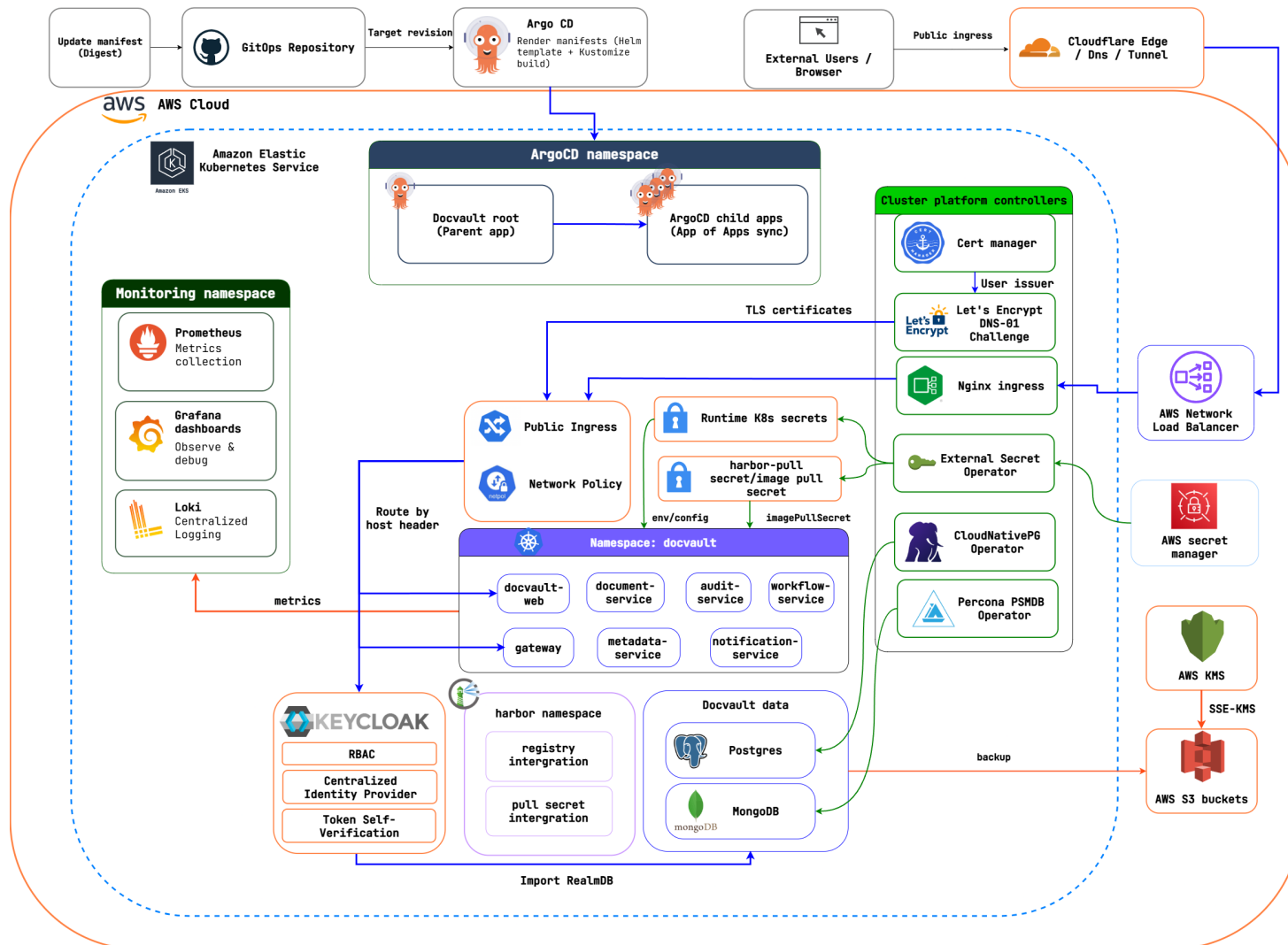
Hình 6.2: EKS cluster `docvault-eks` đang hoạt động trên khu vực Singapore



Hình 6.3: Managed node group và các worker node ở trạng thái sẵn sàng



Hình 6.4: Resource map của VPC gồm public/private subnet trên hai Availability Zone



Hình 6.5: Kiến trúc triển khai DocVault trên AWS EKS

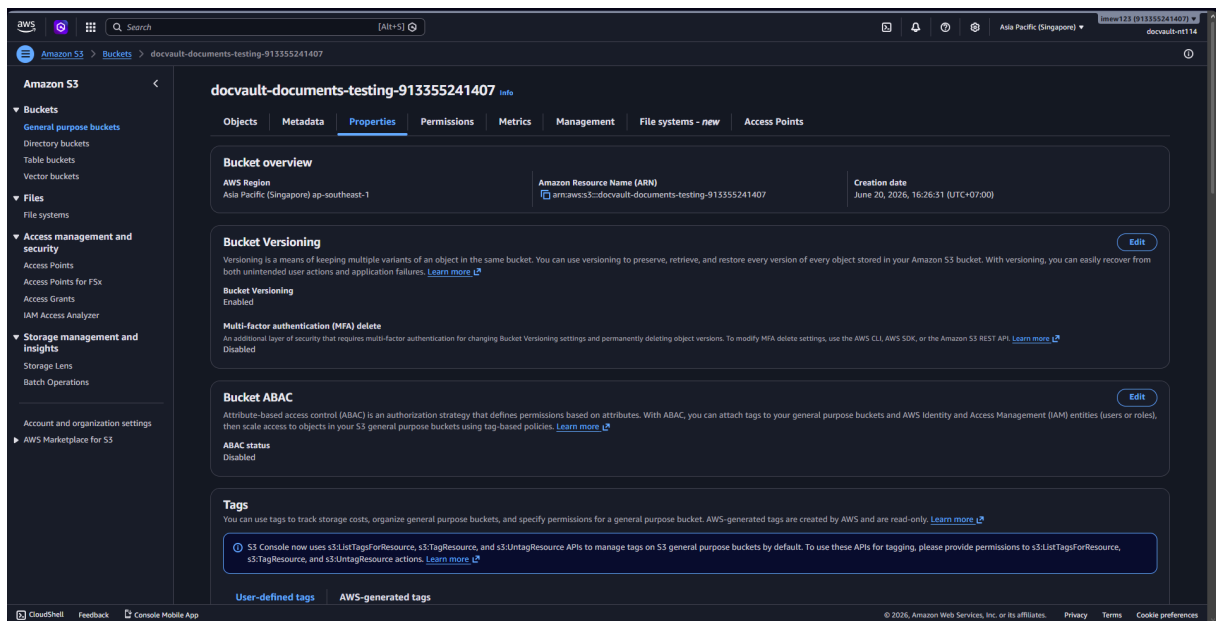
DocVault được triển khai trên AWS EKS theo bốn cụm thành phần, liên kết GitOps repository, Argo CD, các namespace ứng dụng và giám sát, nhóm controller nền tảng, Harbor, Keycloak cùng các dịch vụ AWS (Hình ??). Cụm điều khiển triển khai gồm GitOps repository và Argo CD: Argo CD theo dõi nhánh GitOps và đồng bộ desired state vào cụm, đóng vai trò cầu nối duy nhất giữa Git và Kubernetes. Cụm workload ứng dụng là các namespace chạy gateway, web và sáu backend service, được đóng gói bằng Helm. Cụm nền tảng gồm các controller dùng chung như ingress-nginx, cert-manager, External Secrets Operator và CloudNativePG. Cụm dịch vụ AWS bên ngoài cluster gồm S3 (file và backup), KMS (khóa mã hóa), Secrets Manager (bí mật) và IAM/IRSA (cấp quyền cho ServiceAccount).

Thiết kế hạ tầng dựa trên nguyên tắc không để workload lộ trực tiếp ra Internet và không lưu bí mật trong cụm. Lưu lượng truy cập từ người dùng đi từ Cloudflare đến AWS Load Balancer rồi vào ingress-nginx trước khi tới web/gateway. Bí mật đi từ AWS Secrets Manager qua External Secrets Operator để tạo Kubernetes Secret tại runtime, nên repository chỉ chứa tham chiếu chứ không chứa giá trị bí mật. Theo cùng nguyên tắc đó, file tài liệu được lưu trên S3 và mã hóa bằng KMS thay vì nằm trong cụm.

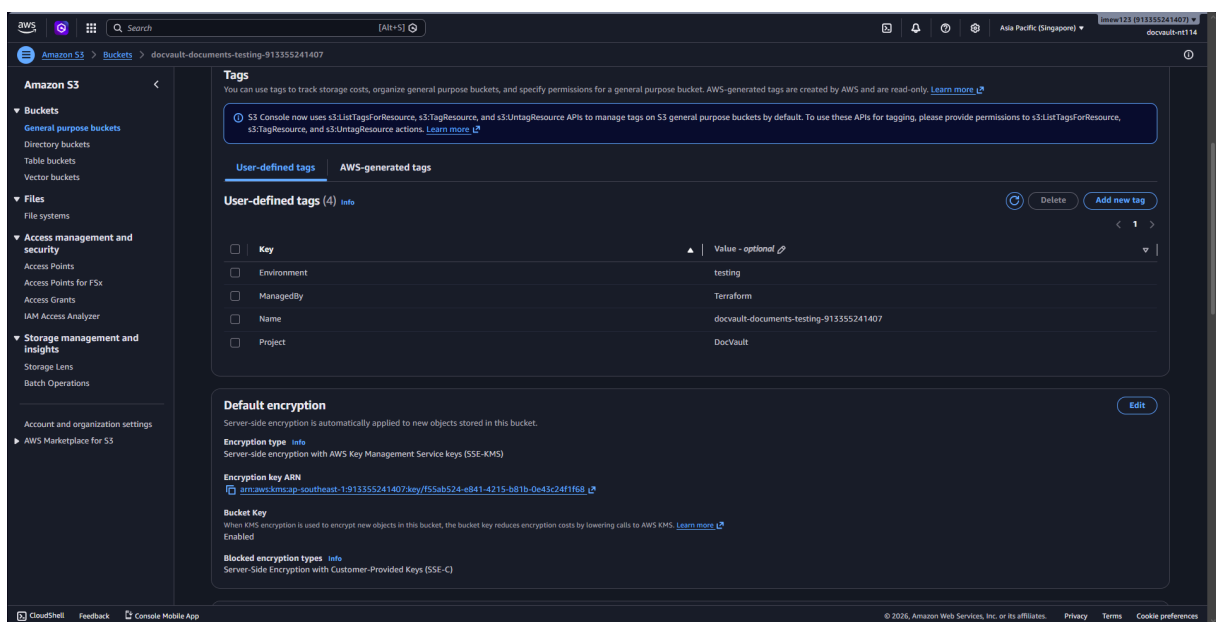
6.2.2 Lưu trữ tài liệu trên S3

Trong overlay `testing-s3`, document-service đã được chuyển từ MinIO sang Amazon S3. Terraform khởi tạo bucket tài liệu, customer-managed KMS key và IAM role dành cho ServiceAccount của document-service. Bucket được chặn truy cập công khai, bật versioning, mã hóa mặc định bằng SSE-KMS, ghi access log sang bucket riêng, áp dụng lifecycle và phát sự kiện qua EventBridge. IAM policy chỉ cấp các thao tác cần thiết trên prefix object của document-service và KMS key tương ứng. Cách tổ chức này cho phép quản lý tập trung khóa mã hóa, chính sách sử dụng khóa và dấu vết truy cập [8].

DocVault dùng customer-managed KMS key thay vì key mặc định của AWS để tự kiểm soát key policy và tách khóa theo từng mục đích: một khóa cho EKS, một cho tài liệu, một cho backup PostgreSQL và một cho backup MongoDB. Nhờ vậy mỗi nhóm dữ liệu có ranh giới quyền giải mã riêng và sự cố ở một khóa không lan sang phần còn lại. Mỗi khóa bật xoay vòng tự động hằng năm: AWS thay vật liệu khóa nền theo chu kỳ trong khi ARN của khóa giữ nguyên, nên ứng dụng không phải cập nhật tham chiếu và dữ liệu mã hóa trước đó vẫn giải mã được bằng phiên bản khóa cũ. Cơ chế này giảm rủi ro khi một phiên bản khóa bị lộ mà không gây gián đoạn dịch vụ. Ngoài ra mỗi khóa đặt cửa sổ chờ xóa 30 ngày để tránh việc xóa nhầm làm mất vĩnh viễn dữ liệu đã mã hóa [8].



Hình 6.6: S3 bucket lưu tài liệu đã bật versioning trong môi trường testing



Hình 6.7: S3 bucket tài liệu được Terraform quản lý và mã hóa mặc định bằng SSE-KMS

6.2.3 Sao lưu và khôi phục cơ sở dữ liệu

DocVault có hai cơ sở dữ liệu trạng thái cần bảo vệ với mức ưu tiên khác nhau. Metadata PostgreSQL giữ ảnh xạ object-key, ACL và lịch sử trạng thái tài liệu. Audit MongoDB giữ nhật ký kiểm toán có chuỗi băm SHA-256, là bằng chứng tuân thủ nên việc mất hoặc sai lệch dữ liệu ở đây nghiêm trọng hơn. Vì vậy hai cơ sở dữ liệu được sao lưu vào hai bucket S3 riêng, dùng hai KMS key riêng và hai IAM role IRSA riêng. Tách biệt như vậy giúp ranh giới quyền truy cập, lifecycle và quy trình xử lý sự cố của từng cơ sở dữ liệu

độc lập với nhau, đồng thời tách hẳn khỏi bucket tài liệu.

Metadata PostgreSQL được triển khai bằng CloudNativePG với volume gp3. Cấu hình backup dùng plugin Barman Cloud ghi vào bucket S3 và KMS key riêng, xác thực bằng IRSA thay vì access key tĩnh. Tài nguyên `ScheduledBackup` chạy base backup hằng ngày kèm lưu trữ WAL liên tục và giữ 30 ngày, nhờ đó hệ thống có thể khôi phục về một thời điểm bất kỳ trong cửa sổ lưu giữ thay vì chỉ về mốc backup gần nhất [9].

Vị trí chèn hình minh chứng

Trạng thái backup CloudNativePG metadata

Ảnh chụp `kubectl get cluster,scheduledbackup,backup -n docvault` cho thấy cluster metadata ở trạng thái healthy, base backup gần nhất `Completed` và WAL đang được archive lên S3.

Hình 6.8: Trạng thái backup CloudNativePG metadata (cần bổ sung ảnh minh chứng)

Audit MongoDB được triển khai bằng Percona Server for MongoDB Operator dưới dạng replica set `rs0` ba thành viên, đặt anti-affinity theo hostname để các thành viên không nằm chung node. Sao lưu do Percona Backup for MongoDB (PBM) đảm nhận: một bản backup logical hằng ngày được nén gzip và giữ 30 bản, đẩy lên bucket MongoDB riêng với SSE-KMS [10]. Bên cạnh đó, PBM bật point-in-time recovery qua oplog với chu kỳ cắt lát 10 phút, nên ngoài các mốc backup hằng ngày hệ thống còn có thể khôi phục về gần như mọi thời điểm trong khoảng giữa hai lần backup. Đây là khả năng phục hồi chặt hơn so với chỉ chụp ảnh định kỳ, phù hợp với yêu cầu toàn vẹn của dữ liệu audit.

Vị trí chèn hình minh chứng

Trạng thái Percona MongoDB replica set audit

Ảnh chụp `kubectl get psmdb -n docvault` hoặc `rs.status()` cho thấy cluster `audit-mongodb` ở trạng thái ready với một PRIMARY và hai SECONDARY.

Hình 6.9: Trạng thái Percona MongoDB replica set audit (cần bổ sung ảnh minh chứng)

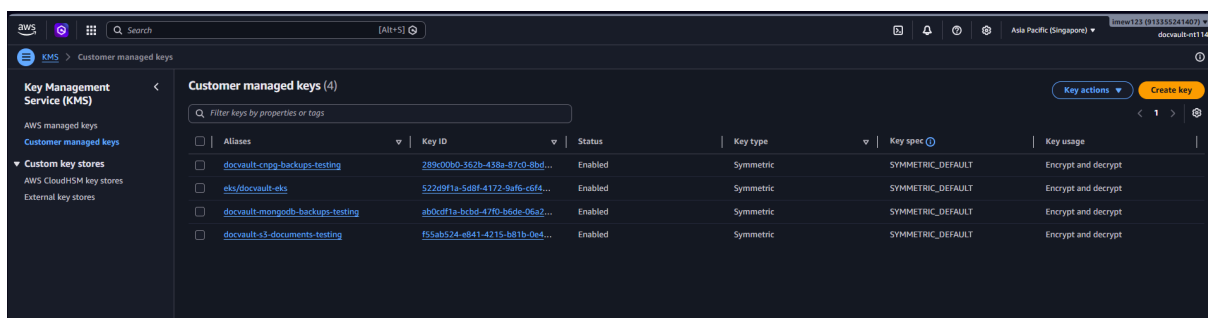
Vị trí chèn hình minh chứng

Trạng thái backup và PITR của PBM

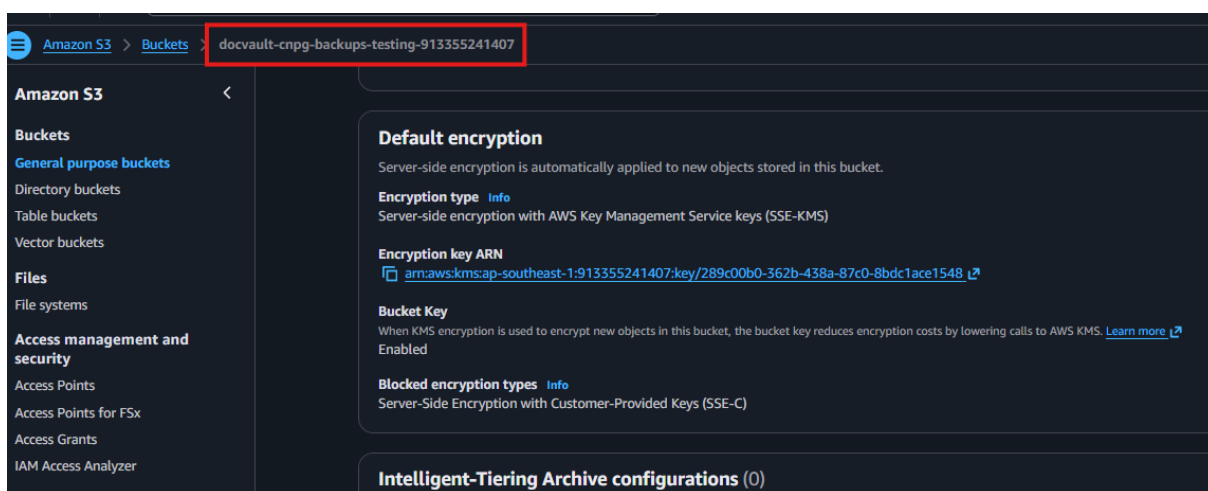
Ảnh chụp `pbm status` hoặc `pbm list` cho thấy bản backup gần nhất hoàn tất, oplog/PITR đang chạy và đối tượng backup nằm trong bucket S3 với SSE-KMS.

Hình 6.10: Trạng thái backup và PITR của PBM (cần bổ sung ảnh minh chứng)

Cả hai cơ chế hiện đã cấu hình và chạy theo lịch. Việc diễn tập khôi phục để đo RPO và RTO thực tế được bàn ở phần vận hành (Mục ??), nơi khép lại mạch giám sát, log và khôi phục.



Hình 6.11: Các customer-managed KMS key dùng cho EKS, tài liệu và backup



Hình 6.12: S3 bucket lưu backup cơ sở dữ liệu

6.3 Tổ chức workload trên Kubernetes

6.3.1 Helm chart và Kustomize overlay

Các service DocVault sử dụng Helm chart dùng chung tại `infra/k8s/charts/docvault-service`. Mỗi workload có một file values riêng cho image, port, biến môi trường, resource request/limit, migration và seed job. Việc dùng một chart chung giúp các service áp dụng đồng nhất những kiểm soát sau:

- Chạy với UID/GID không phải root và `seccompProfile=RuntimeDefault`.
- Tắt privilege escalation, đặt root filesystem chỉ đọc và loại bỏ toàn bộ Linux capability.
- Không tự động mount ServiceAccount token nếu workload không cần truy cập Kubernetes API.

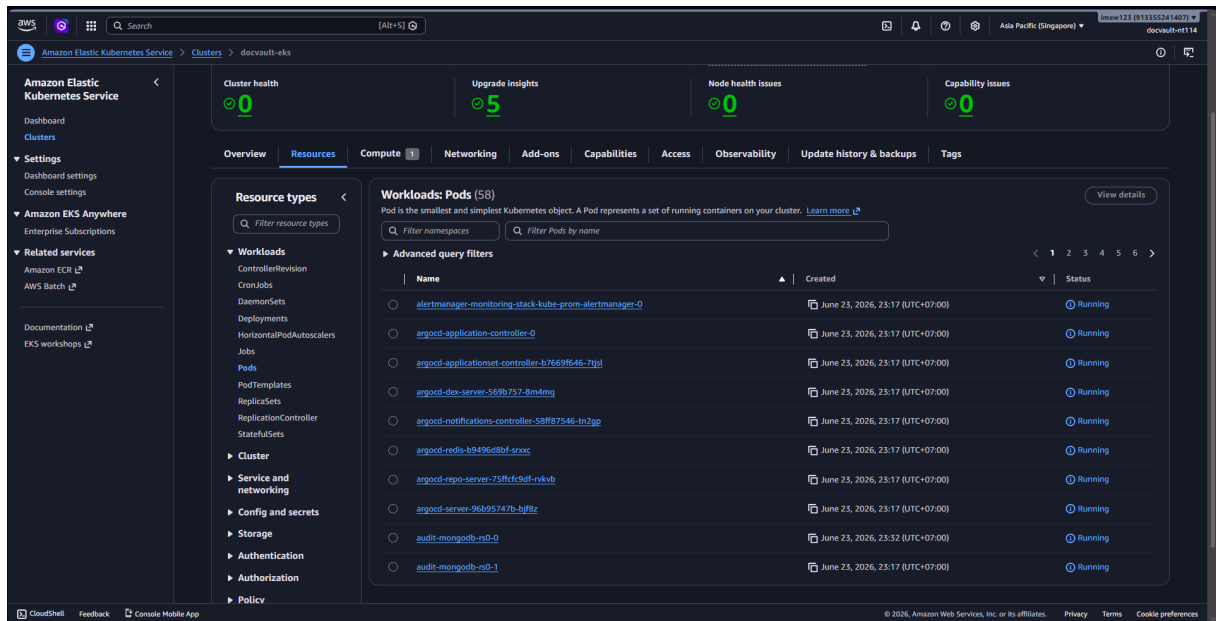
- Cấu hình readiness probe, liveness probe và resource request/limit.
- Mount /tmp bằng `emptyDir` cho tiến trình cần vùng ghi tạm.
- Hỗ trợ image reference theo dạng `repository:tag@sha256:digest`.
- Tạo NetworkPolicy cho từng release.

Các dependency nền tảng được quản lý bằng Kustomize tại `infra/k8s/infra-deps`. Overlay hiện dùng cho EKS triển khai namespace, External Secrets, CloudNativePG, MongoDB và Keycloak; các tài nguyên MinIO được loại khỏi desired state sau khi document-service chuyển sang S3. Cách tách Helm cho application và Kustomize cho dependency giúp giảm lặp cấu hình nhưng vẫn giữ được khả năng tùy biến theo môi trường.

6.3.2 Ingress, DNS và TLS

Lưu lượng public đi qua ingress-nginx. Web được công bố tại `app.docvault.id.vn`, trong khi Keycloak được công bố tại `auth.docvault.id.vn`. cert-manager sử dụng ClusterIssuer `letsencrypt-cloudflare` và DNS-01 challenge để cấp, gia hạn chứng chỉ TLS. Cloudflare quản lý DNS trỏ đến AWS Load Balancer của ingress-nginx. Harbor sử dụng host riêng `harbor.docvault.id.vn` và cấu hình upload không giới hạn kích thước, tắt request buffering và tăng timeout cho thao tác push/pull image.

NetworkPolicy hiện tạo ranh giới truy cập cơ bản cho workload và ingress công khai. Tuy nhiên, egress của chart ứng dụng vẫn được mở để bảo đảm khả năng giao tiếp giữa các dịch vụ trong môi trường kiểm thử. Do đó, cấu hình hiện tại chưa đạt mô hình zero-trust đầy đủ. Giai đoạn tiếp theo cần giới hạn egress theo dịch vụ đích, DNS, cơ sở dữ liệu, S3 endpoint và identity provider [11].



Hình 6.13: Danh sách workload trên cụm EKS và trạng thái vận hành tại thời điểm thu thập bằng chứng

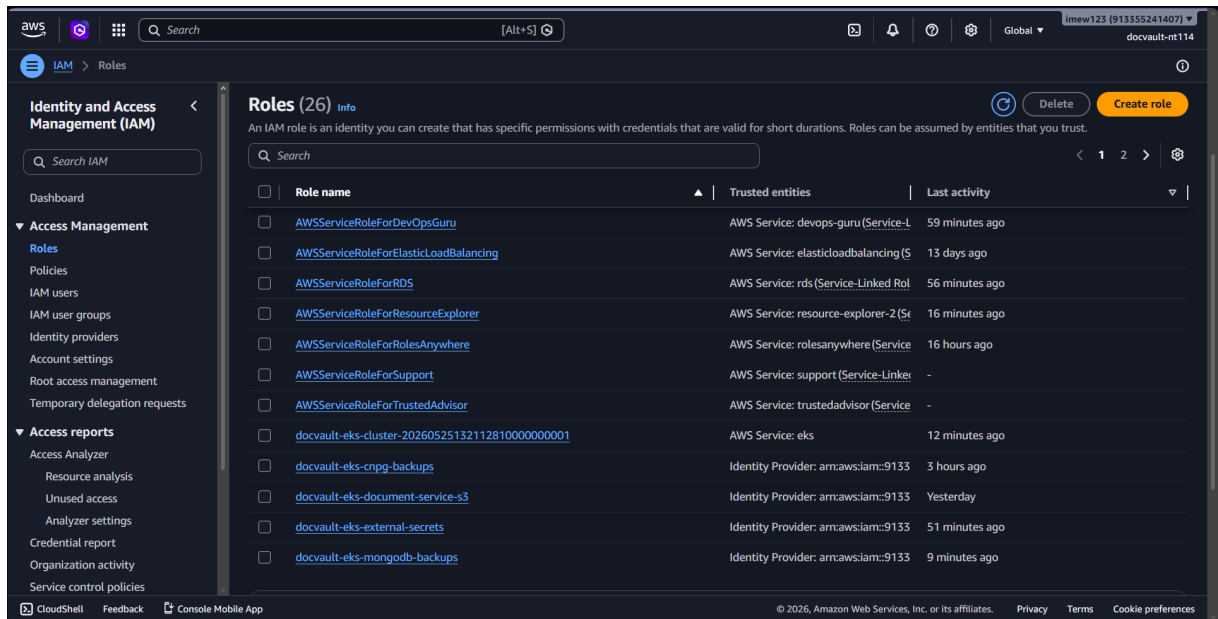
Hình ?? ghi nhận 58 pod được quản lý trên cụm và các workload tiêu biểu của Argo CD, monitoring và cơ sở dữ liệu đang ở trạng thái Running.

6.4 Quản lý secret và quyền truy cập AWS

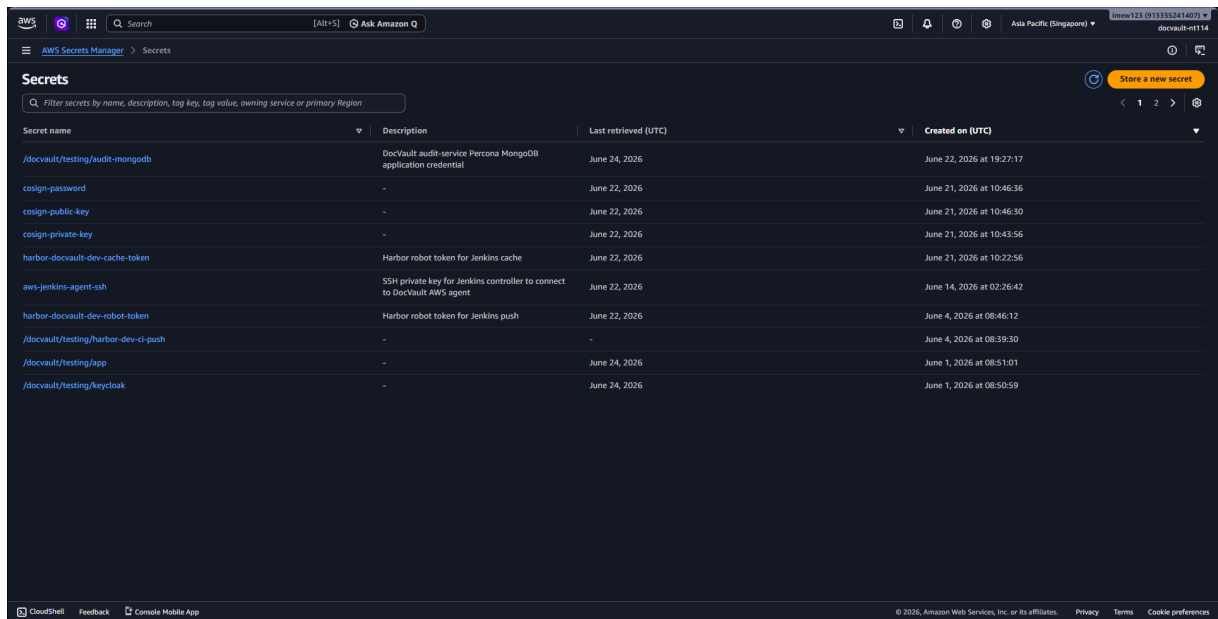
DocVault sử dụng External Secrets Operator làm cầu nối giữa Kubernetes và AWS Secrets Manager. `SecretStore aws-secrets-manager` đọc secret từ khu vực triển khai. ServiceAccount của operator nhận IAM role thông qua IRSA và chỉ có quyền đọc các secret thuộc prefix của môi trường DocVault. Kubernetes Secret tại runtime được tạo từ tài nguyên `ExternalSecret`; vì vậy repository chỉ lưu tham chiếu và cấu trúc ánh xạ, không lưu giá trị bí mật [7], [12].

Pipeline cũng hỗ trợ AWS IAM Roles Anywhere cho Jenkins chạy ngoài AWS. Cơ chế này sử dụng trust anchor, profile và IAM role để Jenkins nhận credential tạm thời bằng chứng thư số thay vì lưu access key dài hạn. Quyền đọc secret của Jenkins được giới hạn theo danh sách tài nguyên cho phép.

Đối với việc kiểm tra trạng thái triển khai, ServiceAccount `jenkins-argocd-reader` chỉ có các quyền `get`, `list` và `watch` trên tài nguyên Argo CD Application. Jenkins không có quyền sửa hoặc xóa application. Thiết kế này áp dụng nguyên tắc đặc quyền tối thiểu cho bước health check sau triển khai.



Hình 6.14: Các IAM role cho document-service, External Secrets và tiến trình backup qua IRSA



Hình 6.15: Các secret của DocVault được quản lý tập trung trong AWS Secrets Manager

Hình ?? thể hiện các nhóm secret dành cho ứng dụng, Keycloak, MongoDB, Harbor, Cosign và Jenkins. Ảnh chỉ hiển thị tên và metadata quản lý, không hiển thị giá trị secret.

6.5 Mô hình GitOps với Argo CD

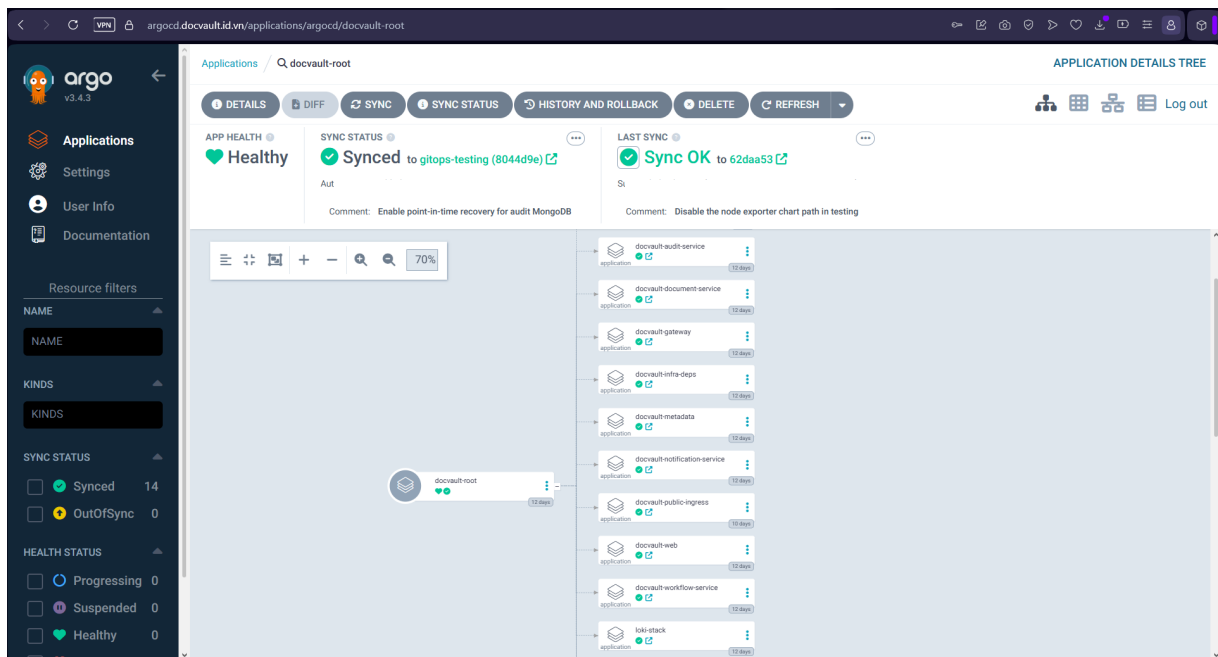
6.5.1 App-of-apps và thứ tự đồng bộ

Argo CD được khởi tạo bằng application gốc `docvault-root`. Application này theo dõi thư mục `infra/argocd-apps` trên nhánh `gitops-testing` và tạo các application con. Mô hình app-of-apps cho phép quản lý thống nhất hạ tầng phụ trợ, dịch vụ ứng dụng, ingress và observability [4].

Thứ tự triển khai được kiểm soát bằng sync wave:

1. Sync wave 0 triển khai dependency hạ tầng qua `docvault-infra-deps`.
2. Sync wave 1 triển khai gateway, web và sáu backend service bằng Helm.
3. Sync wave 2 triển khai public ingress, Prometheus/Grafana và Loki/Promtail.

Các application DocVault bật `selfHeal=true` để Argo CD tự động xử lý sai lệch giữa cụm và Git. `prune=false` được duy trì đối với application nghiệp vụ nhằm giảm nguy cơ xóa ngoài ý muốn trong môi trường kiểm thử. Riêng monitoring và logging bật prune để quản lý toàn bộ vòng đời chart. Khi triển khai production, chính sách prune cần đi kèm quy trình rà soát, backup và cơ chế bảo vệ tài nguyên quan trọng.



Hình 6.16: Application gốc `docvault-root` đạt trạng thái Healthy và Synced

Hình ?? xác nhận application gốc đang theo dõi nhánh `gitops-testing`, đã đồng bộ revision Git và quản lý các application con của DocVault. Thông tin định danh cá nhân trong lịch sử commit đã được che.

6.5.2 Cập nhật desired state

Sau khi image được push, Jenkins lấy digest từ registry và cập nhật ba trường **repository**, **tag**, **digest** trong Helm values tương ứng. Tag mặc định là 12 ký tự đầu của Git commit SHA. Pipeline clone riêng nhánh GitOps, commit thay đổi với `[skip ci]` để tránh vòng lặp và retry push tối đa ba lần nếu có cập nhật đồng thời.

Nếu thay đổi chỉ liên quan đến manifest Kubernetes, pipeline đồng bộ thư mục `infra/k8s` sang nhánh GitOps nhưng giữ lại image reference đang chạy, nên thay đổi cấu hình hạ tầng không vô tình hạ phiên bản image. Argo CD sau đó phát hiện commit mới và reconcile cluster về desired state.

6.6 Harbor và bảo mật chuỗi cung ứng phần mềm

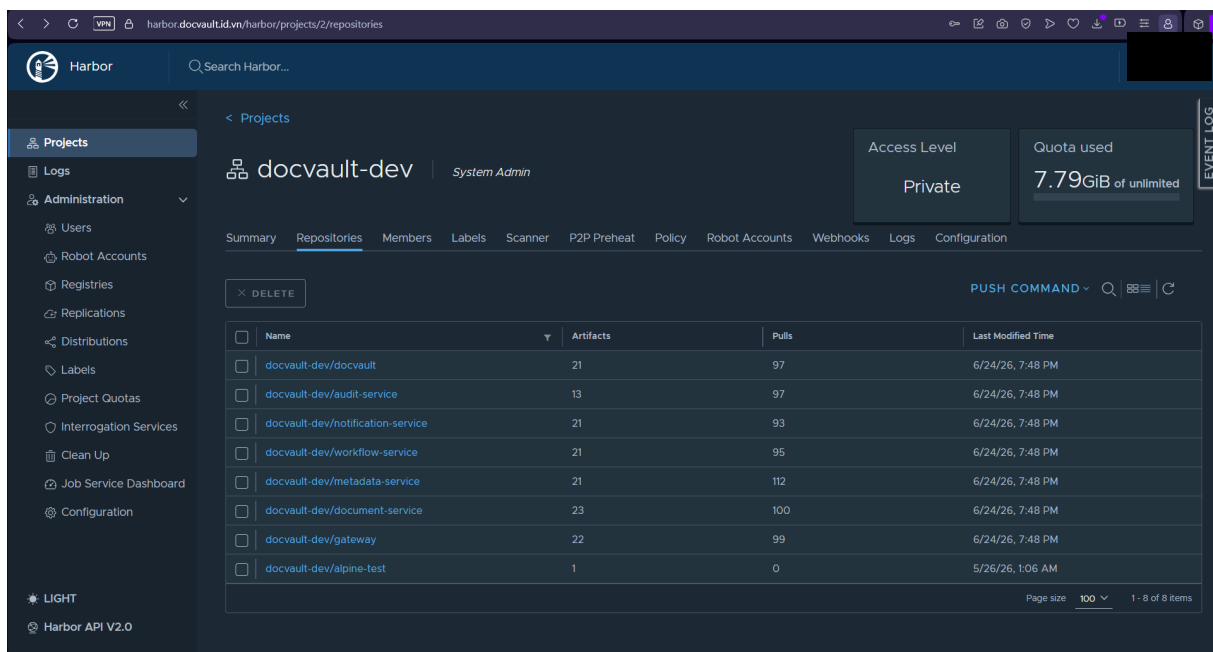
Harbor được triển khai trên EKS với vai trò private container registry của DocVault. Registry sử dụng PersistentVolume gp3, TLS, robot account dành cho Jenkins và image pull secret dành cho workload. Trivy scanner tích hợp hỗ trợ quản lý kết quả quét lỗ hổng của artifact lưu trong registry [13].

Pipeline mặc định không đẩy tag `latest`; mỗi image được định danh bằng Git SHA và digest. BuildKit registry cache sử dụng tag riêng `buildcache` để rút ngắn thời gian build nhưng tag này không được sử dụng khi triển khai. Sau khi push, Jenkins ký image digest bằng Cosign khi tham số `SIGN_IMAGES` được bật và có thể xác minh lại chữ ký bằng public key đã cấu hình [14]. Bằng chứng Harbor thu thập trong đề tài cho thấy nhiều artifact đã có trạng thái ký và có liên kết `SBOM details`.

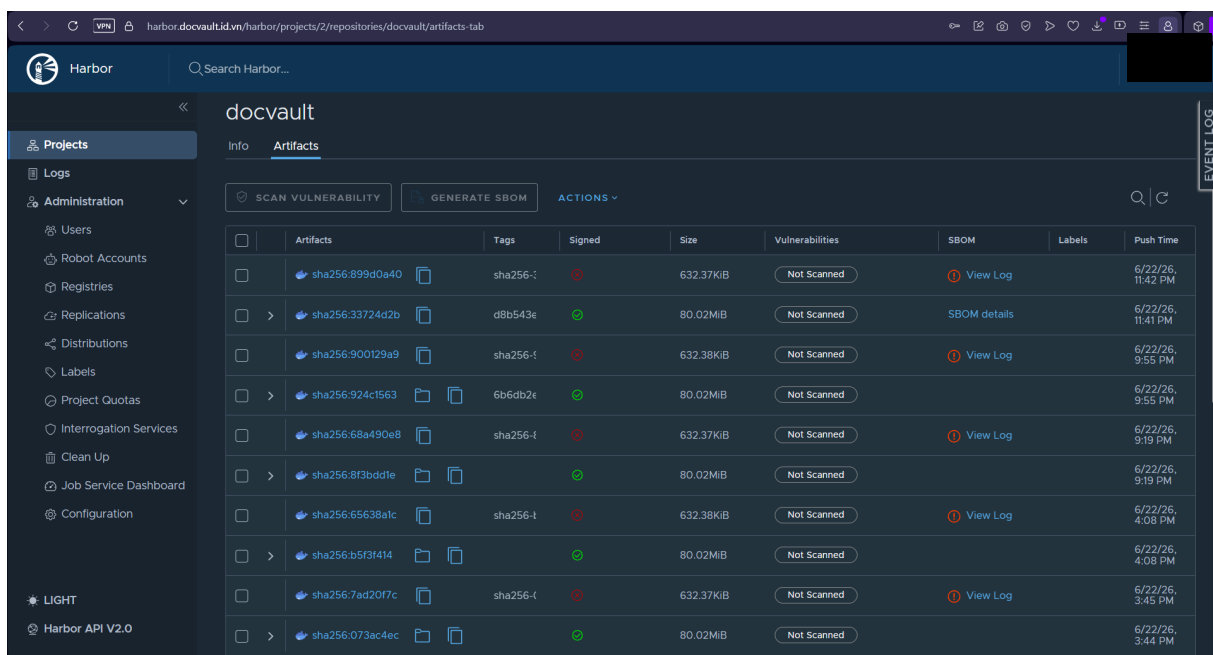
Policy-as-code yêu cầu workload DocVault:

- chỉ dùng image từ project DocVault trên Harbor;
- pin image bằng SHA-256 digest;
- khai báo Harbor image pull secret;
- không chạy privileged hoặc root;
- dùng read-only root filesystem, tắt privilege escalation và drop capability;
- có CPU/memory request và limit;
- không dùng tag `latest`;
- không commit Kubernetes Secret có `data` hoặc `stringData`.

Các policy Kyverno được chạy bằng Kyverno CLI trong Jenkins trên manifest đã render. Báo cáo kết quả hiện có ghi nhận 85 lượt kiểm tra đạt, không có trường hợp không đạt, cảnh báo hoặc lỗi. Đây là cổng kiểm soát trước triển khai [15]. Việc cài Kyverno admission controller trong cụm và xác minh chữ ký Cosign tại thời điểm tiếp nhận tài nguyên là bước gia cố tiếp theo để kiểm soát cả các thay đổi được tạo ngoài pipeline.



Hình 6.17: Project riêng docvault-dev và các repository dịch vụ trên Harbor



Hình 6.18: Artifact Harbor được định danh bằng digest, có trạng thái ký và thông tin SBOM

Hình ?? xác nhận các image của web, gateway và microservice được lưu trong project riêng. Hình ?? cung cấp bằng chứng về digest, chữ ký và SBOM của artifact. Một số artifact trong ảnh vẫn hiển thị trạng thái Not Scanned; vì vậy hình này không được sử dụng để kết luận rằng toàn bộ artifact đã đạt cổng vulnerability scan.

6.7 Jenkins pipeline

6.7.1 Phân tách CI và CD

Pipeline chính nằm trong `Jenkinsfile`, còn logic tái sử dụng được tổ chức thành Jenkins Shared Library trong thư mục `vars`. Pipeline phân biệt hai chế độ:

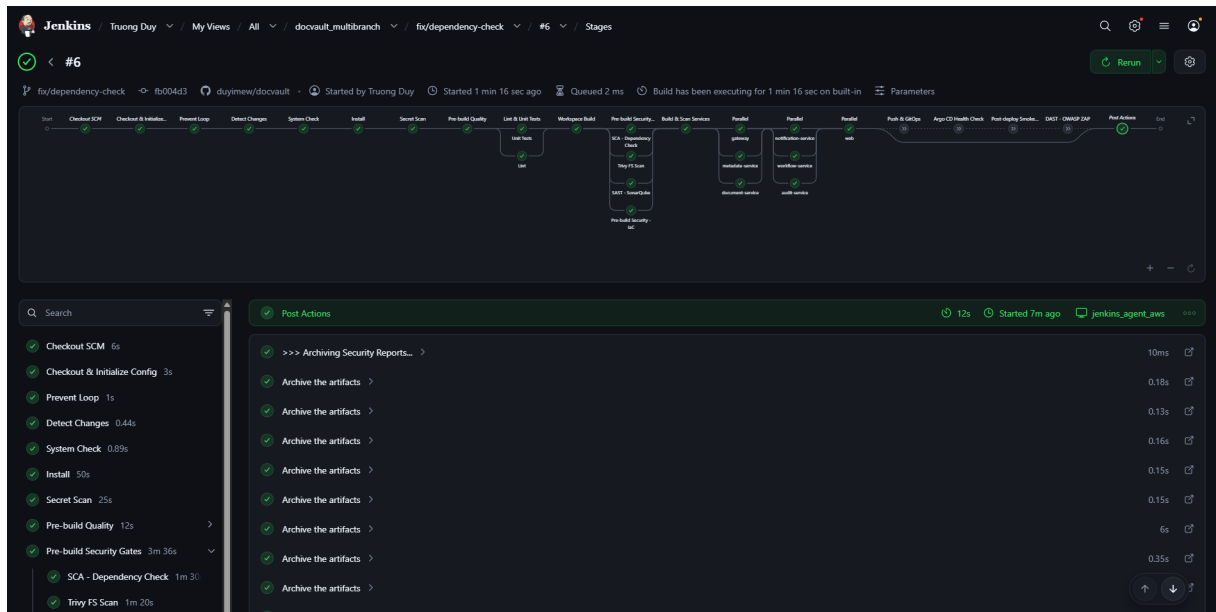
- **CI validation:** chạy trên branch phát triển hoặc pull request, thực hiện secret scan, lint, unit test, workspace build và các security gate.
- **CD release:** chỉ branch tin cậy được khai báo bởi `RELEASE_BRANCH` mới được push image và cập nhật GitOps.

Cách phân tách này ngăn branch không tin cậy phát hành artifact. Trước khi chọn stage, pipeline xác định file thay đổi để chỉ chạy phần liên quan. Thay đổi ở service chỉ build service bị ảnh hưởng; thay đổi ở shared library, lockfile hoặc Dockerfile có thể kích hoạt nhiều image. Khi không xác định được diff range tin cậy, pipeline build toàn bộ để ưu tiên tính đúng đắn.

6.7.2 Các stage chính

Luồng pipeline hiện tại gồm:

1. **Checkout & Initialize Config:** checkout source, hợp nhất parameter và xác định chế độ CI/CD.
2. **Prevent Loop:** bỏ qua commit GitOps có `[skip ci]`.
3. **Detect Changes:** phân loại thay đổi ứng dụng, bảo mật, IaC và image build.
4. **System Check:** kiểm tra agent và công cụ cần thiết.
5. **Install:** cài dependency bằng `pnpm` và sinh Prisma client khi cần.
6. **Secret Scan:** dùng TruffleHog phát hiện credential bị commit.
7. **Pre-build Quality:** chạy unit test và lint song song, sau đó build toàn workspace.
8. **Pre-build Security Gates:** chạy SCA, Trivy filesystem, SonarQube và kiểm tra IaC song song.
9. **Build & Scan Services:** build image theo batch, dùng BuildKit cache và quét image bằng Trivy.
10. **Push & GitOps:** push Harbor, lấy digest, ký image tùy chọn và cập nhật nhánh GitOps.
11. **Argo CD Health Check:** đợi application đạt `Synced` và `Healthy`.
12. **Post-deploy Smoke Test:** kiểm tra web root và API health.
13. **DAST – OWASP ZAP:** quét ứng dụng đang chạy và lưu report.
14. **Post Cleanup:** archive artifact và dọn workspace.



Hình 6.19: Jenkins build và Stage View

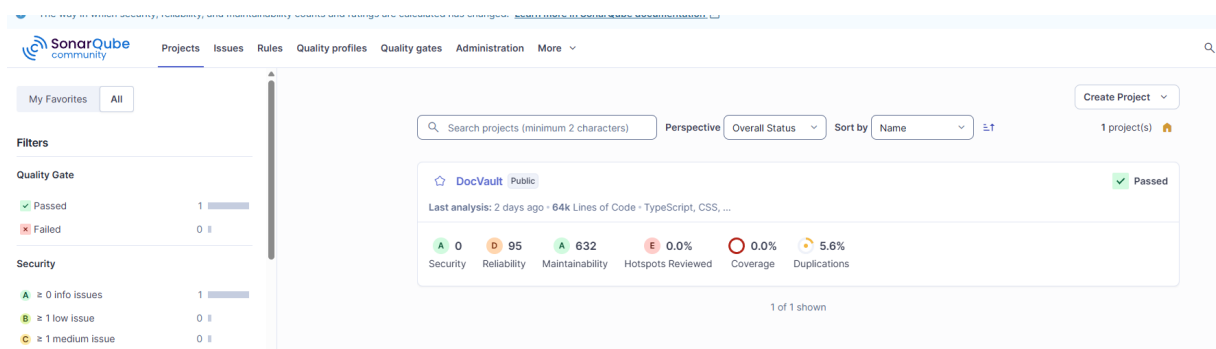
6.8 Các công kiểm soát chất lượng và bảo mật

6.8.1 Secret scanning

TruffleHog quét filesystem repository trước các bước build. Các thư mục dependency, cache, Terraform working directory và file state/plan được loại trừ để giảm nhiễu. Nếu phát hiện secret, stage fail và report được archive. Cơ chế này giảm nguy cơ credential bị đóng gói vào image hoặc lưu trong lịch sử phát hành.

6.8.2 Kiểm thử và chất lượng mã nguồn

Unit test và lint chạy song song để giảm thời gian phản hồi; workspace build chạy sau đó để kiểm tra khả năng biên dịch của toàn monorepo. SonarQube phân tích source trong apps, services và libs. Tham số ENFORCE_SONAR_QG mặc định bật, vì vậy pipeline fail nếu Quality Gate không đạt hoặc hết thời gian chờ [16].



Hình 6.20: SonarQube Quality Gate của DocVault

6.8.3 SCA và filesystem scanning

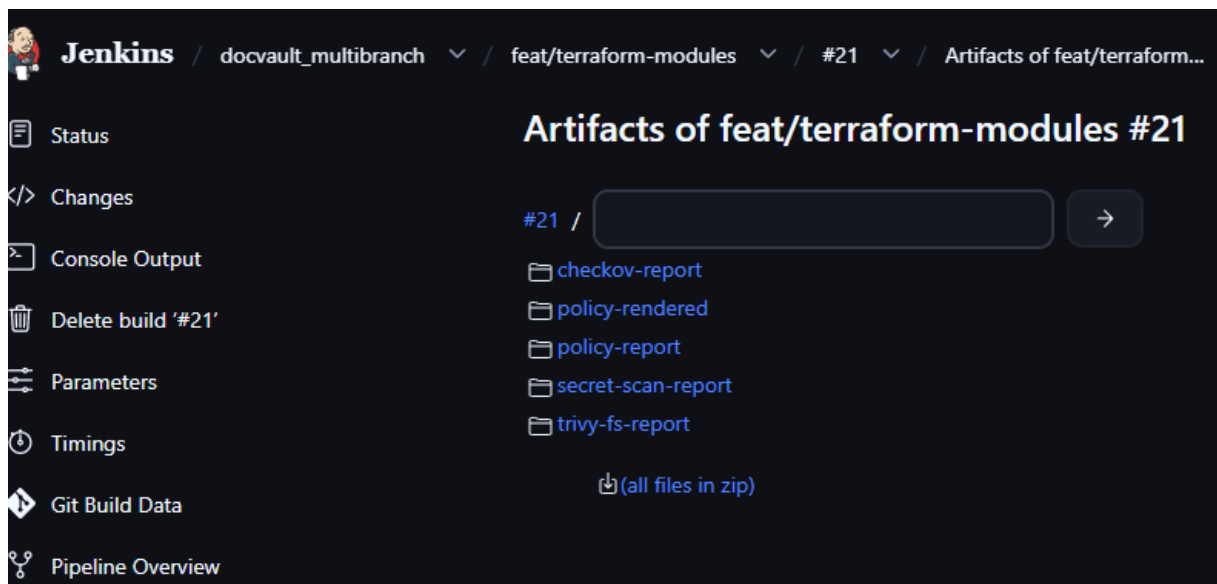
OWASP Dependency-Check phân tích dependency và tạo report HTML/JSON/log. Chính sách kỹ thuật đặt ngưỡng fail ở CVSS từ 7 trở lên. Tuy nhiên tham số `ALLOW_DEPENDENCY_CHECK_FAILURE` cho phép tạm đánh dấu build `UNSTABLE` thay vì fail, phục vụ trường hợp NVD database hoặc dependency scanner không ổn định. Mọi ngoại lệ phải được ghi nhận và không được xem là kết quả đạt hoàn toàn.

Trivy filesystem quét vulnerability, secret và misconfiguration trên snapshot sạch tạo từ Git. Các finding `HIGH/CRITICAL` làm fail gate. Phạm vi misconfiguration tập trung vào Dockerfile, Kubernetes và Helm; Terraform được kiểm tra riêng bằng Checkov và Terraform Validate để tránh finding trùng từ module tải về [17].

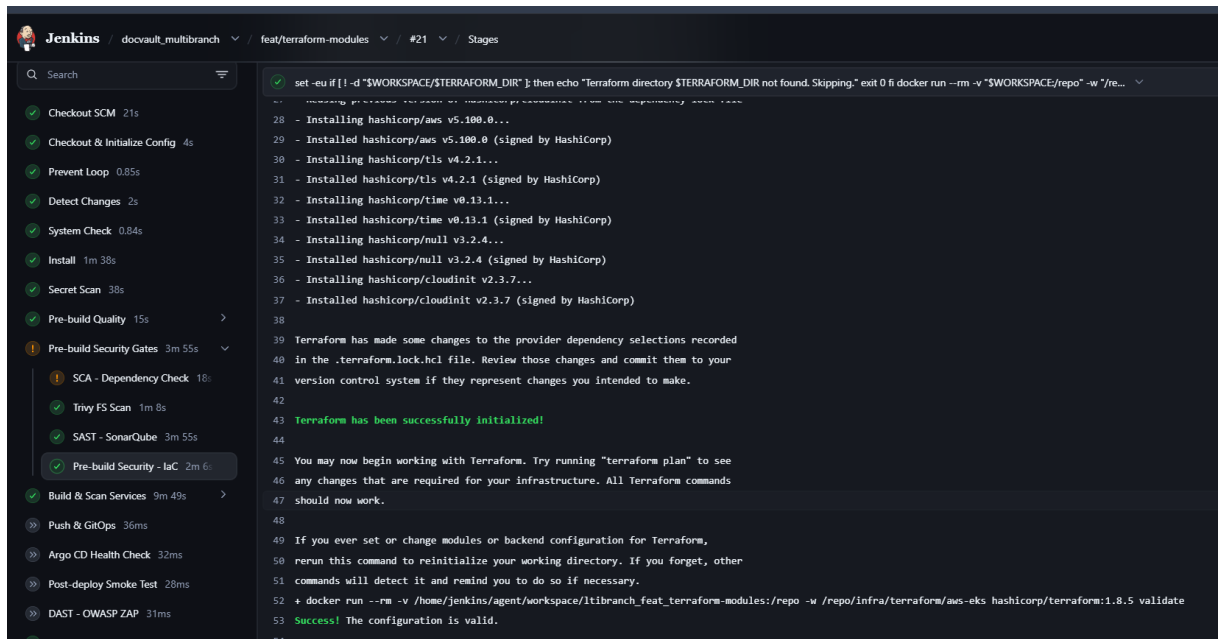
6.8.4 IaC và policy-as-code

Checkov quét Terraform, Kubernetes và Helm để phát hiện cấu hình rủi ro [18]. Terraform Validate chạy `fmt -check, init -backend=false` và `validate`. Kyverno CLI render toàn bộ Helm values thực tế, sau đó áp dụng policy lên manifest Kubernetes trước khi image được phát hành [15].

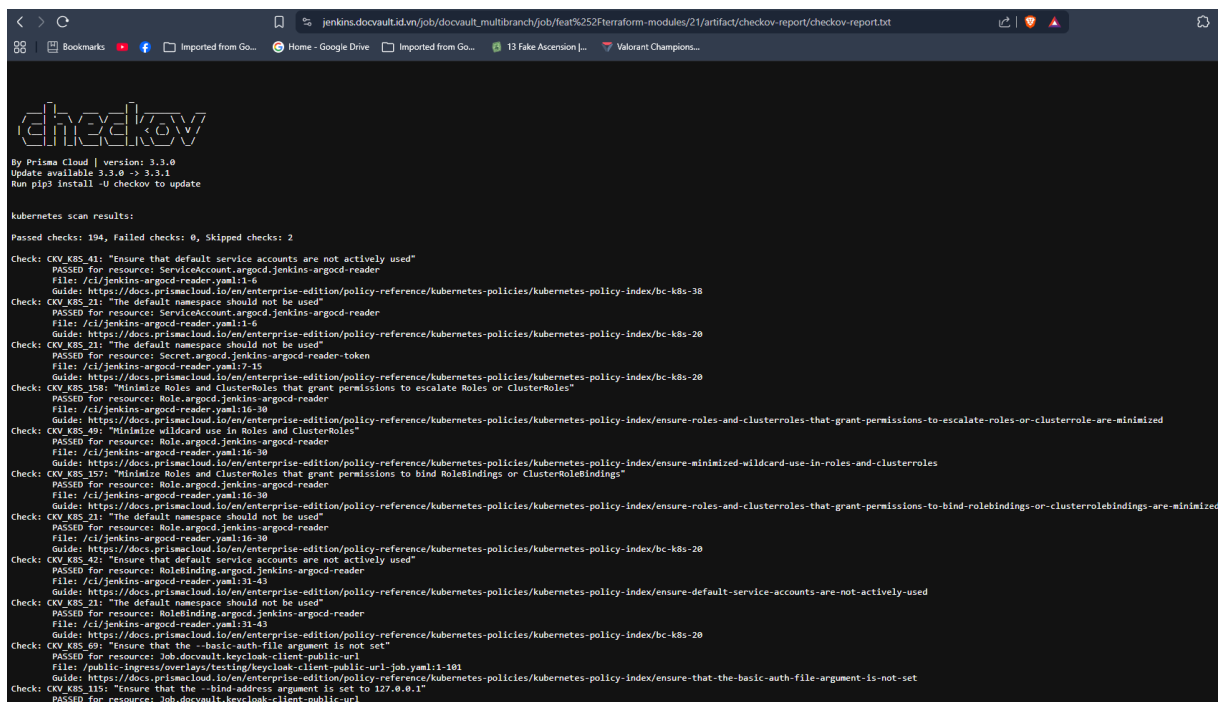
Ba lớp kiểm tra bổ sung cho nhau: Terraform Validate kiểm tra tính hợp lệ cú pháp và provider schema; Checkov kiểm tra best practice của hạ tầng; Kyverno kiểm tra policy runtime trên manifest cuối cùng. Artifact gồm Checkov report, Kyverno report và manifest đã render.



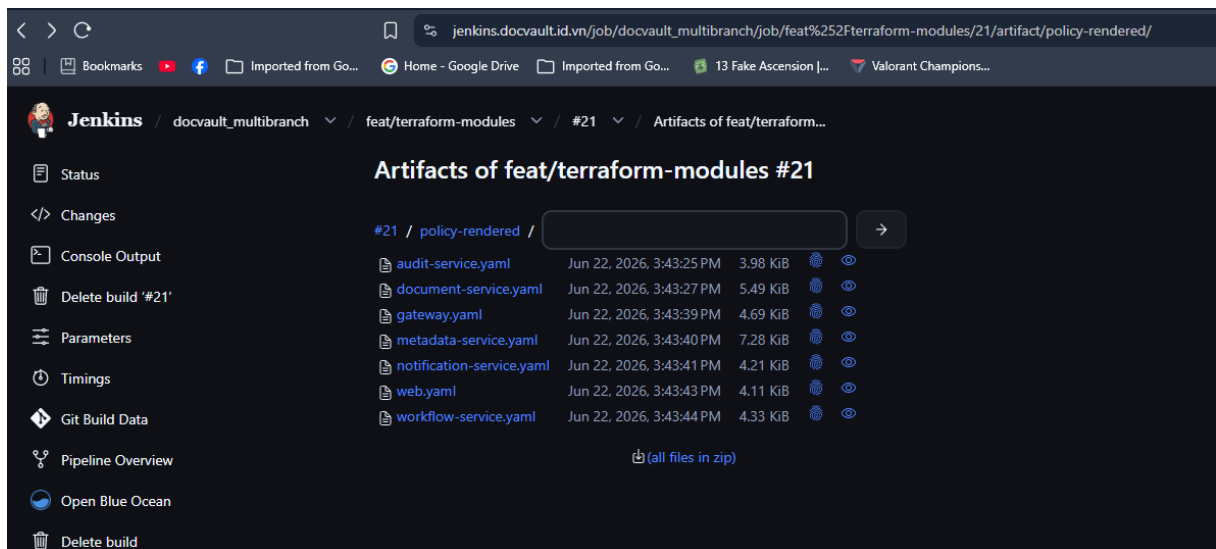
Hình 6.21: Log và artifact cho ba bước IaC



Hình 6.22: Kết quả Terraform Validate



Hình 6.23: Kết quả Checkov



Hình 6.24: Kết quả Kyverno

6.9 Build, quét và phát hành container image

Backend sử dụng `Dockerfile.backend` với build argument xác định service; frontend sử dụng `Dockerfile` riêng trong `apps/web`. Các image được build theo batch để tận dụng song song nhưng vẫn giới hạn tải trên Jenkins agent. BuildKit lấy và cập nhật cache trong Harbor, giúp giảm thời gian build lại dependency không đổi.

Sau build, Trivy quét từng image và fail nếu có lỗi hỏng CRITICAL. Chỉ các image vượt qua gate mới được push. Pipeline đăng nhập registry bằng credential tạm trong thư mục Docker config riêng và xóa sau khi hoàn tất. Image triển khai không phụ thuộc tag mutable mà được cập nhật cả tag Git SHA lẫn registry digest.

So với triển khai chỉ dựa trên tag, digest pin bảo đảm Kubernetes kéo đúng artifact đã được quét. Nếu cùng một tag bị ghi đè, digest trong Helm values vẫn không thay đổi. Đây là kiểm soát quan trọng đối với tính toàn vẹn của chuỗi cung ứng phần mềm.

6.10 Kiểm tra sau triển khai

6.10.1 Argo CD health gate

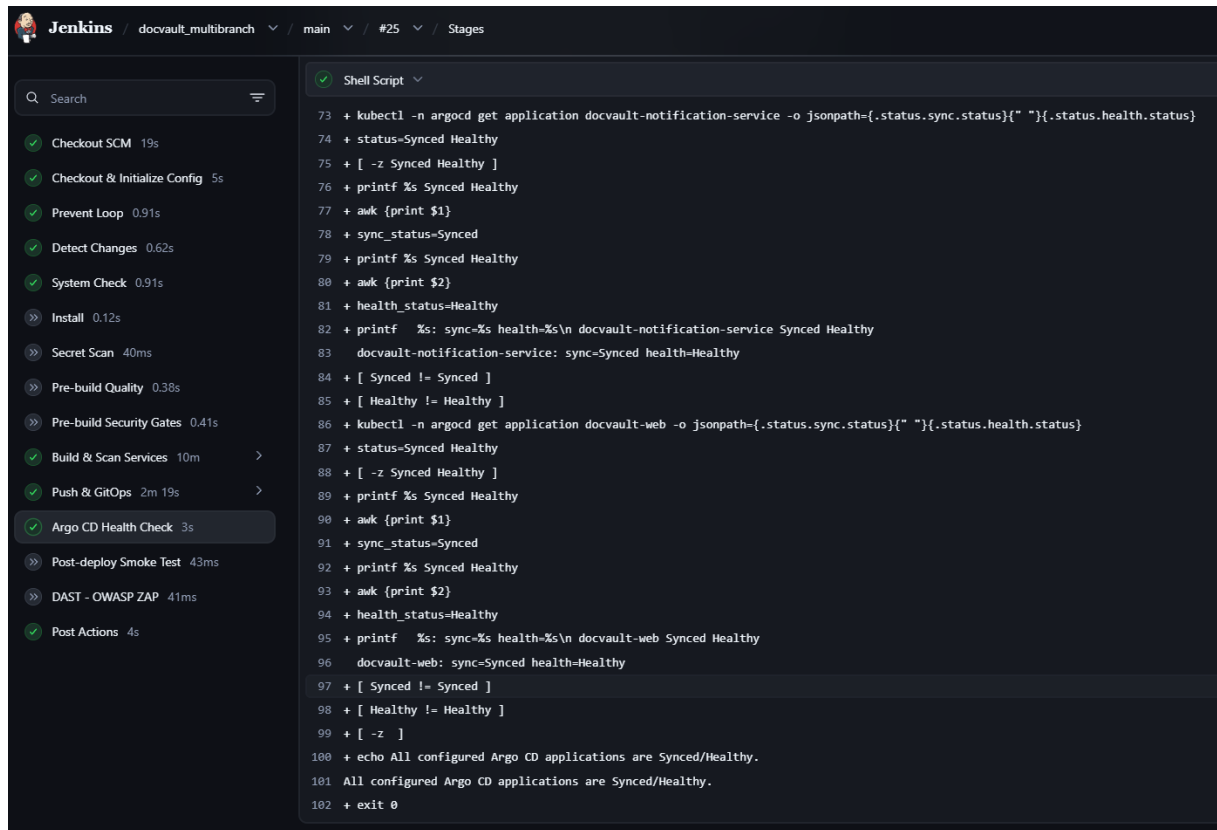
Khi `RUN_ARGO_HEALTH_CHECK=true`, Jenkins dùng kubeconfig credential hoặc context hiện tại để đọc các Argo CD Application. Pipeline chờ đến khi gateway, metadata, document, workflow, audit, notification và web đồng thời đạt **Synced** và **Healthy**. Nếu quá timeout, release fail. Vai trò của Argo CD ở đây vượt ra ngoài việc thực hiện CD: nó còn cung cấp trạng thái reconcile có thể kiểm chứng [4].

6.10.2 Smoke test

Smoke test gọi web root và endpoint `/api/health`. Web root chấp nhận HTTP 2xx hoặc 3xx, còn health endpoint yêu cầu 2xx. Mỗi request được retry để hấp thụ độ trễ rollout ngắn. Bước này không thay thế E2E nghiệp vụ, nhưng phát hiện nhanh lỗi ingress, service routing, container startup hoặc cấu hình runtime.

6.10.3 DAST bằng OWASP ZAP

OWASP ZAP baseline scan chạy khi `RUN_ZAP=true` và target có thể truy cập từ Jenkins. Pipeline tạo report HTML và JSON, archive làm bằng chứng, đồng thời fail khi report chứa finding High/Critical. Cảnh báo mức thấp hơn được giữ để review thay vì bỏ qua [19].



```
73 + kubectl -n argocd get application docvault-notification-service -o jsonpath={.status.sync.status}{ " "}{.status.health.status}
74 + status=Synced Healthy
75 + [ -z Synced Healthy ]
76 + printf %s Synced Healthy
77 + awk {print $1}
78 + sync_status=Synced
79 + printf %s Synced Healthy
80 + awk {print $2}
81 + health_status=Healthy
82 + printf %s: sync=%s health=%s\n docvault-notification-service Synced Healthy
83   docvault-notification-service: sync=Synced health=Healthy
84 + [ Synced != Synced ]
85 + [ Healthy != Healthy ]
86 + kubectl -n argocd get application docvault-web -o jsonpath={.status.sync.status}{ " "}{.status.health.status}
87 + status=Synced Healthy
88 + [ -z Synced Healthy ]
89 + printf %s Synced Healthy
90 + awk {print $1}
91 + sync_status=Synced
92 + printf %s Synced Healthy
93 + awk {print $2}
94 + health_status=Healthy
95 + printf %s: sync=%s health=%s\n docvault-web Synced Healthy
96   docvault-web: sync=Synced health=Healthy
97 + [ Synced != Synced ]
98 + [ Healthy != Healthy ]
99 + [ -z ]
100 + echo All configured Argo CD applications are Synced/Healthy.
101 All configured Argo CD applications are Synced/Healthy.
102 + exit 0
```

Hình 6.25: Kết quả Argo CD health check

Jenkins / docvault_multibranch / main / #44 / Stages

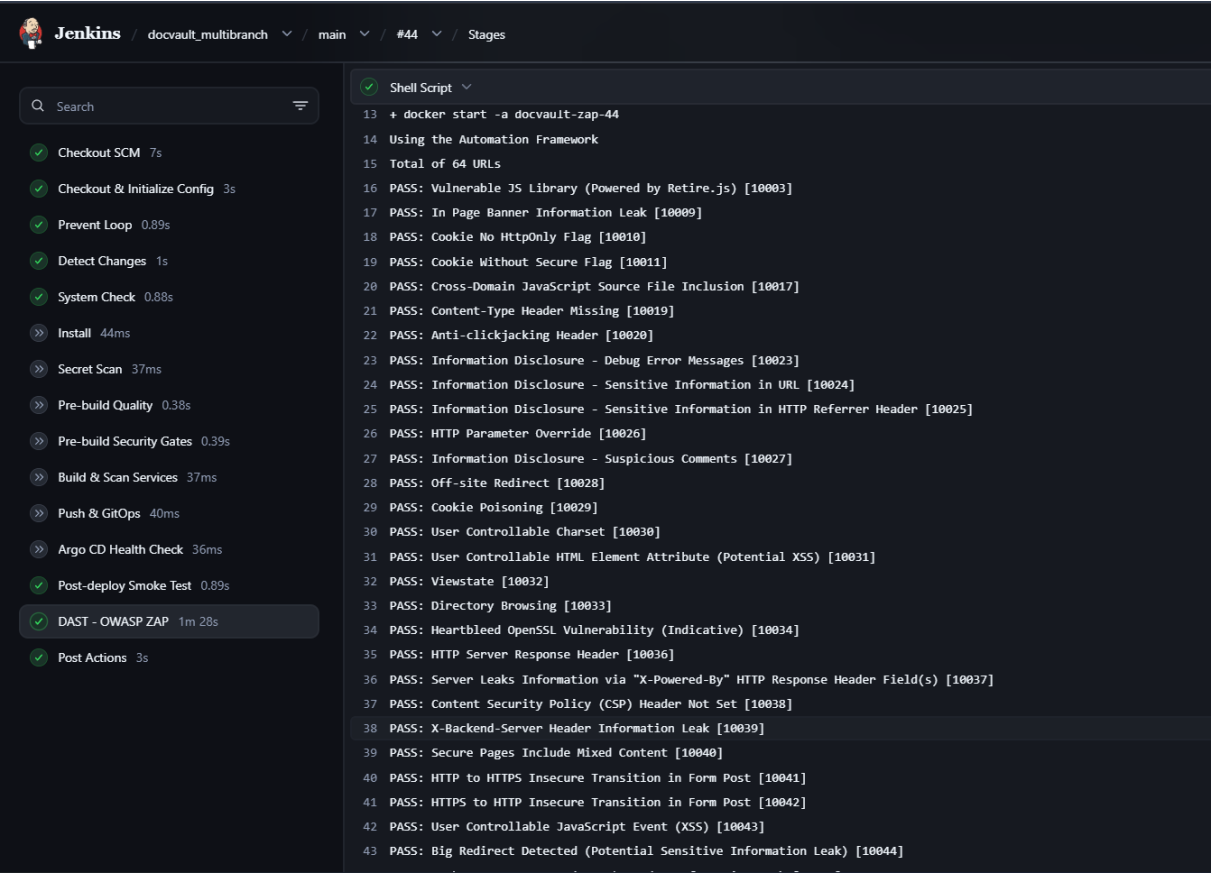
Search

- ✓ Checkout SCM 7s
- ✓ Checkout & Initialize Config 3s
- ✓ Prevent Loop 0.89s
- ✓ Detect Changes 1s
- ✓ System Check 0.88s
- » Install 44ms
- » Secret Scan 37ms
- » Pre-build Quality 0.38s
- » Pre-build Security Gates 0.39s
- » Build & Scan Services 37ms
- » Push & GitOps 40ms
- » Argo CD Health Check 36ms
- ✓ Post-deploy Smoke Test 0.89s
- ✓ DAST - OWASP ZAP 1m 28s
- ✓ Post Actions 3s

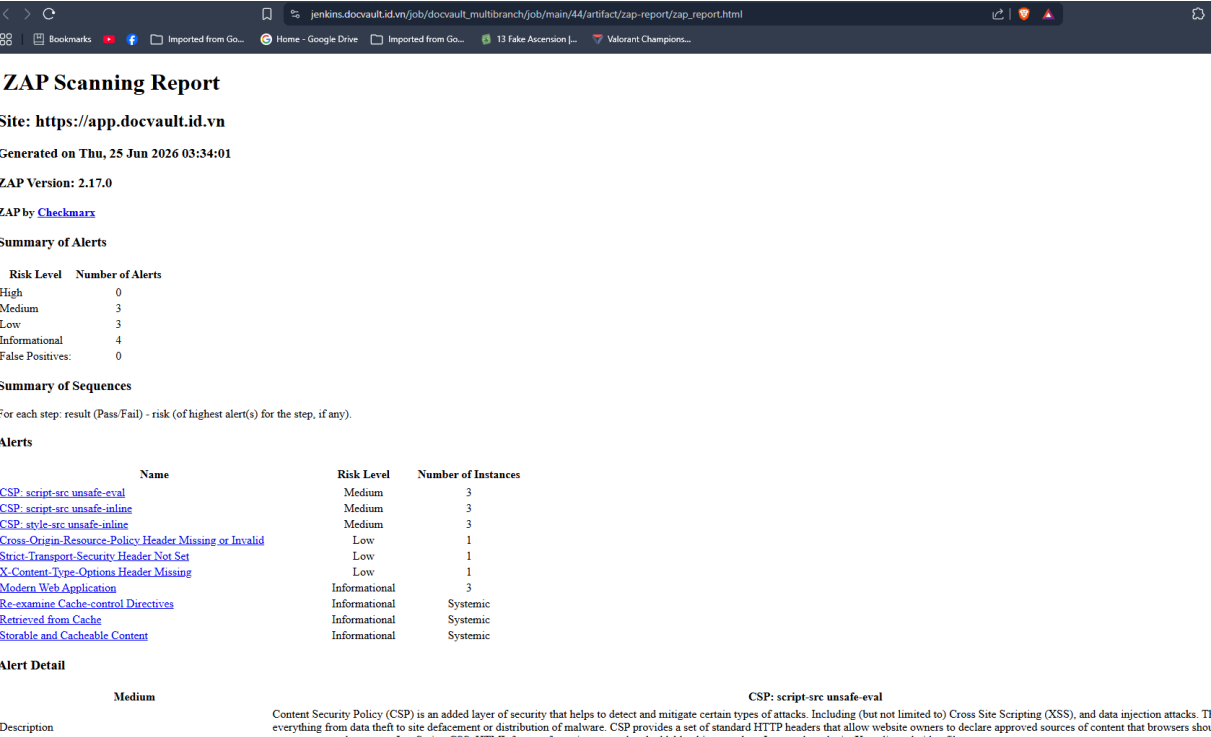
Shell Script

```
0 + set -eu
1 + check_url web-root https://app.docvault.id.vn root
2 + name=web-root
3 + url=https://app.docvault.id.vn
4 + expected=root
5 + attempt=1
6 + [ 1 -le 30 ]
7 + curl -sS -o /dev/null -w %{http_code} --max-time 20 https://app.docvault.id.vn
8 + status=200
9 + echo web-root attempt 1/30: https://app.docvault.id.vn -> HTTP 200
10 web-root attempt 1/30: https://app.docvault.id.vn -> HTTP 200
11 + [ root = root ]
12 + return 0
13 + check_url api-health https://app.docvault.id.vn/api/health health
14 + name=api-health
15 + url=https://app.docvault.id.vn/api/health
16 + expected=health
17 + attempt=1
18 + [ 1 -le 30 ]
19 + curl -sS -o /dev/null -w %{http_code} --max-time 20 https://app.docvault.id.vn/api/health
20 + status=200
21 + echo api-health attempt 1/30: https://app.docvault.id.vn/api/health -> HTTP 200
22 api-health attempt 1/30: https://app.docvault.id.vn/api/health -> HTTP 200
23 + [ health = root ]
24 + [ health = health ]
25 + return 0
26 + echo Post-deploy smoke test passed.
27 Post-deploy smoke test passed.
```

Hình 6.26: Log post deploy smoke test



Hình 6.27: Log OWASP ZAP



Hình 6.28: Kết quả OWASP ZAP

6.11 Vận hành: giám sát, log và khôi phục

Phần này gom các năng lực vận hành của hệ thống sau khi triển khai: giám sát số liệu, tập trung log và khả năng khôi phục dữ liệu. Hai năng lực đầu được trình bày ngay dưới đây; năng lực khôi phục dựa trên cơ chế sao lưu cơ sở dữ liệu đã mô tả ở Mục ??.

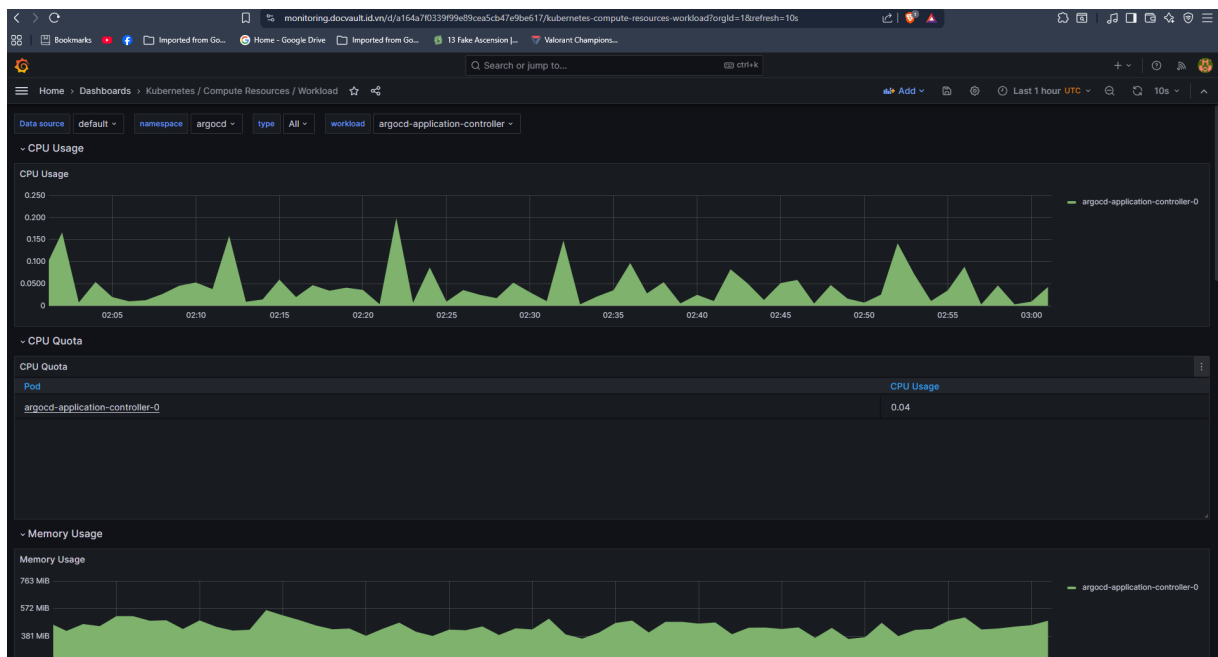
6.11.1 Giám sát và tập trung log

Khả năng quan sát được triển khai qua hai Argo CD Application trong namespace `monitoring`. `kube-prometheus-stack` cung cấp Prometheus, Grafana và Alertmanager; `loki-stack` cung cấp Loki và Promtail. Promtail thu thập log container trong cụm, còn Grafana sử dụng cả Prometheus và Loki làm nguồn dữ liệu [20], [21].

Các nhóm thông tin cần theo dõi gồm:

- Trạng thái pod, deployment, node và số lần restart.
- CPU, memory, saturation và resource request/limit.
- Lỗi HTTP, latency và health endpoint của gateway/service.
- Log theo namespace, pod, container, request ID và thời gian.
- Cảnh báo về pod crash, thiếu tài nguyên, volume hoặc endpoint không sẵn sàng.

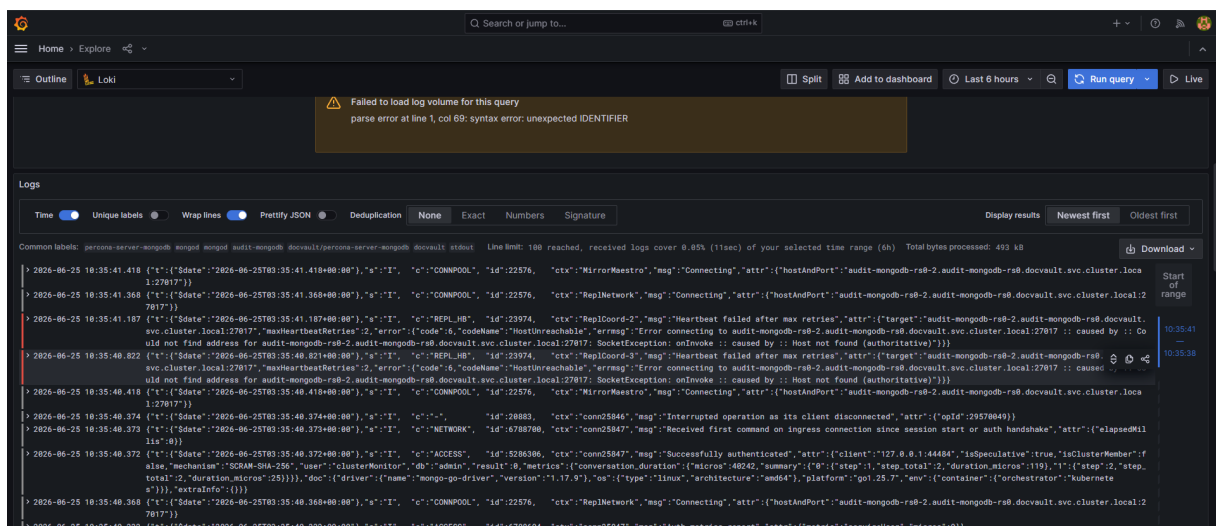
Cấu hình hiện tại đáp ứng nhu cầu trình diễn và thu thập bằng chứng. Tuy nhiên, Loki chưa bật persistence và thông tin quản trị Grafana cần được quản lý hoàn toàn qua External Secrets trước khi vận hành dài hạn. Hệ thống cũng cần bổ sung dashboard nghiệp vụ, alert rule, chính sách retention và kênh tiếp nhận cảnh báo để đáp ứng yêu cầu production.



Hình 6.29: Grafana dashboard giám sát workload DocVault trong namespace `docvault`



Hình 6.30: Grafana dashboard giám sát EKS cluster



Hình 6.31: Loki tập trung log của namespace DocVault

6.11.2 Khôi phục dữ liệu và sẵn sàng vận hành

Giám sát và log cho biết hệ thống đang chạy ra sao; khôi phục quyết định chuyện gì xảy ra khi nó hỏng. Hai mảnh ghép này cùng tạo nên năng lực vận hành. Cơ chế sao lưu ở Mục ?? đã đặt nền cho khôi phục: PostgreSQL có base backup hằng ngày kèm WAL liên tục, MongoDB có backup hằng ngày kèm PITR qua oplog, cả hai đều ghi vào S3 mã hóa KMS.

Phần còn thiếu để khép vòng là diễn tập khôi phục định kỳ. Backup chỉ có giá trị khi đã chứng minh khôi phục được, nên bước tiếp theo là khôi phục vào môi trường cô lập, đối chiếu số bản ghi và index, chạy lại xác minh chuỗi băm audit, rồi ghi nhận RPO và RTO đo được. Cho tới khi có kết quả diễn tập, các tham số như chu kỳ oplog 10 phút

chỉ phản ánh năng lực đã cấu hình. Tương tự, Loki chưa bật persistence nên log lịch sử chưa bền vững qua restart, và alert rule nghiệp vụ vẫn cần bổ sung trước khi vận hành dài hạn.

6.12 Mapping kiểm soát, công cụ và bằng chứng

Bảng 6.1: Mapping kiểm soát DevSecOps với công cụ và bằng chứng

Khu vực	Công cụ	Bằng chứng chính	Điều kiện không đạt
Secret scanning	TruffleHog	Report và log Jenkins	Phát hiện secret xác thực
Quality	pnpm, Jest, ESLint, Turbo	Unit test, lint và build log	Test, lint hoặc build lỗi
SAST	SonarQube	Dashboard và Quality Gate	Quality Gate fail/timeout
SCA	Dependency-Check	HTML/JSON/log	CVSS vượt ngưỡng hoặc ngoại lệ chưa duyệt
Filesystem scan	Trivy	JSON report	HIGH/CRITICAL finding
IaC syntax	Terraform	Fmt/validate log	Terraform không hợp lệ
IaC security	Checkov	Text report	Policy nghiêm trọng fail
Policy-as-code	Kyverno CLI	Policy report và manifest render	Workload vi phạm policy
Container image	Docker Build-Kit, Trivy	Build log, image scan	Build lỗi hoặc CRITICAL CVE
Registry	Harbor	Repository, tag, digest, scan	Push lỗi hoặc artifact thiếu digest
Artifact integrity	in-Cosign	Sign/verify log	Ký hoặc xác minh thất bại khi được bật
GitOps	Git, Argo CD	GitOps commit, revision, sync status	Push lỗi hoặc app OutOf-Sync/Degraded

Khu vực	Công cụ	Bằng chứng chính	Điều kiện không đạt
Post-deploy	Smoke test, ZAP	HTTP log, HTML/JSON report	Target lỗi hoặc High/Critical finding
Observability	Prometheus, Grafana, Loki	Dashboard, alert và log query	Mất metric/log hoặc workload unhealthy
Cloud security	IRSA, Secrets Manager, KMS	IAM role, ExternalSecret, encrypted bucket	Quyền quá rộng hoặc secret bị lộ

6.13 Đánh giá trạng thái triển khai

Kết quả đối chiếu mã nguồn, cấu hình hạ tầng và ảnh chụp thực nghiệm cho thấy DocVault đã triển khai các thành phần cốt lõi của quy trình DevSecOps: IaC cho EKS, các cổng kiểm soát trong CI, private registry, image digest, SBOM, ký artifact, GitOps, kiểm tra sau triển khai, quản lý bí mật, thu thập metric/log và backup cơ sở dữ liệu. Tài liệu nội bộ trước đây ghi nhận Jenkins build #61 đã chạy qua các stage chính; tuy nhiên pipeline hiện tại đã được mở rộng thêm quét secret, phát hiện phạm vi thay đổi, Quality Gate, Kyverno, Harbor, digest pin và Cosign. Do đó, bộ bằng chứng Jenkins cuối cùng cần được thu thập từ một build mới phản ánh đúng phiên bản mã nguồn hiện tại.

Các giới hạn cần nêu rõ gồm:

- Môi trường EKS ưu tiên chi phí kiểm thử; worker node công khai và phạm vi CIDR rộng của API chỉ nên được sử dụng tạm thời.
- Dependency-Check có chế độ ngoại lệ mềm; build UNSTABLE không được xem tương đương SUCCESS.
- Ký image bằng Cosign đã có bằng chứng trên Harbor nhưng vẫn phụ thuộc tham số pipeline và chưa phải cổng admission bắt buộc.
- Kyverno hiện kiểm tra manifest trong CI; cụm chưa bắt buộc admission policy theo bằng chứng hiện có.
- NetworkPolicy chưa giới hạn egress theo từng dependency.
- Loki chưa có persistence và cấu hình Grafana cần được đưa hoàn toàn sang secret manager.
- Backup PostgreSQL và MongoDB đã được cấu hình nhưng cần diễn tập restore và ghi nhận RPO/RTO.
- Harbor đã lưu SBOM cho artifact, nhưng provenance chưa được chuẩn hóa theo SLSA/in-toto.

Những giới hạn trên xác định ranh giới giữa môi trường đồ án có kiểm soát và hệ thống production. Các bước gia cố tiếp theo gồm remote Terraform state có locking, private node group, admission control, provenance chuẩn hóa, bắt buộc xác minh chữ ký image, NetworkPolicy theo nguyên tắc đặc quyền tối thiểu, cảnh báo vận hành và kiểm thử phục hồi định kỳ.

Chương 7

Kiểm thử, đánh giá và bằng chứng

7.1 Chiến lược kiểm thử

Kiểm thử DocVault được tổ chức theo nhiều lớp. Ở lớp code, các service backend có unit test cho policy, controller, security guard và các helper quan trọng. Frontend có test cho permission helper, form, component và một số behavior UI. Ở lớp runtime, script E2E kiểm tra các luồng nghiệp vụ và bảo mật qua API thật. Ở lớp pipeline, Jenkins chạy các bước test/scan/build/deploy/smoke/DAST để kiểm tra toàn bộ vòng đời phát hành.

Chiến lược này phù hợp với đặc thù của hệ thống bảo mật. Một unit test có thể kiểm tra một rule policy cụ thể, nhưng không chứng minh được toàn bộ luồng từ frontend đến gateway, metadata-service, document-service và audit-service. Ngược lại, E2E runtime có thể chứng minh luồng chính hoạt động, nhưng khó bao phủ mọi nhánh policy nhỏ. Vì vậy, hệ thống cần kết hợp nhiều loại kiểm thử.

7.2 Kiểm thử runtime

Theo docs/verification-report.md, lệnh E2E runtime chính là:

Listing 7.1: Lệnh kiểm thử E2E runtime

```
pnpm test:e2e
```

Kết quả gần nhất được ghi nhận trong tài liệu là **All required E2E checks passed**. Các nhóm kiểm thử runtime gồm:

- Auth boundary: request thiếu token và token hết hạn bị từ chối.
- Create policy: viewer không được tạo tài liệu, editor được tạo.
- Metadata policy: truy cập trái phép vào confidential document bị từ chối ở detail, history, comments và ACL.
- GROUP ACL: group `finance-team` được normalize và kiểm tra đúng.
- Confidential download posture: viewer bị deny, editor nhận stream-only response.

- Malware scan: EICAR upload bị chặn.
- DLP: upload có dữ liệu nhạy cảm bị phát hiện và nâng classification.
- Workflow: submit, approve và duplicate approve conflict.
- Retention: approval stamp retention, retention job archive due record.
- Compliance Officer: được phép xem metadata, audit, retention; bị chặn preview, presign, stream và download.
- Audit: query, verify-chain, security summary và audit ingest boundary.

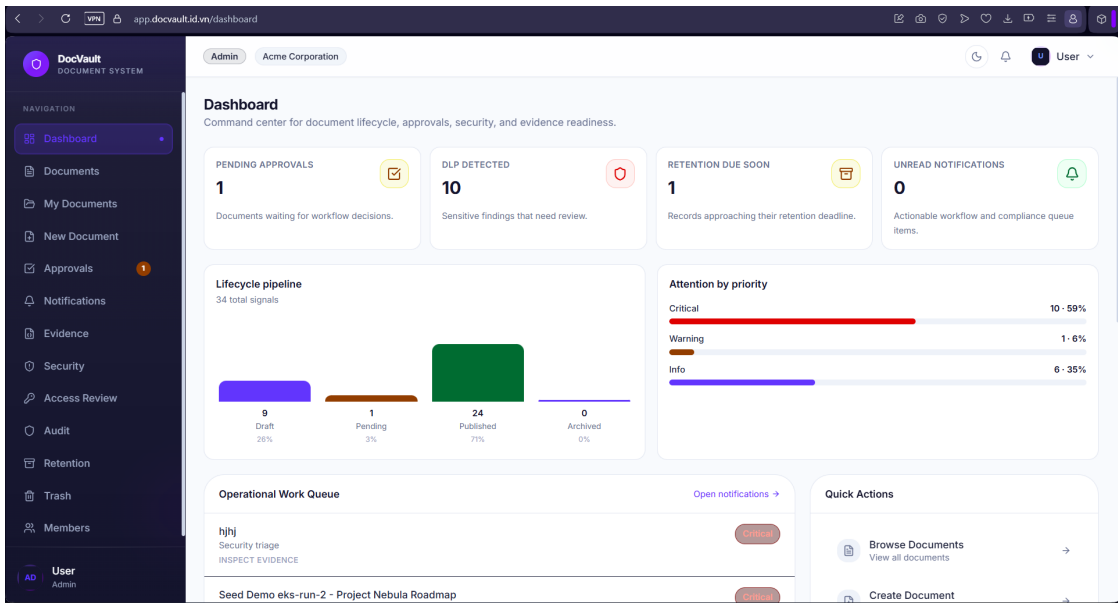
Bảng 7.1: Tóm tắt bằng chứng kiểm thử runtime

Khu vực	Bằng chứng	Kết quả
Authentication	Missing/expired token rejected	PASS
Authorization	Viewer create denied, editor create allowed	PASS
Metadata policy	Viewer guessed confidential access denied	PASS
ACL	GROUP READ ACL metadata access	PASS
Download posture	Confidential presign withheld direct URL	PASS
Malware	EICAR upload blocked before object/version creation	PASS
DLP	Sensitive upload escalated classification	PASS
Workflow	Submit/approve/duplicate approve conflict	PASS
Retention	Auto-archive due record and workflow history	PASS
Compliance boundary	Compliance file content access denied	PASS
Audit	Verify-chain and security summary available	PASS
Audit ingest	Viewer cannot ingest fake audit event	PASS

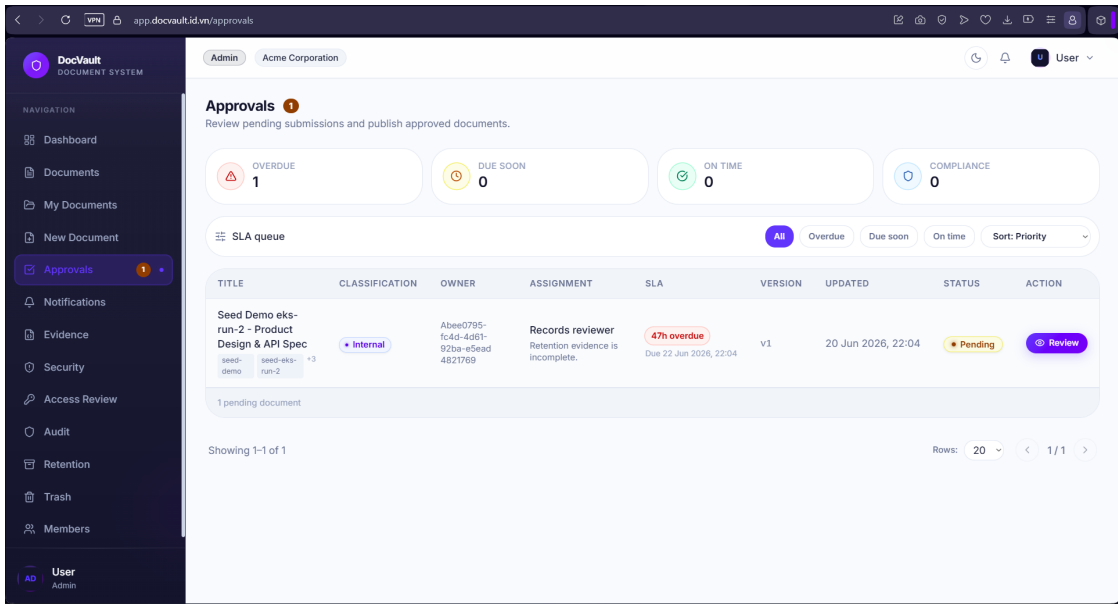
7.3 Kiểm thử frontend và UI evidence

Frontend được kiểm tra bằng lint, build, typecheck và test component/helper. Ngoài ra, bộ screenshot trong `report/images/screen` ghi nhận các màn hình trọng yếu như Dashboard, Documents, Approvals, Audit, Security và Access Review trên môi trường triển khai. Các screenshot này vừa minh họa UI, vừa cung cấp bằng chứng trực quan

rằng các màn hình chính render được và hiển thị trạng thái dữ liệu phù hợp.



Hình 7.1: Bảng chứng giao diện dashboard command center



Hình 7.2: Bảng chứng giao diện hàng đợi phê duyệt và trạng thái SLA

7.4 Security evidence

Tài liệu docs/web-security-evidence.md tổng hợp các kiểm soát bảo mật runtime. Các điểm nổi bật gồm:

- Compliance Officer bị chặn preview, stream, presign và download nội dung file.
- Metadata access boundary được áp dụng cho detail, workflow history, comments và ACL list.

- Frontend permission helper được align với backend policy ở mức UI hint.
- Grant token cho download/preview có TTL 300 giây và hỗ trợ rotation bằng `kid`.
- MinIO bucket local bật SSE-S3 trong init script.
- Malware scanning chặn EICAR trước khi ghi object.
- DLP phát hiện pattern nhạy cảm và nâng classification.
- Audit ingest cần service token.
- Audit verify-chain và security summary được expose qua gateway.
- Evidence packet không chứa nội dung file, presigned URL, grant token hoặc object key.

7.5 Pipeline evidence

Bảng chứng pipeline gồm log, báo cáo và trạng thái thực thi từ Jenkins, Harbor, Git, Argo CD và EKS. Tài liệu kiểm chứng trước đây ghi nhận build #61 đã hoàn thành các stage chính. Tuy nhiên, pipeline hiện tại đã được mở rộng thêm TruffleHog, phát hiện phạm vi thay đổi, lint và build toàn workspace, SonarQube Quality Gate, Kyverno policy-as-code, cố định digest trên Harbor và ký artifact bằng Cosign. Vì vậy, bộ bằng chứng Jenkins cuối cùng cần được thu thập từ một build mới chạy đúng phiên bản `Jenkinsfile` hiện tại.

Các bằng chứng được sử dụng để đối chiếu kết quả triển khai gồm:

- Screenshot Jenkins build xanh và stage view.
- TruffleHog secret scan, lint, unit test và workspace build.
- SonarQube report hoặc quality gate.
- Dependency Check report.
- Trivy filesystem/image scan log.
- Checkov, Terraform Validate và Kyverno report.
- Harbor image tag theo Git SHA, digest, SBOM, trạng thái ký và kết quả quét lỗ hổng.
- Log ký và xác minh Cosign tương ứng với image digest.
- GitOps commit cập nhật repository, tag và digest trong Helm values.
- Argo CD application `Synced/Healthy`.
- Smoke test log cho web root và `/api/health`.
- ZAP HTML/JSON report.
- Grafana/Loki screenshot cho observability.
- AWS EKS, S3, KMS, IAM/IRSA, Secrets Manager và CloudNativePG backup.

Các ảnh EKS, IAM/IRSA, S3/KMS, Secrets Manager, Argo CD và Harbor đã được

đưa vào Chương 6. Các placeholder còn lại được giữ cho bằng chứng Jenkins, SonarQube, SCA/Trivy, IaC, kiểm tra sau triển khai, Grafana, Loki và bucket backup.

7.6 Đánh giá theo yêu cầu

Bảng 7.2: Mapping yêu cầu với hiện thực và bằng chứng

Yêu cầu	Hiện thực	Bằng chứng
Đăng nhập và phân quyền	Keycloak, JWT, gateway role guard, frontend permission helper	README, API contract, E2E auth checks
Quản lý vòng đời tài liệu	Documents, versions, workflow history, workflow-service transitions	Project status, E2E workflow checks
Kiểm soát download	Metadata authorization, grant token, document-service pre-sign/stream	web-security-evidence, E2E confidential posture
Compliance boundary	Compliance meta-data/audit/evidence allowed, file content denied	web-security-evidence, E2E compliance checks
Audit tamper-evident	audit-service hash-chain, verify-chain endpoint	verification report, audit docs
Malware/DLP	Malware scanner, DLP scanner, classification escalation	E2E malware/DLP checks
Retention evidence	Retention fields, retention run, audit event	E2E retention checks
DevSecOps pipeline	Jenkins gates, Harbor/digest, Kyverno, GitOps, Argo CD, smoke, ZAP	Jenkinsfile, shared library, manifest hạ tầng

7.7 Hạn chế hiện tại

Kết quả kiểm thử và đối chiếu cấu hình cho thấy hệ thống vẫn còn các giới hạn sau:

- Notification-service hiện vẫn sử dụng cơ chế lưu nhận phục vụ phát triển, chưa tích hợp đầy đủ email, webhook hoặc push notification.

- Kiểm thử end-to-end ở cấp trình duyệt chưa phải bằng chứng chính cho toàn bộ luồng nghiệp vụ; phạm vi kiểm chứng hiện tập trung vào API và runtime.
- Thư viện xác thực và contract dùng chung cần tiếp tục được hợp nhất để giảm trùng lặp giữa các dịch vụ.
- Harbor đã thể hiện SBOM và trạng thái ký của artifact; tuy nhiên provenance theo chuẩn SLSA/in-toto và chính sách bắt buộc xác minh chữ ký tại admission time chưa được triển khai.
- Kyverno đã được sử dụng như công policy-as-code trong CI với manifest đã render; cụm EKS chưa có bằng chứng về admission controller chặn trực tiếp tài nguyên vi phạm.
- NetworkPolicy vẫn mở egress, Loki chưa có persistence và cấu hình quản trị monitoring cần được chuyển hoàn toàn sang External Secrets.
- Backup PostgreSQL (CloudNativePG) và audit MongoDB (Percona PBM, có PITR qua oplog) đã được cấu hình trên S3/KMS và chạy theo lịch; quy trình khôi phục vẫn cần được diễn tập và đo lường để xác minh RPO/RTO.

7.8 Nhận xét chung

DocVault đã đáp ứng các mục tiêu cốt lõi của một hệ thống quản lý tài liệu bảo mật trong phạm vi đồ án chuyên ngành. Kiến trúc, luồng nghiệp vụ, kiểm soát bảo mật backend, audit, evidence và pipeline DevSecOps đều có hiện thực tương ứng trong repository. Các kết luận trong báo cáo được đối chiếu với mã nguồn, cấu hình hạ tầng, báo cáo kiểm thử và ảnh chụp từ môi trường triển khai.

Chương 8

Kết luận và hướng phát triển

8.1 Kết quả đạt được

Đề tài đã xây dựng và triển khai được hệ thống DocVault theo hướng quản lý tài liệu bảo mật gắn với DevSecOps. Phần ứng dụng gồm frontend Next.js, một API Gateway và các microservice NestJS phụ trách metadata, nội dung tài liệu, workflow, audit và notification. Các chức năng đã chạy được trải từ tạo tài liệu, quản lý phiên bản, gửi duyệt, phê duyệt, preview/download, audit, security dashboard, evidence center đến retention.

Điểm nhóm chú trọng nhất là đặt kiểm soát bảo mật runtime ở backend chứ không dừng ở giao diện. Quyết định cho phép hay từ chối một thao tác được tổng hợp từ RBAC, ACL, trạng thái tài liệu, mức phân loại và quyền sở hữu, thay vì chỉ dựa vào vai trò. Compliance Officer chỉ chạm được metadata, audit và evidence; nội dung file được bảo vệ bằng grant token ngắn hạn và đường stream có kiểm soát. Audit-service dùng hash-chain để chuỗi sự kiện trở nên tamper-evident; malware scanner chặn mẫu EICAR ngay trước khi ghi object; DLP phát hiện dữ liệu nhạy cảm và nâng mức phân loại; evidence packet được lọc để không bao giờ chứa nội dung file hay thông tin xác thực.

Ở lớp hạ tầng, toàn bộ được mô tả bằng Terraform và chạy trên Amazon EKS, với workload đóng gói bằng Helm và trạng thái triển khai đồng bộ theo GitOps qua Argo CD. Phần nhóm thấy đáng giá nhất ở lớp này là cách dữ liệu và bí mật được giữ: file nằm trên S3 mã hóa bằng SSE-KMS, quyền truy cập AWS của workload bị bó hẹp bằng IRSA thay vì dùng key tĩnh, và secret được giữ trong AWS Secrets Manager rồi mới đồng bộ vào cụm qua External Secrets Operator. Các thành phần vận hành đi kèm như backup cơ sở dữ liệu, Harbor và bộ Prometheus/Grafana/Loki cũng đã được dựng.

Pipeline Jenkins là nơi gom phần lớn các cổng kiểm soát: ngoài kiểm thử, nó chạy các bước quét bảo mật (SAST, SCA, quét secret, filesystem và container image, IaC), kiểm tra policy-as-code bằng Kyverno CLI, rồi mới build và đẩy image lên Harbor trước khi cập nhật GitOps và để Argo CD đồng bộ. Một điểm nhóm cố ý làm cho chặt là truy vết: image được định danh bằng Git SHA và digest, Harbor giữ SBOM cùng trạng thái ký,

nên một workload trên cụm luôn lần ngược được về commit sinh ra nó. Nhờ vậy, ba nhóm kiểm soát đặt mục tiêu từ đầu, gồm bảo mật chuỗi cung ứng, policy-as-code và quản lý bí mật, đã thành hiện thực chứ không còn là đề xuất.

8.2 Mức độ đáp ứng mục tiêu

Đối chiếu với các mục tiêu ban đầu, đề tài đã đạt được các kết quả sau:

- Xây dựng kiến trúc microservices với ranh giới trách nhiệm tương đối rõ giữa các dịch vụ.
- Hoàn thiện các luồng nghiệp vụ chính của hệ thống quản lý tài liệu và áp dụng kiểm soát truy cập ở backend.
- Cung cấp bằng chứng kiểm thử cho xác thực, RBAC/ACL, truy cập tài liệu nhạy cảm, malware, DLP, retention, audit và ranh giới quyền của Compliance Officer.
- Triển khai hạ tầng cloud-native trên EKS bằng Terraform, Helm, Kustomize và GitOps.
- Tích hợp các cổng chất lượng và bảo mật vào pipeline, bao gồm kiểm tra mã nguồn, dependency, secret, IaC, policy Kubernetes và container artifact.
- Áp dụng S3/KMS, IRSA, External Secrets, SBOM, ký artifact, backup và observability trong môi trường triển khai.

8.3 Hạn chế

Mặc dù các mục tiêu cốt lõi đã được đáp ứng, hệ thống vẫn còn một số giới hạn cần ghi nhận. Notification-service hiện chủ yếu đóng vai trò lưu nhận thông báo trong môi trường phát triển, chưa tích hợp đầy đủ email, webhook hoặc push notification. Kiểm thử end-to-end ở cấp trình duyệt chưa bao phủ toàn bộ luồng nghiệp vụ; bằng chứng hiện tại nghiêng nhiều về API và runtime.

Ở lớp hạ tầng, môi trường EKS đang ưu tiên chi phí kiểm thử nên chưa phản ánh đầy đủ mô hình production với node private, giới hạn chặt Kubernetes API và remote Terraform state có cơ chế khóa. NetworkPolicy chưa giới hạn egress theo từng dependency; Loki chưa bật persistence; một phần cấu hình quản trị monitoring cần tiếp tục được đưa vào External Secrets. Backup đã được cấu hình, nhưng chưa có kết quả diễn tập khôi phục để lượng hóa RPO và RTO.

Đối với chuỗi cung ứng phần mềm, SBOM và chữ ký artifact đã có bằng chứng trên Harbor, nhưng provenance chưa được chuẩn hóa theo SLSA/in-toto. Kyverno hiện được sử dụng trong CI, chưa được chứng minh là admission controller trên cụm. Vì vậy, thay đổi được tạo ngoài pipeline chưa chịu cùng mức thực thi, và chữ ký image chưa phải điều kiện bắt buộc trước khi Kubernetes tiếp nhận workload.

8.4 Hướng phát triển

Các hướng phát triển tiếp theo tập trung vào nâng mức bảo đảm vận hành và khả năng kiểm chứng của hệ thống:

1. **Chuẩn hóa provenance và phê duyệt GitOps:** tạo attestation theo SLSA/intoto, liên kết commit, SBOM, kết quả quét và image digest; chuyển thay đổi GitOps quan trọng sang pull request có phê duyệt.
2. **Enforcement tại admission và phân đoạn mạng:** triển khai Kyverno admission controller, bắt buộc xác minh chữ ký/attestation và giới hạn NetworkPolicy egress theo đúng dependency của từng workload.
3. **Tăng cường khả năng phục hồi:** tổ chức diễn tập restore PostgreSQL và MongoDB, kiểm chứng tính toàn vẹn backup, xác định RPO/RTO và xây dựng quy trình disaster recovery.
4. **Gia cố hạ tầng production:** sử dụng private node group, giới hạn endpoint EKS, remote Terraform state có locking, tách môi trường và bổ sung cơ chế phê duyệt thay đổi hạ tầng.
5. **Hoàn thiện notification và xử lý bất đồng bộ:** tích hợp email, webhook hoặc push notification; sử dụng message queue/event bus cho workflow, audit và notification.
6. **Mở rộng kiểm thử trình duyệt:** bổ sung Playwright hoặc Cypress cho các luồng đăng nhập, tạo tài liệu, phê duyệt, tải xuống và kiểm tra quyền truy cập.
7. **Nâng cao observability:** xây dựng dashboard nghiệp vụ và bảo mật, alert rule, log retention, SLO/SLI và kênh xử lý cảnh báo.
8. **Chuẩn hóa bộ bằng chứng tuân thủ:** tự động xuất evidence theo kỳ, theo tài liệu, theo bản phát hành và theo sự cố; bổ sung kiểm tra tính toàn vẹn của gói bằng chứng.
9. **Phát triển chức năng AI có kiểm soát:** triển khai tóm tắt và hỏi đáp tài liệu theo nguyên tắc metadata-only, kiểm tra quyền nội dung và lưu audit đầy đủ cho mọi truy vấn.

8.5 Kết luận chung

Nhìn lại, phần khó nhất của đề tài không nằm ở việc dựng từng thành phần riêng lẻ mà ở chỗ giữ cho quyết định bảo mật nhất quán khi một thao tác đi qua nhiều service. Một lần tải file chạm tới metadata-service, document-service và audit-service, nên chính sách phải khớp ở cả ba nơi chứ không thể chỉ đặt ở một điểm. Ở phía ứng dụng web, việc đồng bộ permission helper của frontend với policy thật ở backend cũng tốn nhiều công: phải đảm bảo những gì giao diện ẩn đi đúng bằng những gì backend từ chối, tránh trường hợp

giao diện hiển thị một chính sách nhưng API lại cho phép một đường truy cập khác.

Phía vận hành và pipeline thì khó theo kiểu khác. Nhóm làm trong điều kiện tài nguyên và chi phí cloud hạn chế, nên môi trường EKS phải tiết chế so với một mô hình production đầy đủ. Pipeline chạy lâu, đôi lúc công cụ quét bị treo làm mất thời gian, và việc xử lý các CVE phát hiện trong quá trình quét chiếm một phần đáng kể công sức trước khi pipeline qua được. Quá trình đưa ứng dụng lên Kubernetes phát sinh nhiều vấn đề hơn so với môi trường local: nhiều lỗi chỉ lộ ra trên cụm, điển hình là service đọc sai biến môi trường nên phải dò và cập nhật lại cấu hình, và việc giữ secret trong AWS rồi trở vào cụm qua External Secrets Operator cần nhiều thời gian để cấu hình ổn định. Nhóm cũng phải chấp nhận vài đánh đổi: ưu tiên chiều rộng (đủ luồng nghiệp vụ và đủ cổng DevSecOps để minh họa quy trình) hơn là gia cố sâu một điểm cho mức production, và để bằng chứng nghiêng về API/runtime thay vì E2E trình duyệt vì runtime cho tín hiệu chắc chắn hơn trong phạm vi đề án. Một bài học rút ra là cần tiếp tục tìm hiểu thêm các công cụ bảo mật khác để bổ sung và cải tiến hệ thống, thay vì coi bộ công cụ hiện tại là đã đủ.

Kết quả cho thấy DocVault vừa làm được việc lưu trữ, quản lý tài liệu, vừa đưa bảo mật, khả năng truy vết và bằng chứng tuân thủ vào ngay trong kiến trúc ứng dụng lẫn quy trình phát hành. Việc gộp microservices, EKS, GitOps, kiểm soát chuỗi cung ứng và các cổng DevSecOps tạo ra một đường đi có thể kiểm chứng từ mã nguồn đến workload. Những hạn chế còn lại chủ yếu thuộc giai đoạn gia cố production và chuẩn hóa vận hành, không làm thay đổi việc các mục tiêu kỹ thuật cốt lõi của đề tài đã hoàn thành.

Phụ lục A

Hướng dẫn chạy hệ thống

A.1 Yêu cầu môi trường

Môi trường phát triển DocVault cần các công cụ sau:

- Node.js 20 trở lên.
- pnpm 9 trở lên.
- Docker Desktop hoặc Docker Engine kèm Docker Compose.
- Git.
- Các port local trống cho gateway, microservices, frontend, Keycloak, PostgreSQL, MongoDB và MinIO.

A.2 Cài đặt phụ thuộc

Từ thư mục gốc repository, chạy:

```
pnpm install
```

A.3 Khởi động hạ tầng local

Hạ tầng local nằm trong `infra/docker-compose.dev.yml`. Lệnh chạy:

```
docker compose -f infra/docker-compose.dev.yml --env-file infra/.env up -d
```

Các thành phần được khởi động gồm PostgreSQL, MongoDB, MongoDB Express, MinIO, MinIO init job và Keycloak.

A.4 Chạy migration

Sau khi PostgreSQL đã sẵn sàng:

```
pnpm --filter metadata-service prisma:deploy
pnpm --filter audit-service prisma:deploy
```

A.5 Khởi động backend

Khuyến nghị chạy từng service ở terminal riêng theo thứ tự:

```
pnpm --filter metadata-service start:dev
pnpm --filter audit-service start:dev
pnpm --filter document-service start:dev
pnpm --filter notification-service start:dev
pnpm --filter workflow-service start:dev
pnpm --filter gateway start:dev
```

Các service mặc định:

- Gateway: <http://localhost:3000>
- Metadata-service: <http://localhost:3001>
- Document-service: <http://localhost:3002>
- Workflow-service: <http://localhost:3003>
- Audit-service: <http://localhost:3004>
- Notification-service: <http://localhost:3005>

A.6 Khởi động frontend

Frontend chạy trên port 3006 theo script dev của apps/web:

```
pnpm --filter web dev
```

Truy cập:

```
http://localhost:3006
```

A.7 Tài khoản demo

Các tài khoản demo thường dùng:

- viewer1 (vai trò viewer)
- editor1 (vai trò editor)
- approver1 (vai trò approver)
- co1 (vai trò compliance_officer)

- `admin1` (vai trò `admin`)

Mật khẩu demo nằm trong tài liệu hướng dẫn chạy nội bộ của repository và cần được thay đổi nếu triển khai ngoài môi trường dev.

A.8 Kiểm thử nhanh

Sau khi backend hoạt động:

```
pnpm test:e2e
```

Lệnh này kiểm tra các luồng chính: auth boundary, deny viewer, create/upload/submit của editor, approve của approver, download published document, audit của compliance và deny download của compliance.

Phụ lục B

Tóm tắt API contract

B.1 Nguyên tắc chung

Frontend gọi toàn bộ API thông qua gateway với base URL:

```
http://localhost:3000/api
```

Các request cần header:

```
Authorization: Bearer <keycloak_jwt>
```

Gateway xác thực JWT, kiểm tra role ở mức route và proxy request xuống service tương ứng. Các endpoint chi tiết tài liệu, workflow history, comments và ACL được policy-filter ở backend.

B.2 Nhóm endpoint chính

Bảng B.1: Tóm tắt endpoint chính

Chức năng	Endpoint	Vai trò chính
Login	GET /auth/login	Public
Current user	GET /auth/me	Authenticated
List documents	GET /metadata/documents	All roles
Create document	POST /metadata/documents	Editor, Admin
Document detail	GET /metadata/documents/:docId	Policy-filtered
Update metadata	PATCH /metadata/documents/:docId	Owner Editor, Admin
ACL list/upsert	GET/POST /metadata/documents/:docId/acl	Policy-filtered, Editor/Admin

Chức năng	Endpoint	Vai trò chính
Workflow history	GET /metadata/documents/ :docId/workflow-history	Policy-filtered
Upload file	POST /documents/:docId/upload	Editor, Admin
Authorize download	POST /metadata/documents/ :docId/download-authorize	Policy-filtered
Presign download	POST /documents/:docId/ presign-download	Policy-filtered
Stream file	GET /documents/:docId/ versions/:version/stream	Policy-filtered
Submit	POST /workflow/:docId/submit	Editor, Admin
Approve	POST /workflow/:docId/approve	Approver, Admin
Reject	POST /workflow/:docId/reject	Approver, Admin
Archive	POST /workflow/:docId/archive	Editor, Admin
Audit query	GET /audit/query	Compliance, Admin
Verify chain	GET /audit/verify-chain	Compliance, Admin
Security summary	GET /audit/security-summary	Compliance, Admin
Retention list	GET /metadata/retention/ documents	Compliance, Admin
Retention run	POST /metadata/retention/run	Admin
Evidence packet	GET /metadata/documents/ :docId/evidence-packet	Compliance, Admin

B.3 Compliance Officer rule

Compliance Officer được phép xem audit, retention, security summary và evidence packet. Tuy nhiên, vai trò này không được preview, stream, presign hoặc download file content. Quy tắc này là một trong các acceptance criteria quan trọng của đề tài.

B.4 Grant token

Download và preview grant token được cấp bởi metadata-service sau khi policy check thành công. Token có thời hạn ngắn và được document-service xác minh trước khi trả nội dung. Thiết kế này giúp tách bước quyết định quyền khỏi bước phát file, đồng thời tránh để frontend tự quyết định quyền.

Tài liệu tham khảo

- [1] National Institute of Standards and Technology. “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities”, visited on June 14, 2026. address: <https://csrc.nist.gov/pubs/sp/800/218/final>
- [2] OWASP Foundation. “OWASP DevSecOps Guideline”, visited on June 14, 2026. address: <https://owasp.org/www-project-devsecops-guideline/>
- [3] Keycloak. “Keycloak Documentation”, visited on June 14, 2026. address: <https://www.keycloak.org/documentation>
- [4] Argo CD Project. “Argo CD: Declarative GitOps CD for Kubernetes”, visited on June 14, 2026. address: <https://argo-cd.readthedocs.io/en/stable/>
- [5] Microsoft. “Threat Modeling: STRIDE Model”, visited on June 25, 2026. address: <https://learn.microsoft.com/en-us/security/engineering/threat-modeling-tool-threats>
- [6] HashiCorp. “Backend Block Configuration Overview”, visited on June 25, 2026. address: <https://developer.hashicorp.com/terraform/language/backend>
- [7] Amazon Web Services. “IAM Roles for Service Accounts”, visited on June 25, 2026. address: <https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html>
- [8] Amazon Web Services. “Using Server-Side Encryption with AWS KMS Keys”, visited on June 25, 2026. address: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingKMSEncryption.html>
- [9] CloudNativePG Project. “Backup and Recovery”, visited on June 25, 2026. address: <https://cloudnative-pg.io/documentation/current/backup/>
- [10] Percona. “Percona Backup for MongoDB Documentation”, visited on June 25, 2026. address: <https://docs.percona.com/percona-backup-mongodb/index.html>
- [11] Kubernetes Authors. “Network Policies”, visited on June 25, 2026. address: <https://kubernetes.io/docs/concepts/services-networking/network-policies/>

- [12] External Secrets Operator Project. “AWS Secrets Manager Provider”, visited on June 25, 2026. address: <https://external-secrets.io/latest/provider/aws-secrets-manager/>
- [13] Harbor Project. “Working with Images and Tags”, visited on June 25, 2026. address: <https://goharbor.io/docs/main/working-with-projects/working-with-images/>
- [14] Sigstore Project. “Signing Containers with Cosign”, visited on June 25, 2026. address: https://docs.sigstore.dev/cosign/signing/signing_with_containers/
- [15] Kyverno Project. “Kyverno CLI Reference”, visited on June 25, 2026. address: <https://kyverno.io/docs/kyverno-cli/>
- [16] SonarSource. “SonarQube Server Documentation”, visited on June 14, 2026. address: <https://docs.sonarsource.com/sonarqube-server>
- [17] Aqua Security. “Trivy Documentation”, visited on June 14, 2026. address: <https://trivy.dev/docs/latest/guide/>
- [18] Checkov Project. “Checkov Documentation”, visited on June 14, 2026. address: <https://www.checkov.io/1.Welcome/What%20is%20Checkov.html>
- [19] ZAP by Checkmarx. “ZAP Baseline Scan”, visited on June 14, 2026. address: <https://www.zaproxy.org/docs/docker/baseline-scan/>
- [20] Prometheus Authors. “Prometheus Overview”, visited on June 25, 2026. address: <https://prometheus.io/docs/introduction/overview/>
- [21] Grafana Labs. “Grafana Loki Overview”, visited on June 25, 2026. address: <https://grafana.com/docs/loki/latest/get-started/overview/>